

UNIT STUDY NOTES

The Data Analysis Process

Analysis is a process, not a library call. This unit covers acquiring data, assessing and cleaning it, running exploratory analysis, and the ETL habits that make results defensible and repeatable.

 9 source notebooks 277 topic sections extracted 7 study sections

Unit Overview

Topic: 09 Data Analysis Process · **Level:** ● Intermediate · **Type:** SESSION

Where every analysis really begins

No analysis survives contact with bad data-gathering. Before a single chart or model, the data must first travel *into* Python, and that voyage decides everything after it: column names, data types, missing markers and encodings are all fixed at read time. This session is the boot camp for that skill — reading data from CSV files, from remote URLs, from Excel, JSON and text files, from SQL databases, from public APIs, and finally by scraping an HTML page. By the end you will not just call `pd.read_csv`; you will know, for every parameter and every source, exactly what happens under the hood and which knobs to turn when the data misbehaves.

How the session is organised

The heart of the notebook is a **parameter-by-parameter tour of** `pd.read_csv` — `sep`, `index_col`, `header`, `usecols`, `squeeze`, `skiprows` / `nrows`, `encoding`, `on_bad_lines`, `dtype`, `parse_dates`, `converters`, `na_values`, and reading in chunks. Each tiny titled cell exists to give you one parameter, one snippet, one clear *when would I ever use this* — the format is deliberately repetitious so the parameters stick. From there the session widens the funnel:

1. **More files** — Excel (`read_excel`), raw text, JSON (`read_json`).
2. **Databases** — connecting to MySQL and pulling tables straight into a DataFrame.
3. **The internet** — REST APIs with `requests`, then static HTML scraping with BeautifulSoup.
4. **Browsers** — Selenium locators and waits for dynamic pages.
5. **Export** — and the familiar endgame of writing tidy results to `csv` / `json`.

Read with suspicion

Data comes in with assumptions baked in — a missing cell that is actually the string `NA`, a column of numbers with a stray rupee sign, a date read as text. The single habit this session most wants to install is: **never trust a read until `info()`, `head()` and `dtypes` confirm it**. The parameters you study here are precisely the tools that rebuild your control over incoming data.

Topic: 09 Data Analysis Process · **Level:** ● Intermediate · **Type:** SESSION

The Sherlock half of data analysis

The best analyst in the world cannot fix problems they never saw. That is why this session is framed as detective work: the data is the crime scene, and the first duty is **assessment** — looking hard enough to actually see the imperfections before touching a single value. Only then do we move to **cleaning** — the repairs that make the data trustworthy enough to analyse. All of this happens on a genuinely messy dataset: the **clinical trials** files (`patients` , `treatments` , `adverse_reactions` , `treatments_cut`), which arrive dirty on purpose so every defect in the textbook has a live example.

Session structure

The notebook moves through the full cleaning lifecycle, and each stage is a distinct skill:

1. **Load and orient** — import the four files, eye-ball the frames, and record their shapes.
2. **Assess** — a systematic audit against the two classic categories: **quality issues** (missing, duplicate, invalid and inaccurate values) and **tidiness issues** (a value living in a column name, rows where they should be columns).
3. **Clean** — change only what the assessment justified: dropping empties, replacing invalid values, splitting columns, reshaping, and standardising categorical labels.
4. **Define and document** — before-and-after summaries, so that every transformation is traceable to a defect you actually observed.
5. **Iterate** — scrub, re-checks, and the honest admission that good cleaning loops until the audit comes back clean.

Why this discipline matters more than any function

Dropped rows and renamed columns are easy; *knowing which rows to drop and why* is the hard, valuable part. The professors, product-owners and interviewers who judge your work are all asking the same foundational question: **can you be trusted with imperfect data?** This session is where you build the answer — assessment before action, a defined list, and a dataset that tells the story of how it became clean.

Topic: 09 Data Analysis Process · **Level:** ● Intermediate · **Type:** SESSION

What this session is, and is not

This is EDA performed with the *lights on*: a full step-by-step tour of the Titanic dataset where every phase — univariate, bivariate, feature engineering, correlation — is executed cell by cell and then summarised in its own deep-dive. It is deliberately more concrete than the lecture-style session: here the code is the lecture. You will run each chunk, watch the output appear, and read the reasoning that tells you what the numbers mean.

The flow you can expect

1. **Load and first glance** — read the train file, `head()` , `describe()` the columns you will spend the day with.
2. **Univariate analysis** — dissect `Age` and `Fare` , then the categoricals (`Sex` , `Embarked` , `Pclass`) with counts and proportions.
3. **Bivariate analysis** — `Survived` against class, sex, age — the cross-tabulations that reveal the story.

4. **Feature engineering** — ticket groups, individual fare, family size and type, titles, deck — columns built to expose structure that raw columns hide.
5. **Multivariate and correlations** — heatmaps and pair plots that compress the whole dataset into one readable image.

The golden thread running through it: **every chart is a question, every conclusion a claim you can defend**. When you finish, you should be able to point at each plot and say what it proves — because that is exactly what an analyst is paid to do.

Two practical notes before you start. Everything here runs on the classic Titanic files (`train.csv`), and later (`test.csv`), typically inside a Colab runtime, so the code cells are near-finished — your job is to run them, read the numbers, and follow the deep-dive explanations. Keep a notebook open beside it for your own annotations: write the one-line insight a chart is showing *before* reading the notebook's conclusion, then compare. Predicting then checking is the fastest way to convert this tour from watched content into owned skill, and it makes the corner plots in the correlation phase land twice as hard.

Topic: 09 Data Analysis Process · **Level:** 🟡 Intermediate · **Type:** CASE STUDY

What you are about to do

This case study drops you into a genuinely messy **smartphones dataset** and lets you watch — cell by cell — how a professional cleans it. The notebook is structured as a single loop repeated until it is quiet: **assess** the data for quality and tidiness issues, **clean** the specific defects found, **restructure** columns that hide structure, and **export** a tidy file. The sheet lists over twenty quality issues and seven tidiness issues up front, so from the very first `info()` you know precisely which messes to hunt for.

The assessment-first mindset

The opening cells do the detective work: `df.info()` and `df.describe()` give the shape and the ranges, then a long bullet list names the offenders — model names written inconsistently (OPPO / oppo), prices stored as strings with a rupee symbol and commas, columns whose values leaked into their neighbours (card text in the camera column, OS text in the card column), and a long catalogue of specific row numbers where a value is shifted, missing, or wrong. The important trick to internalise: **the assessment comes before any cleaning code, and the cleaning code always names the defect it is answering**.

What to pay attention to

Watch the three deep moves the notebook uses again and again. First, **targeted repair over wholesale removal** — rows are dropped only when they are genuinely junk, and most broken rows are *fixed* by moving or re-parsing values. Second, **surgical pandas**: `df.copy()` keeps the original safe, `.str` accessors vectorise every text fix, `shift(1, axis=1)` realigns shifted columns in one call, and `value_counts()` after every repair proves the column is clean. Third, **typed, analysis-ready output** — every cleaned column ends as a sensible dtype (int, float, boolean) before the notebook exports `smartphone_cleaned_v2.csv`. By the end you should be able to say, for any cell, which defect it fixes and why that fix is safe.

Prerequisites

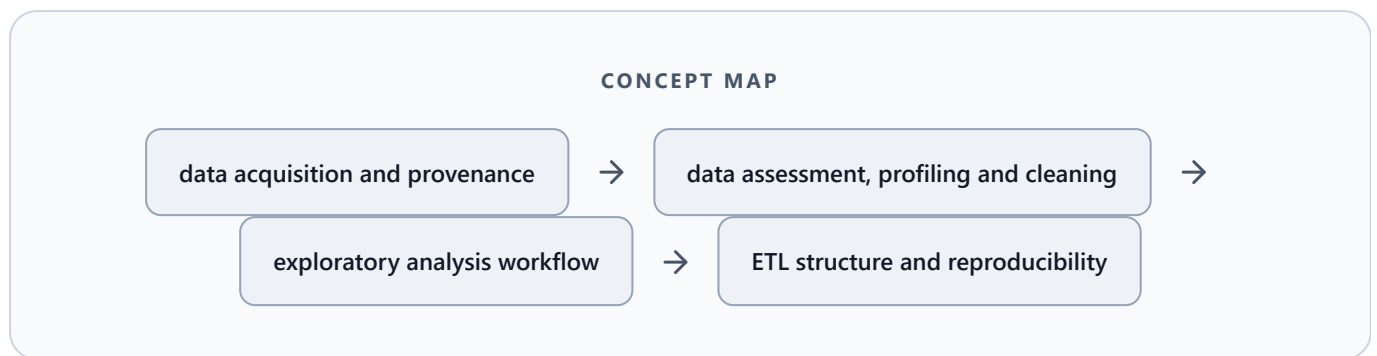
→ Pandas

→ Data Visualization

What you will be able to do

- ✓ Acquire data from files and sources with provenance in mind
- ✓ Profile a dataset for quality issues before analysing it
- ✓ Run exploratory analysis that answers a stated question
- ✓ Structure ETL steps so results are reproducible

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Data Gathering — Reading Data into Python

What is Data Analysis?

Data Analysis is the systematic process of **inspecting, cleaning, transforming, and modelling** data with the goal of **discovering useful information, drawing conclusions, and supporting decision-making**.

It is not just "making charts". A real analysis answers a **business or scientific question** and produces an **actionable recommendation**.

Why does it matter?

REASON	EXPLANATION
Evidence over opinion	Decisions based on data instead of guesswork
Find hidden patterns	Relationships that are invisible in raw data
Predict the future	Forecasting sales, churn, risk, demand
Measure performance	KPIs like revenue, profit, conversion, retention
Tell a story	Communicate findings so others act on them

Data → Information → Insight → Decision

PYTHON

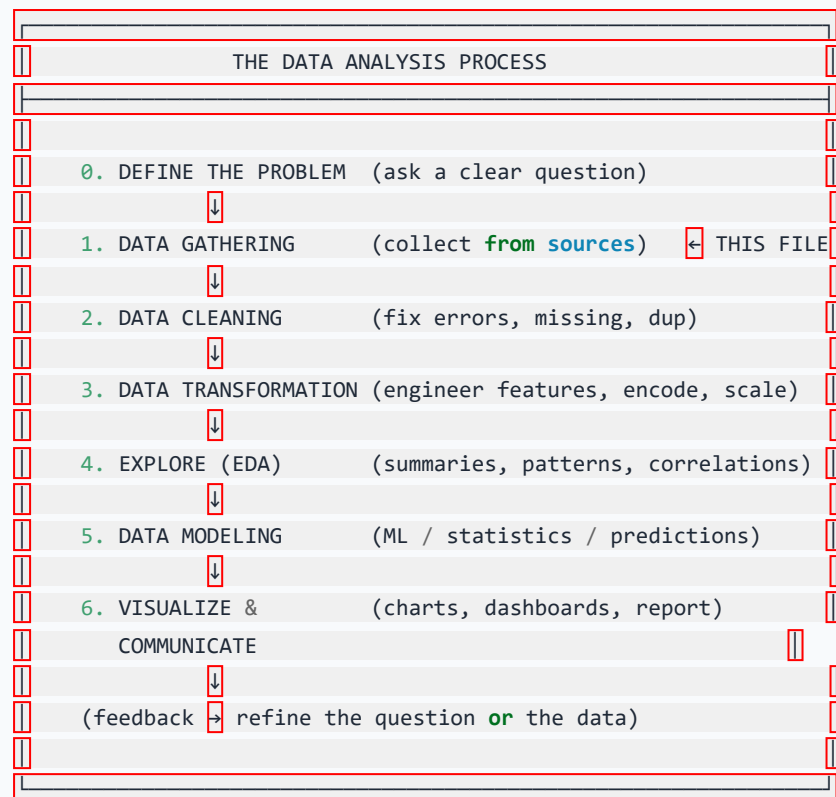
```
Raw Data (unstructured, messy numbers)
↓ (clean, organise)
Information (structured, meaningful values)
↓ (analyze, compare)
Insight (what it means, why it matters)
↓ (act)
Decision (business action / recommendation)
```

The Complete Data Analysis Process — Overview

Every data project, no matter how big or small, follows the same loop. The steps are **iterative** — you often jump back a step when you learn something new.

⚠ This is a **guide**, not a prison. In practice the steps overlap and loop constantly. But every project goes through these stages at least once.

PYTHON



Step 0 — Define the Problem (before touching any data)

The single most important step. **You cannot gather the right data if you do not know the question you are trying to answer.**

SMART questions

- ✗ "Analyze the sales data." (too vague)
- ✓ "Which product categories are losing money, and why?" (actionable)

What to define

- Objective** — what decision will this support?
- Key questions** — the specific questions to answer (e.g. "who are our most valuable customers?")
- Metrics / KPIs** — how success is measured (revenue, profit, churn...)
- Data needed** — which variables you will have to collect
- Stakeholders** — who will act on the result

Deliverable

A short **problem statement** with the questions you will answer. Everything downstream (what data to gather, what to clean, what to plot) is driven by this statement.

Step 1 — Data Gathering

Definition: The process of **collecting and importing** the raw data you need from their original sources into your analysis environment (Python / Pandas).

Why it is the foundation

- No data → no analysis.
- The **quality of your analysis can never exceed the quality of your data** (garbage in, garbage out).
- Different sources demand different tools, which is exactly what this notebook teaches.

The five main sources

SOURCE	TYPICAL FORMAT	MAIN PYTHON TOOL
Local / cloud files	CSV, Excel, JSON, TSV, text	<code>pd.read_csv()</code> , <code>pd.read_excel()</code> , <code>pd.read_json()</code>
Databases	SQL, NoSQL	<code>pd.read_sql()</code> , <code>mysql.connector</code> , <code>sqlalchemy</code>
Web APIs	JSON, XML over HTTP	<code>requests</code> + <code>pd.read_json()</code>
Web scraping (static)	HTML	<code>requests</code> + <code>BeautifulSoup</code>
Web scraping (dynamic / JS)	rendered HTML	<code>selenium</code>
Streaming	real-time events	Kafka, custom consumers

Key considerations before gathering

- **Where** is the data? (file, DB, API, website)
- **Format** is it in? (CSV, JSON, SQL...)
- **Size** — does it fit in memory, or must it be read in chunks?
- **Cleanliness** — what problems will you have to handle later?
- **Access** — authentication, rate limits, encoding?

Step 2 — Data Cleaning

Definition: Finding and fixing data-quality problems so your analysis is trustworthy.

Common problems you will fix

PROBLEM	EXAMPLE	TOOL
Missing values	<code>NaN</code> , blank cells, "N/A"	<code>dropna()</code> , <code>fillna()</code>
Duplicates	same customer twice	<code>drop_duplicates()</code>
Outliers	age = 300, salary = 1 crore	quantiles, IQR, domain rules
Wrong data types	numbers stored as text	<code>to_numeric()</code> , <code>astype()</code>
Inconsistent formatting	"new york", "New York", "NY"	string methods <code>.str.title()</code>
Impossible values	negative price, future birth date	validation rules

The cleaning mantra

Inspect first → **decide a rule** → **apply it** → **verify**. Always *look* at the data with `.info()`, `.isna().sum()`, `.duplicated().sum()` and `.describe()` **before** you change anything.

Step 3 — Data Transformation

Definition: Converting the clean data into a shape that is ready for analysis and modelling.

Typical transformations

1. **Feature engineering** — create new, useful columns
2. **Encoding categoricals** — text → numbers
 - Label encoding (`pd.factorize`), One-hot encoding (`pd.get_dummies`)
3. **Scaling / normalising** — bring features to a similar range
 - Min-Max, Z-score (Standardisation)
4. **Aggregation** — summarize groups
5. **Reshaping** — pivot / melt / stack
6. **Joining / merging** datasets together (`merge`, `join`, `concat`)

Deliverable: a tidy, analysis-ready table.

PYTHON

```
df["Revenue"] = df["Price"] * df["Quantity"]
df["AOV"] = df["Revenue"] / df["Orders"]
```

PYTHON

```
df.groupby("Category")["Revenue"].sum()
```

Step 4 — Data Exploration (EDA)

Exploratory Data Analysis is the process of **understanding** the data through summary statistics and visualisations **before** formal modelling.

What EDA answers

- What does the data *look* like? (shape, dtypes, head)
- What are the main statistics? (`.describe()` — mean, median, std, min, max)
- How are values distributed? (histograms, boxplots)
- What correlates with what? (correlation matrix, heatmap)
- Which groups behave differently? (groupby comparisons)
- Are there patterns or anomalies?

Typical EDA commands

EDA is where you *discover* the story; modelling is where you *formalise* it.

PYTHON

```
df.shape          # (rows, columns)
df.info()         # dtypes + non-null counts
df.describe()     # numeric stats
df.isna().sum()   # missing values
df.groupby("...").mean()
df.corr()         # correlation
```

Step 5 — Data Modeling

Definition: Using statistics or **Machine Learning** to find relationships and make predictions.

Two broad flavours

- **Supervised** — learn from labelled examples
 - *Classification* (spam / not spam, churn / not churn) → `LogisticRegression`, `RandomForest`
 - *Regression* (predict price, sales) → `LinearRegression`, `XGBoost`
- **Unsupervised** — find structure without labels
 - *Clustering* (customer segments) → `KMeans`
 - *Dimensionality reduction* → `PCA`

A typical modelling loop

1. Split data → train / validation / test
2. Train a model on the training set

3. Evaluate on validation (tune hyperparameters)
4. Evaluate once on the **test** set (unseen data)
5. Choose the best model

This notebook focuses on **gathering** the data. Modelling is covered in later, ML-focused notebooks — but the whole pipeline leads here.

Step 6 — Data Visualisation & Communication

Definition: Turning the results into charts, tables and narratives that **other people understand and act upon**.

Visualisation libraries

LIBRARY	BEST FOR
<code>matplotlib</code>	low-level control, static plots
<code>seaborn</code>	statistical plots, beautiful defaults
<code>plotly</code>	interactive charts
<code>pandas</code> <code>.plot()</code>	quick charts straight from DataFrames

Communication tools

- Reports & presentations
- Interactive dashboards (e.g. **Streamlit**, Power BI, Tableau)
- Charts embedded in notebooks

Good communication rules

- Choose the right chart for the message (line = trends, bar = compare, pie = share...)
- Label axes, add titles, keep it simple
- Lead with the **insight**, not the method
- Always end with a **recommendation**

The Whole Picture — Summary

#	STEP	MAIN QUESTION	MAIN OUTPUT
0	Define the problem	What do we need to know?	Problem statement
1	Gather data	Where is the data?	Raw datasets
2	Clean data	Is it trustworthy?	Clean datasets
3	Transform data	How to shape it?	Analysis-ready table
4	Explore (EDA)	What patterns exist?	Insights & hypotheses
5	Model	Can we predict it?	Trained model
6	Visualize & communicate	What should we do?	Report / dashboard

The rest of this notebook goes **deep into Step 1: Data Gathering**.

DATA GATHERING

Now we go step by step through everything you need to **bring data into Python**. Each section explains the **concept**, the **syntax**, and the **why** behind each parameter.

Tools of the Trade

Almost every gathering task uses the same handful of libraries:

pandas is the star. It turns almost any messy source into a clean **DataFrame**.

PYTHON

```
# Essentials
import pandas as pd      # DataFrame / Series, read & write everything
import numpy as np       # numerical arrays, missing values
import json              # parse JSON
from io import StringIO   # treat a string as a file

# Network / web
import requests          # make HTTP requests to APIs & pages

# Databases
import sqlalchemy        # connect to SQL databases (engine)
import mysql.connector    # MySQL-specific driver

# Scraping
from bs4 import BeautifulSoup # parse HTML
from selenium import webdriver # control a real browser
```

Reading a Local CSV File

CSV = Comma-Separated Values. It is the most common data format because it is simple, small, and understood by Excel, SQL, and pandas alike.

Basic syntax

PYTHON

```
df = pd.read_csv("path/to/file.csv")
```

1. Importing pandas

Before anything else, the working session declares its tools. `import pandas as pd` binds the whole pandas namespace to the nickname `pd` — short, and so universal that every pandas programme in the world reads the same way. The cell then loads NumPy as `np` alongside it, because the two libraries share their foundation: pandas' numeric behaviour builds directly on NumPy arrays, and having `np` available lets you use things like `np.nan` when you need an explicit missing marker. This first cell is not ceremony — every later read, table and chart in the session silently depends on it. Get into the habit that the first cell of any data notebook is always imports, always first, and always run before anything else. If you ever see a `NameError: name 'pd' is not defined`, the story of the cause is always the same: the import cell was skipped, reselected, or run out of order.

PYTHON

```
import pandas as pd
```

2. Opening a local CSV file

The most common object in all of data work is a plain CSV sitting on disk, and `pd.read_csv('file.csv')` is the one-liner that opens it. The result is a **DataFrame**: a labelled table of rows and columns with a built-in integer index. One line of pandas is enough to turn a comma-separated text file into something you can `head()`, `info()`, filter, group and plot.

The cell pairs the read with `df.head()` because a first look is mandatory: you want to see the column names, a sample of values, and any obvious garbage before doing anything else. Also notice `df.shape` — rows, then columns — used to confirm the read captured the whole file. Two habits belong to every read: look at the first rows immediately, and record the shape so you know the data's size before you transform it.

Keep the two-step reflex even when the file is tiny: data that looks right in the terminal is data that behaves right in the pipeline.

PYTHON

```
df = pd.read_csv('Datasets/aug_train.csv')
df
```

3. Opening a CSV file from a URL

Plain files are not the only source — datasets routinely live at a public URL, and pandas can read them directly. The classic trick shown here is the two-step: `requests.get(url)` downloads the file's content over HTTP, and `.text` gives that content as one long string; wrapping it in `io.StringIO(...)` hands pandas a *file-like object* it can parse exactly as if the file were local.

The indirection matters because it buys **control**: you can inspect the status code, add headers, or handle authentication before the bytes ever touch pandas — things a raw URL read cannot do. Once the DataFrame exists, everything is identical to a local file: a `.head()` to check, a `.shape` to size, and the analysis can begin. Remote data is just local data with a download step — and the download step is the part you now understand rather than trust.

PYTHON

```
import requests
from io import StringIO

url = "https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv"
headers = {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:66.0) Gecko/20100101 Firefox/66.0"}
req = requests.get(url, headers=headers)
data = StringIO(req.text)

pd.read_csv(data)
```

PYTHON

```
# Documentation of read_csv
pd.read_csv??
```

4. The `sep` parameter

CSV stands for comma-separated values, but hardly any real dataset uses only commas. `sep` tells pandas which character actually separates the columns: `sep='\t'` for tab-separated `.tsv` exports, `sep=';'` for European-style CSVs where commas are decimal points, and `sep='|'` for pipe-separated logs. If you ever see a whole column read into a single string, or a frame that has exactly one column, the first suspect is a wrong separator.

Two notes make this practical. `sep='\t'` is the escaped tab character, and whitespace-heavy files answer to `sep=r'\s+'` — a regex meaning any run of whitespace. And remember the reverse direction: `df.to_csv(path, sep=';')` writes with your chosen separator, so code that reads unusual files can also produce them. Know the separator, and a misparse becomes a two-second fix instead of a mystery.

One extra path: some exports smuggle quoted fields that contain the separator itself. If such a line misbehaves, that is exactly what the `quoting` and `escapechar` knobs exist for — recognise the symptom (one column, wrong length) and reach for the parameter, not a hack.

PYTHON

```
pd.read_csv('Datasets/movie_titles_metadata.tsv')
```

PYTHON

```
pd.read_csv('Datasets/movie_titles_metadata.tsv', sep='\t', names=['sno', 'name', 'release_year', 'rating', 'vot
```

5. The `index_col` parameter

A DataFrame always carries an index — the labels printed in the unshaded first column. By default pandas numbers the rows `0, 1, 2, ...`, but when the file already contains a natural key (an `id`, a `date`, an `order_no`), `index_col` promotes that column to become the index instead of leaving it an ordinary column and inventing an integer index.

The benefit is bookkeeping: look-ups like `df.loc[key]`, joins on the index, and time-series alignment all become cheaper and clearer when the index is meaningful. Pass `index_col=0` for the first column, `index_col='order_id'` for a named column, or a list like `index_col=[0, 1]` to build a hierarchical MultiIndex from several columns. Everyday work rarely needs it; the moment your data has a stable identifier, this parameter turns a label from data into structure.

One mental check keeps it honest: the index is not a column, so `reset_index()` is the door back. If a downstream step expects a plain column, call `df = df.reset_index()` to reclaim that key before you merge or export.

PYTHON

```
pd.read_csv('Datasets/aug_train.csv', index_col='enrollee_id')
```

6. The `header` parameter

When pandas reads a file, it uses the first row as the column names — unless you tell it otherwise.

`header=None` says there is no header row, generate integer column names instead, and `header=4` says skip the first four rows of preamble and use row five as the header. Both appear constantly in the wild, because real files begin with comment lines, title lines or blank rows before the actual table starts.

Use `header=None` for computer-to-computer exports that carry no names; use an explicit integer when the true header sits a few rows down. The symmetric writer counterpart is `df.to_csv(path, header=False)`. One extra tip: if the file has junk rows *after* the table, that is `skipfooter`'s role — but the moment you need `header=`, also check `comment='#'` to drop lines beginning with a marker. Read the file's shape twice, and the header parameter will never surprise you again.

PYTHON

```
pd.read_csv('Datasets/test (1).csv')
```

PYTHON

```
pd.read_csv('Datasets/test (1).csv', header=1)
```

7. The `usecols` parameter

Large files are rarely fully useful: you may need only a handful of columns and loading the rest wastes memory and clutters every `head()`. `usecols=['name', 'age', 'salary']` reads only the columns you list — named by label, or by position with `usecols=[0, 2, 4]`.

Do this at *read time*, not after the fact: reading a 200-column file and then dropping 190 columns still loads all 200 into memory, while `usecols` keeps the whole parse lean from the start. It also does double duty as documentation — the short list is a concise statement of which fields the analysis actually depends on. When a file is wide or when you loop over many files, making `usecols` part of every read is a habit that pays in speed and in clarity.

One symmetry worth noticing: `usecols` and `dtype` solve overlapping problems — the first drops columns you never wanted, the second pins the types of the ones you keep. Together they make a read both lean and honest, and both are decisions you can copy from one file to the next.

PYTHON

```
pd.read_csv('Datasets/aug_train.csv')
```

PYTHON

```
pd.read_csv('Datasets/aug_train.csv', usecols=['enrollee_id', 'gender', 'education_level'])
```

8. The `squeeze` parameter

A single-column file read by pandas becomes a one-column DataFrame — rows with a column. With `squeeze=True`, a file that resolves to exactly one column is returned as a **Series** instead: a 1-D labelled array that behaves like a vector, with `.iloc[i]`, `.mean()`, `.value_counts()` and plotting semantics that suit a bare sequence.

The catch is in the name: `squeeze` only activates when the result genuinely is one-column (or one-row); multi-column files return a normal DataFrame untouched. So this is a *shorthand*, not a coercion — use it when you genuinely want the data as a Series and find the extra column wrapper noise. If you need a Series regardless of shape, `df.iloc[:, 0]` is the deterministic alternative. Know both, keep them separate, and the squeeze surprise (the Series you expected, the DataFrame you got) stops happening.

When in doubt, verify the return with `type(df)` — a two-character habit that settles the Series-versus-DataFrame question instantly and forever.

PYTHON

```
pd.read_csv('Datasets/aug_train.csv', usecols=['gender'])
```

PYTHON

```
pd.read_csv('Datasets/aug_train.csv', usecols=['gender']).squeeze()
```

9. The `skiprows` / `nrows` parameters

Two knobs control *how much* of the file pandas sees. `nrows=1000` reads only the first 1,000 data rows — the standard move for a first exploration of a huge file: fast, and enough to see column structure, dtypes and sample values before committing to a full read. `skiprows=5` discards the first five lines entirely, which is how you skip banner comments and other preamble that would otherwise corrupt the header.

The two combine naturally: `skiprows=1, nrows=500` means drop the junk line, then peek at the next five hundred rows. The parameter stays honest about scale too — `nrows` does not sample randomly, it reads the *head* of the file, so use it for orienteering, and reach for actual sampling later when you need a representative slice. Reading a preview first is the cheapest data-quality decision there is.

One caution: `skiprows` skips before the header is detected, so count carefully when the first data line also holds column names — a miscounted skip yields a header that is really a row.

PYTHON

```
pd.read_csv('Datasets/aug_train.csv',nrows=100)
```

10. The `encoding` parameter

Text files store characters as bytes, and the mapping from bytes back to characters is the file's **encoding**. Modern files are usually UTF-8; older Windows exports often use latin-1/cp1252. When the bytes and the assumed encoding disagree you get `UnicodeDecodeError`, or worse, a file that reads clean but prints mojibake where accented letters should be. The fix is declaring the truth: `encoding='utf-8'`, `encoding='latin-1'`, or `encoding='utf-16'` for some exotic exports.

The robust workflow has three moves. First, try the read; second, when it fails, look at the *actual* bytes and match them; third, pin the correct encoding in the read so every later run is deterministic. Also remember the writer side: `pd.to_csv(path, encoding='utf-8')` and read it back with the same value. Encodings are invisible patience — they cost nothing when they work and cost an hour when they silently do not.

One practical shortcut: when the error message names a codec it cannot decode, the hint usually reveals the culprit — read the first raw bytes and match the encoding to them.

PYTHON

```
pd.read_csv('Datasets/zomato.csv')
```

PYTHON

```
pd.read_csv('Datasets/zomato.csv',encoding='latin-1')
```

11. The `on_bad_lines` parameter

No data file arrives perfect: a line with too few fields, a stray comma, a quote that never closes. In newer pandas these are handled deliberately via `on_bad_lines`: `'error'` (the default) raises and stops the read; `'warn'` prints a warning but continues past the offending line; `'skip'` quietly drops bad lines; and a callable lets you implement custom handling per malformed line.

The important shift is that a parse error is **information**: whichever mode you choose, the message should prompt a look at the raw file rather than a shrug. In practice, `'warn'` balances visibility with progress for exploratory reads, while rigorous pipelines end by keeping `'error'` so bad rows can never slip in silently. Pair it with `comment='#'` and deliberate `dtype` choices, and you have turned the least cooperative text files into something pandas can be trusted with.

One naming note: very old pandas called this `error_bad_lines`; if a Stack Overflow answer you copy uses that name, you are reading pre-1.3 material and should translate the modern spelling.

PYTHON

```
pd.read_csv('Datasets/BX-Books.csv', sep=';', encoding="latin-1", on_bad_lines='skip')
```

12. The `dtype` parameter

pandas infers a column's type from its contents — and that inference is wrong disturbingly often. Mixed or dirty values push numeric columns into `object`; padded IDs like `0012` get their leading zeros stripped; downstream `.astype()` calls are then a correction cascade. `dtype={'id': str, 'price': 'float32'}` short-circuits all of it by declaring the type at read time.

The parameter earns its keep three ways: as **memory control** (`'float32'` instead of `'float64'`, or explicit `'category'`), as **correctness** (keeping a real identifier a string), and as **stability** (the file's columns keep their intended types regardless of which rows exist today). The honest discipline is to read the file, inspect `df.dtypes`, and then decide which columns the inference *should not* be trusted with — those get pinned in `dtype`. A column whose type you declare is a column you have understood.

One follow-up instinct: after a `dtype`-pinned read, run `df.dtypes` anyway and confirm the declaration took — the file may quietly disagree, and the check keeps the two in sync.

PYTHON

```
pd.read_csv('Datasets/aug_train.csv').info()
```

PYTHON

```
pd.read_csv('Datasets/aug_train.csv', dtype={'target': int}).info()
```

13. Handling dates

Dates arrive as strings, and strings cannot be sorted, filtered by day, or plotted on a proper time axis.

`parse_dates=['date']` (or `parse_dates=['a', 'b']` for several) tells pandas to convert the listed columns to `datetime64` at read time. The payoff is immediate: `.dt.year`, `.dt.month`, `.dt.day` accessors, year-over-year compares, resampling, and time-aware plotting all become available.

Two refinements matter. When the date is split into separate year/month/day columns, pass a list of lists: `parse_dates=[['year', 'month', 'day']]` and pandas combines them while reading. And when the string format is unusual, `date_format` or a post-hoc `pd.to_datetime(col, format=...)` gives you full control. A column formally typed as a date is the difference between a chart that knows trends and a table that only thinks it does.

One habit to install now: make the format explicit whenever the values are at all unusual.

`pd.to_datetime(col, format='%d-%m-%Y')` fails loudly on a wrong guess, whereas silent inference can misread `01-02-2026` as February first or January second. Ambiguous dates decoded wrong are a classic silent bug — decide the order, document it in the code, and the analysis never has to wonder.

PYTHON

```
pd.read_csv('Datasets/IPL Matches 2008-2020.csv').info()
```

PYTHON

```
pd.read_csv('Datasets/IPL Matches 2008-2020.csv', parse_dates=['date']).info()
```

14. The `converters` parameter

The `converters` parameter applies a function to a column *while the cell is being parsed*, before pandas even finishes reading: `converters={'price': lambda x: float(x.replace('INR', ''))}` strips currency symbols inside the read. It is per-column, runs once per value, and is the cleanest place to fix a column whose raw text is systematically dirty.

Compare it with the alternatives: `dtype` handles types but not cleaning, and a post-read swap performs cleaning but runs after the file is already in memory. Converters win when the transformation is simple, column-specific and known up front — parse failures mid-read stay isolated to that one column's function instead of bubbling up later. Reserve them for small, pure functions; anything complex deserves a named function, and anything requiring context across rows belongs after the read, in the DataFrame itself.

A clean mental division: converters are per-value honest, per-column narrow, and cheap to test in isolation — which makes them the right first reflex for a column like `price` that misbehaves the same way on every row.

PYTHON

```
pd.read_csv('Datasets/IPL Matches 2008-2020.csv', converters={'team1': rename})
```

15. The `na_values` parameter

Real files mark missing data in dozens of ways — `NA`, `N/A`, `null`, `-`, `?`, `Not Available`, even a stray `nan` as text. By default pandas recognises empty cells plus a few common tokens, and everything else survives as a confusing *string* that looks like data. `na_values=['NA', 'N/A', 'null', '-', '?']` teaches pandas exactly which tokens mean *missing for this file*, so they arrive as `NaN` where missing-value logic (`dropna`, `fillna`, `isnull`) actually works.

The counterpart is `keep_default_na=False`, which switches off the built-in set so you can fully own the list — useful when a file legitimately contains the string `NA` as a real value, or when your missing list is deliberately minimal. Read the raw file once, list every junk token you can find, and pin them in `na_values`: missing data that arrives typed as missing is missing data you can finally handle.

One symmetry to keep in mind: `na_values` governs *reading*, while `fillna` and `dropna` govern *using* — decide the tokens once at the door, then lean on the standard missing-value toolkit everywhere after.

PYTHON

```
pd.read_csv('Datasets/aug_train.csv')
```

PYTHON

```
pd.read_csv('Datasets/aug_train.csv',na_values=['Male',])
```

16. Loading a huge dataset in chunks

When a file is too big for memory, do not read it at all — **stream it**. `chunksize=10000` returns an iterator of DataFrames; with `for chunk in reader:` you process ten thousand rows at a time and never hold the whole file. The classic pattern aggregates each chunk (`chunk.groupby(...).sum()`) and accumulates the partial results, which is how you compute totals and summaries over a dataset that would otherwise be unreadable.

Two design rules keep chunked code honest: every per-chunk operation must reduce (each chunk returns a *summary*, not the raw rows), and the accumulation must be idempotent so re-running the loop produces the same answer. Chunking trades a little code for a huge drop in memory pressure, and it is the pattern behind every big-file workflow from here to distributed data. Same pandas, same answers — just taken in bites instead of one gulp.

PYTHON

```
dfs = pd.read_csv('Datasets/aug_train.csv',chunksize=5000)
```

PYTHON

```
for chunks in dfs:  
    print(chunks.shape)
```

Summary of `pd.read_csv` parameters

PARAMETER	WHAT IT DOES
<code>sep</code>	the delimiter (<code>,</code> default, <code>\t</code> for TSV, <code>;</code> ...)
<code>header</code>	which row holds column names
<code>names</code>	supply your own column names
<code>index_col</code>	use a column as the row index
<code>usecols</code>	load only selected columns
<code>dtype</code>	force column data types
<code>parse_dates</code>	parse date columns
<code>encoding</code>	how to decode the bytes
<code>na_values</code>	extra missing-markers
<code>converters</code>	run a function on a column while reading
<code>nrows</code>	read only the first N rows
<code>skiprows</code>	skip the first N / given rows
<code>chunksize</code>	read big files in batches
<code>on_bad_lines</code>	how to treat malformed rows

Reading JSON Files

JSON (JavaScript Object Notation) is a lightweight, nested data format used by almost all **APIs** and many modern databases.

JSON structure basics

A **list of objects** → a **DataFrame** (object keys become columns).

PYTHON

```
import pandas as pd
```

PYTHON

```
df = pd.read_json("file.json")           # from local file
df = pd.read_json(open_url_or_string)    # from URL / API
```

Working with JSON

JSON is the internet's default data format — every API response is a JSON document — and pandas reads it almost as easily as CSV. `read_json('file.json')` handles the common shape, an array of objects whose keys become columns; for nested or tricky documents, normalise first. The notebook also leans on `pd.json_normalize(...)`, which flattens nested dictionaries and lists into a tidy column-per-key table, and Python's own `json.loads` / `requests` for grabbing records over HTTP before stacking them into a DataFrame.

The mental model to keep: JSON is recursive — objects inside objects — while a DataFrame is flat. Reading JSON is therefore two steps in one: *parse* the document and *flatten* it to rows and columns. Get the second step right, and everything you already know about pandas — `info()`, dtypes, cleaning, plotting — applies to the API data with zero extra effort.

One warning to keep handy: JSON has no comment syntax and tolerates odd nesting, so a file that parses may still surprise you — print `df.shape`, `head()`, and `dtypes` as you would for any other source.

PYTHON

```
pd.read_json('Datasets/train (2).json')
```

PYTHON

```
pd.read_json('https://api.exchangerate-api.com/v4/latest/INR')
```

Reading Excel Files

Excel files (`.xlsx`) can hold **multiple sheets**. `pandas` uses the `openpyxl` / `xlrd` engines to read them.

Pandas fundamentals

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

PYTHON

```
pd.read_excel??
```

Reading Text Files

Plain text files are read with `pd.read_csv` + the right `sep` (since pandas needs to know how columns are separated).

Pandas fundamentals

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

PYTHON

```
# Reading a tab-separated text file
data = "question\tanswer\nQ1\tA1\nQ2\tA2"
df_txt = pd.read_csv(StringIO(data), sep="\t")
df_txt
```

Reading from SQL Databases

SQL databases are where most business data actually lives. Two common APIs:

Working with SQL

Databases are where most real data proudly lives, and the golden bridge is `pd.read_sql_query(query, conn)`: run a SQL `SELECT` and receive the result rows as a DataFrame, columns labelled and types inferred. The pattern shown uses `mysql.connector` to open a connection to a MySQL server — host, user, password, database — and then hands a hand-written query to pandas. Whatever `SELECT` you write is basically a request that the *database* does the heavy lifting: joins, filters and aggregations happen in SQL, and only the final result crosses into Python.

The whole `pandas + SQL` relationship is two-way. `read_sql_query` pulls; the reverse (`df.to_sql`) pushes a DataFrame into a table when the driver supports it. Which to use where is a judgement call — SQL is the right engine for filtering rows on the server, pandas for reshaping and analysis on the client. When in doubt, do the filtering in SQL and the analysis in pandas: each is excellent at what the other finds painful.

PYTHON

```
import mysql.connector
import pandas as pd
```

PYTHON

```
conn = mysql.connector.connect(host='localhost',user='root',password='',database='world')
```

Web APIs with `requests`

An **API** lets you fetch data over HTTP. The common pattern:

1. Build the request URL (often from a `base_url` + `endpoint` + `parameters`)
2. `GET` the endpoint, usually with headers for auth
3. Parse the JSON response
4. Convert the useful part into a DataFrame

PYTHON

```
import pandas as pd
import requests
```

PYTHON

```
response = requests.get('https://api.themoviedb.org/3/movie/top_rated?api_key=6c07d6cd00520704d95b3bad4468')
```

Web Scraping Static HTML with BeautifulSoup

When a website has **no API**, you can parse its **HTML** directly.

When scraping is appropriate

- No public API exists, and
- The site's terms allow it, and
- You respect rate limits and robots.txt.

The toolkit

- `requests` — download the HTML
- `BeautifulSoup` — parse the HTML and find elements

PYTHON

```
import pandas as pd
import requests
from bs4 import BeautifulSoup
import numpy as np
```

Handling the 403 — server-side blocks

A `403 Forbidden` response is the server's polite refusal: the request was received and understood, but access is not granted. For data gathering this is usually one of two stories — the endpoint genuinely restricts public access, or a bot-guard is fingerprinting the client. The notebook's response is an honest engineer's response: *look at the request itself first*. Is a required header missing? Is a cookie absent? Is the `User-Agent` one a guard would reject?

Fixing a 403 ethically means playing by the site's rules: add the missing header, respect the documented API, slow down, and never circumvent controls you do not have permission to bypass. If legitimate access keeps failing, step back and ask whether the data should be fetched through an official API or a downloadable export instead. A blocked request is often a sign to change your *source*, not your aggressiveness.

PYTHON

```
webpage=requests.get('https://www.ambitionbox.com/list-of-companies?page=1').text
```

PYTHON

```
soup=BeautifulSoup(webpage, 'lxml')
```

To find the names of companies

This cell is the turning point where the static-HTML scraping skills click into a real, repeatable loop: the site lists companies, and we want every name on the page. The recipe is already familiar — `requests.get` the page, wrap it in `BeautifulSoup`, and `soup.find_all(...)` to pull the repeated block that contains each name — but now the extraction is *programmatic* rather than visual. `get_text().strip()` per element, a list built by comprehension, and the results are clean strings.

Two habits matter for loops like this. First, verify the target with the browser's developer tools *before* writing the loop: one correct selector beats five guesses. Second, make the cell fail loudly — assert that a meaningful count was found, so a silently-changed page layout reveals itself immediately instead of yielding an empty list you might not notice until much later in the notebook.

And once the loop is trustworthy, keep the extraction to one cell with two outputs — the collected list plus its length — because a count you typed by hand is only useful while it matches the page.

PYTHON

```
for i in soup.find_all('h2'):
    print(i.text.strip())
```

Reading the company-name extraction loop

The cells that follow build the company list in three deliberately separated steps — inner text extraction, accumulation, and a final sanity count. The split is not accidental: each step is independently inspectable, so when a site's layout changes you can tell *exactly* which step broke. The loop collects the names from every page's repeated block, accumulating results into one list, and prints a total near the expected count — the data-quality check that separates a scraper that ran from a scraper that finished *right*. Run each cell in order and read the printed counts; the moment the total drifts, the layout has changed and the selector story has to be retold.

A few reading pointers for the three cells. The extraction step should stay *dumb* — one field, one selector, `get_text().strip()` — because clever inline logic is hard to audit later. The accumulation step owns all the bookkeeping, so its print should show how many names came from *this* page and how many exist in total. And the final count is your contract with the site: an explicit expected number, or at least a threshold, turns an empty scrape into an alarm instead of a silent wrong dataset.

PYTHON

```
for i in soup.find_all('p'):
    print(i.text.strip())
```

Common BeautifulSoup methods

METHOD	WHAT IT DOES
<code>soup.find("tag")</code>	find the first matching element
<code>soup.find_all("tag")</code>	find all matching elements
<code>soup.select("css")</code>	find with a CSS selector (<code>"div.item h2"</code>)
<code>.text</code> / <code>.get_text()</code>	extract the visible text
<code>["attr"]</code> / <code>.get("attr")</code>	read an attribute (e.g. <code>href</code>)
<code>.find(class_="price")</code>	find by class

PYTHON

```
# Extracting text and attributes
html = '''<a class="product" href="/p/1">Laptop</a>
<a class="product" href="/p/2">Mouse</a>'''
soup = BeautifulSoup(html, "html.parser")

names = [a.text for a in soup.find_all("a", class_="product")]
links = [a["href"] for a in soup.find_all("a", class_="product")]
print("names:", names)
print("links:", links)
```

Building a complete (small) scraper

The standard loop:

1. **Fetch** the HTML
2. **Parse** with BeautifulSoup
3. **Find** the repeated container elements
4. **Extract** fields into lists
5. **Assemble** lists into a DataFrame
6. **Save** to CSV/JSON

Always wrap in try/except — real pages change and break scrapers.

PYTHON

```
import pandas as pd
from bs4 import BeautifulSoup

# Simulated page with several products
html = '''<div class="card">
  <h2 class="name">Phone X</h2><p class="price">₹15000</p>
</div>
<div class="card">
  <h2 class="name">Phone Y</h2><p class="price">₹20000</p>
</div>'''
soup = BeautifulSoup(html, "html.parser")

names = [n.text for n in soup.find_all("h2", class_="name")]
prices = [p.text for p in soup.find_all("p", class_="price")]

df_scrape = pd.DataFrame({"name": names, "price": prices})
df_scrape
```

Handling errors with try / except

If one page fails, you **skip it and continue** instead of crashing the whole job.

PYTHON

```
import requests
from bs4 import BeautifulSoup

def scrape_page(url):
    try:
        r = requests.get(url, timeout=10)
        r.raise_for_status()
        return BeautifulSoup(r.text, "html.parser")
    except requests.exceptions.RequestException as e:
        print("Failed:", url, "->", e)
        return None

# soup = scrape_page("https://example.com") # replace with a real URL
print("Robust scraper function defined.")
```

PYTHON

```
try:
    r = requests.get(url, timeout=10)
    r.raise_for_status()
    soup = BeautifulSoup(r.text, "html.parser")
    ...extract...
except requests.exceptions.RequestException as e:
    print("Skipping page:", e)
```

B24 · Web Scraping Dynamic Sites with Selenium

Some sites render content with **JavaScript** — the HTML you download with `requests` is empty. `Selenium` controls a **real browser** (Chrome/Firefox), executes the JS, and lets you read the *rendered* page.

Basic workflow

1. Create a `WebDriver` (Chrome options: headless mode, no sandbox)
2. `driver.get(url)` — load the page (JS runs)
3. Wait for elements (explicit waits, **not** `time.sleep()`)
4. Find elements (`By.ID`, `.CLASS_NAME`, `.CSS_SELECTOR` ...)
5. Read / click / scroll
6. `driver.quit()` to close it

BASH

```
pip install selenium
# also need a matching chromedriver (or use Selenium Manager)
```

Selenium locators (`By`)

Selenium finds elements by explicit locator type, and the import `from selenium.webdriver.common.by import By` gives you the whole menu. `By.ID` for ids, `By.CLASS_NAME` / `By.CSS_SELECTOR` for classes, `By.NAME` for named fields, `By.XPATH` for structural paths, and `By.TAG_NAME` for plain tags like `links = driver.find_elements(By.TAG_NAME, 'a')`. Every watched element answers to at least one of these — the skill is choosing the one that is (a) stable and (b) pinpoints exactly what you want.

Rank them in practice: **CSS selectors first** (readable, robust to markup shifts), **XPath** when you must climb the tree or match on text, **id/name** when the page author provides them (they are uniquely designed for this), and raw tag names only for quick sweeps. The wrong locator is the classic flaky-test culprit: fine on Monday, broken on Tuesday, for no visible reason. Pick locators as carefully as you pick column names.

One last tip: when a selector stops matching, verify it in a fresh browser session before editing code — a stale page or a closed driver causes more false alarms than the site's markup does.

Waiting strategies — explicit waits

JavaScript changes a page *after* it loads, and a scraper that reads too early sees empty containers. The robust answer is **explicit waits**: `wait = WebDriverWait(driver, 10)` and then `wait.until(EC.presence_of_element_located((By.ID, 'result')))` — the driver actively polls until the condition holds, or raises after ten seconds. That is fundamentally safer than a fixed `time.sleep`, because it waits for the *right thing* (the element existing) rather than a guessed amount of time.

Choose the condition to match what you actually need: `presence_of_element_located` for it being in the DOM, `visibility_of_element_located` for it being visible and clickable, `element_to_be_clickable` before `.click()`. Explicit waits localise the flake to the one interaction that needs it; implicit waits and blind sleeps are the currency of frustration. Browser automation is only as stable as its waits — engineer them deliberately.

One tuning note: set the timeout slightly longer than the slowest legitimate load, and combine the explicit wait with a guard — `wait.until(...)` then one defensive check — so a genuinely dead page raises a clear message instead of a vague one.

PYTHON

```
# Selenium setup (illustration – several lines are commented to avoid
# launching a real browser in this notebook)
# from selenium import webdriver
# from selenium.webdriver.common.by import By
# from selenium.webdriver.chrome.options import Options
# from selenium.webdriver.support.ui import WebDriverWait
# from selenium.webdriver.support import expected_conditions as EC
#
# opts = Options()
# opts.add_argument("--headless")
# driver = webdriver.Chrome(options=opts)
# driver.get("https://quotes.toscrape.com/js/")
# wait = WebDriverWait(driver, 10)
# cards = wait.until(EC.presence_of_all_elements_located((By.CLASS_NAME, "quote")))
# for c in cards:
#     print(c.find_element(By.CLASS_NAME, "text").text)
# driver.quit()
print("Selenium example shown (commented) – requires a browser + driver.")
```

Selenium vs BeautifulSoup — when to use which

SITUATION	USE
Static HTML, simple selectors	<code>BeautifulSoup</code> (fast, light)
Data behind JS / login / interaction	<code>Selenium</code> (real browser)
Huge volume, polite & fast	<code>BeautifulSoup</code> + <code>requests</code>
Complex clicks, scrolling, sessions	<code>Selenium</code>

Best practice: always try `requests` + `BeautifulSoup` first; only bring `Selenium` in when the page truly requires a rendered browser.

B25 · Exporting Data (Writing Out)

After gathering (and eventually analysing) data you usually **save** the result. `pandas` writes nearly every format it can read.

`to_csv`

Key parameters: `sep` (delimiter), `index` (keep index?), `encoding`, `header` (names?), `columns` (subset).

`to_excel`

`to_json`

`to_sql`

`tohtml` / `tostring`

Note: the cell above uses a small in-memory CSV. In practice you would gather a real dataset, then export with `df.to_csv("result.csv", index=False)`. The full set of export methods is shown in the table above.

PYTHON

```
# Demonstration DataFrame (gathering a small dataset in memory)
data = "id,name,score\n1,Alice,90\n2,Bob,85\n3,Carol,88"
df_out = pd.read_csv(StringIO(data))
df_out
```

PYTHON

```
df.to_csv("out.csv", index=False)
```

B26 · Summary — A Complete Data-Gathering Cheatsheet

SOURCE	METHOD	NOTES
Local CSV	<code>pd.read_csv()</code>	many params (<code>sep</code> , <code>dtype</code> , <code>parse_dates</code> ...)
CSV from URL	<code>pd.read_csv(url)</code> or <code>requests</code>	handle headers/errors
JSON	<code>pd.read_json()</code> , <code>json_normalize</code>	flatten nested data
Excel	<code>pd.read_excel()</code>	<code>sheet_name</code> for multi-sheet
Text/TSV	<code>pd.read_csv(sep="\t")</code>	set the delimiter
SQL	<code>pd.read_sql_query()</code> / <code>read_sql_table()</code>	<code>mysql.connector</code> / <code>sqlalchemy</code>
API	<code>requests</code> + <code>pd.DataFrame()</code>	pagination + rate limits
Scrape static	<code>requests</code> + <code>BeautifulSoup</code>	parse HTML
Scrape dynamic	<code>selenium</code>	JS-rendered pages
Export	<code>to_csv/excel/json/sql/html</code>	<code>index=False</code> recommended

The universal "load then inspect" habit

Get this habit down and the rest of the Data Analysis Process flows naturally.

PYTHON

```
df = <gather the data>
df.shape      # rows, columns
df.info()     # dtypes + nulls
df.head()     # a peek
df.isna().sum() # missing values
```

B27 · Practice Problems

Try these yourself — they combine *everything* above.

Problem 1 — CSV → cleaned DataFrame Take any real CSV (e.g. from Kaggle). Load it, inspect `.info()`, then:

- read only 3 useful columns (`usecols`)
- force one string-ID column to stay text (`dtype`)
- parse a date column (`parse_dates`)
- save the result to a new CSV without the index.

Problem 2 — JSON API → DataFrame Find a free public API. Fetch the JSON, inspect its structure, and convert the payload into a tidy DataFrame. Flatten any nested arrays with `pd.json_normalize`.

Problem 3 — Excel multi-sheet Create an Excel file with two sheets, then read them back with `sheet_name`.

Problem 4 — Web scraping (static) Pick a simple static page, scrape a repeated element (e.g. a list of items with name + price), put it in a DataFrame, handle errors with try/except, and export to CSV.

Problem 5 — Web scraping (dynamic / Selenium) Take a JS-rendered page (e.g. quotes.toscrape.com/js), use Selenium with explicit waits to gather several pages, and combine them.

B28 · Wrap-Up

You now have the **full toolkit to bring data into Python**:

- The overall **Data Analysis Process** (define → gather → clean → transform → explore → model → communicate).
- **Data Gathering** in detail: CSV, URL, JSON, Excel, text, SQL, APIs, scraping (static + dynamic), and exporting.

What's next? The data is rarely ready to use. Move on to **DataAssessingand_Cleaning.ipynb** to learn how to inspect and fix the problems you will find in almost every gathered dataset.

Remember: Gathering is only the *first* step. The value is created by cleaning, analysing, and **acting** on the data you collect.

Summary

You practised the widest possible funnel of data gathering: every core `read_csv` parameter, Excel, JSON, text, SQL over MySQL, REST APIs, static-HTML scraping with BeautifulSoup, Selenium locators and waits, and the export step that closes the loop. The transferable lessons are the guardrails: know your source's format, pin your encodings and types, check status codes, verify every read with `head()` / `shape`, and let failures inform you rather than surprise you. Gather cleanly at the door, and every analysis after it stays honest.

Keep this summary as a checklist for the task notebook that follows: read parameters, encodings, types and missing tokens up front; verify every source with `head()` and `shape`; check status codes and page totals; and export each gathered result so the work survives as a file.

This notebook ran the full cleaning lifecycle on a mess on purpose: it loaded four clinical-trial files, assessed each against quality issues (missing, duplicate, invalid, inaccurate) and tidiness issues (values in column names, rows split), then cleaned with triaged priority — rescuing `severity` and `initial_dose`, re-creating `treatments_cut` from the gold-standard table, splitting mixed columns, renaming and standardising labels — and re-assessed after every repair until the audit was quiet. The transferable product is the process itself: assess first, define your list, fix in severity order, verify every change, and loop until clean.

The task notebook now asks you to reproduce this loop under time pressure — so treat today's patient, step-by-step version as the draft you rehearse from.

This notebook executed EDA as a complete loop — load and glance, univariate deep-dives into `Age` and `Fare`, categorical counts, survival cross-tabs with normalised percentages, feature engineering of family, ticket, title and deck columns, and a closing correlation heatmap on the combined train-plus-test frame. The skills banked are the reading habits: detecting skew from `describe()`, slicing every summary by the outcome, converting counts into defensible labels, fixing a wrong observational unit, and compressing many features into one prioritised grid. Every plot you produced here is a claim you can now defend — which is the entire point.

Revisit any plot you skimmed and write its insight in one line — the act of labelling is where the credit for this notebook accrues.

This case study ran the cleaning lifecycle end to end on a messy smartphone file: assessment with `info()` / `describe()` and a page-long defect list, targeted repairs that preferred fixing over dropping, surgical pandas — working copies, `.str` chains, `shift` realignment, `value_counts` verification — and column splitting that turned compound strings into typed, analysis-ready features. It closed by dropping the exhausted raw columns and exporting a tidy `smartphone_cleaned_v2.csv`. The product to take away is the loop itself: assess first, list your defects, fix in priority order, verify every repair, and repeat until the audit is quiet. Run the loop on your next real dataset and you will feel the payoff: a quiet audit means every later step is standing on clean ground.

This notebook ran the EDA loop on the cleaned smartphone dataset: univariate audits of every numeric and categorical column, bivariate barplots and scatterplots of price against features, and a correlation pass — with `KNNImputer` fixing the missing ratings so `.corr()` was trustworthy, and `get_dummies` bringing the categorical columns into the numeric fold. The findings land cleanly: connectivity flags, storage and screen size drive price, while battery alone barely moves it. The skill to keep is the loop of evidence — ask, plot, rank, impute, conclude — because that is the same loop you will point at in any future dataset.

This session advanced the smartphone EDA from a clean base: it audited the `v3` file, fixed leftover tag errors, engineered `fast_charging_available` and `extended_memory`, and checkpointed the result as `v4` then `v5` — normalising `num_cores` to numeric on the way to modelling. The analytical core imputed the 101 missing ratings with `KNNImputer`, correlated price and rating before and after the fill to prove what the imputation changed, and one-hot encoded brand and OS with `get_dummies(drop_first=True)` so every feature takes part. The takeaway is iteration with receipts: versioned exports, before-after correlation compares, and a final CSV that is genuinely model-ready.

This notebook walked the full EDA ladder on Titanic — load and orient, univariate description of numerical and categorical columns, bivariate survival slices, and feature engineering — and then repeated the loop on the insurance dataset to prove the recipe generalises. You should now be fluent at reading `describe()`, value counts and heatmaps, at noticing skew in a histogram, and at turning rows and columns into questions. The habits built here — predict, inspect, conclude, then move on — are exactly the ones you will reuse for every dataset in the course and in the final case studies.

You crossed the full ladder: `requests` to fetch and check status codes, a polite `polite_get` wrapper, BeautifulSoup to parse into a searchable tree, `.get_text()` and attribute extraction for the data itself, DataFrame assembly, pagination loops, retry handling, and Selenium for JavaScript-rendered pages. The transferable lessons are the rules: inspect status before parsing, strip every extracted string, sleep between hits, and treat every failure as a branch to retry. With these four habits you can scrape almost any public page cleanly — and, just as important, politely.

One habit to carry forward: before you scrape a page, open it in a normal browser first and look — the inspect panel answers most selector questions before the first request is even sent.

Data Assessing and Cleaning — the clinical trials dataset

Summary of today's session

This notebook is one continuous detective story on a single, deliberately grubby dataset. The plot: four clinical-trial files arrive with textbook defects that will later bite — a column whose name contains a value, rows where an entire condition is missing, severity labels like Hormonal where High belongs, and fake-looking treatment records that duplicated trailing rows. Your job across the notebook is not to memorise functions but to master the rhythm they sit inside: **assess** → **define** → **clean** → **re-assess** → **repeat**. Everything you do here — from the first `df.duplicated().sum()` to the final `to_csv` of the deep-cleaned frames — is that rhythm made visible. The deepest lesson is the loop itself: cleaning is not a step at the end of analysis, it is the analysis before the analysis.

Use this page as your map: when a later cell seems to jump forward, find which defect it is answering — missing, duplicate, invalid, inaccurate, mixed, or renamed — and you will always know why the repair exists.

1. Data Wrangling

- Data Gathering
- Data Accessing
- Data Cleaning

1b. Data accessing

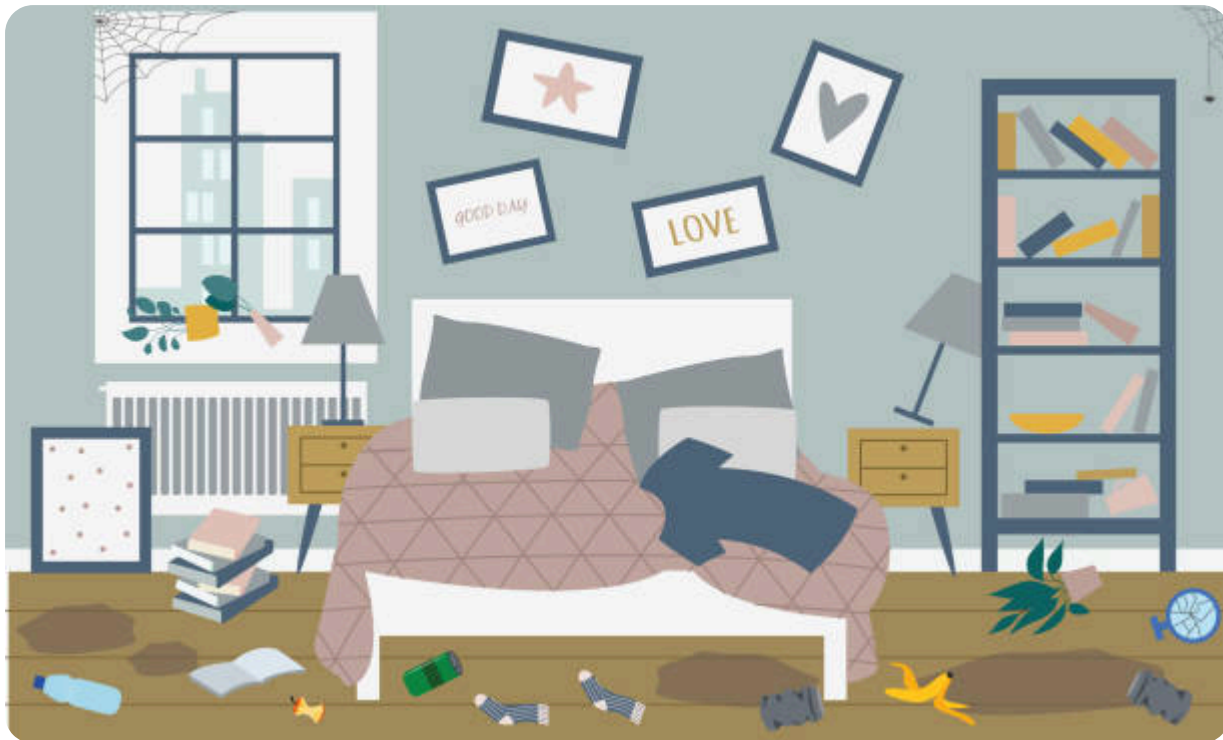
Before assessing we have to *open the doors* — which is exactly what this section does with `os.listdir('/kaggle/input')` and the reads that follow. Even this unglamorous step deserves discipline: each file gets inspected the moment it loads, `head()` to see the shape of the mess, and `shape` + `info()` to record its size in our heads. The four frames are small, but the *habit* scales: in a real project with forty files, the analyst who checks each one on arrival is the one who ships data without incident.

Also notice how the notebook binds the four frames to short, honest names (`patients` , `treatments` , `adverse_reactions`) and keeps `treatments_cut` separate from full `treatments` — because a comparison between the two will *be* the punchline of the assessment later. How you load is already part of how you analyse.

Two loading reflexes repeat everywhere in this notebook: never read without a `head()` straight after, and never assume a column's `dtype` is what it pretends to be — both cost one line and prevent whole classes of confusion.

Types of Unclean Data

There are 2 kinds of unclean data



- **Dirty Data (Data with Quality issues)** : Dirty data, also known as low quality data. Low quality data has content issues.
 - Duplicated data
 - Missing Data
 - Corrupt Data
 - Inaccurate Data
- **Messy Data (Data with tidiness issues)** : Messy data, also known as untidy data. Untidy data has structural issues. Tidy data has the following properties:
 - Each variable forms a column
 - Each observation forms a row
 - Each observational unit forms a table



country	year	rate
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

table3

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

PYTHON

```
import pandas as pd
import numpy as np
```

The four datasets — know who you are auditing

Before any assessment, orient yourself in the four files you just loaded. `patients` holds one row per patient (age, weight, a smoking flag...), `treatments` holds one row per treatment event (dose, outcome, at least two patients' data), `adverse_reactions` links patients to reactions, and `treatments_cut` is the fun of the party — a deliberately truncated copy of `treatments` that sits in this session only so its rows can be re-created correctly from the full table later. The moments that follow lean on these identities: duplicates in `patients`, an invalid severity in `treatments`, homing in on one patient's record... all make sense only because you know which frame is which. Two short habits carry the whole notebook: keep printing `shape` before and after every transformation, and never trust a frame you just read until you have eyed the first rows with your own `head()`.

PYTHON

```
patients = pd.read_csv('patients.csv')
treatments = pd.read_csv('treatments.csv')
adverse_reactions = pd.read_csv('adverse_reactions.csv')
treatments_cut = pd.read_csv('treatments_cut.csv')
```

PYTHON

```
# view datasets
patients.head()
```

1. Write a summary for your data

This is a dataset about 500 patients of which 350 patients participated in a clinical trial. None of the patients were using Novodra (a popular injectable insulin) or Auralin (the oral insulin being researched) as their primary source of insulin before. All were experiencing elevated HbA1c levels.

All 350 patients were treated with Novodra to establish a baseline HbA1c level and insulin dose. After 4 weeks, which isn't enough time to capture all the change in HbA1c that can be attributed by the switch to Auralin or Novodra:

- 175 patients switched to Auralin for 24 weeks
- 175 patients continued using Novodra for 24 weeks

Data about patients feeling some adverse effects is also recorded.

2. Write Column descriptions

Table -> **patients** :

- **patient_id** : the unique identifier for each patient in the Master Patient Index (i.e. patient database) of the pharmaceutical company that is producing Auralin
- **assigned_sex** : the assigned sex of each patient at birth (male or female)
- **given_name** : the given name (i.e. first name) of each patient
- **surname** : the surname (i.e. last name) of each patient
- **address** : the main address for each patient
- **city** : the corresponding city for the main address of each patient
- **state** : the corresponding state for the main address of each patient
- **zip_code** : the corresponding zip code for the main address of each patient
- **country** : the corresponding country for the main address of each patient (all United states for this clinical trial)
- **contact** : phone number and email information for each patient
- **birthdate** : the date of birth of each patient (month/day/year). The inclusion criteria for this clinical trial is age ≥ 18 (there is no maximum age because diabetes is a growing problem among the elderly population)
- **weight** : the weight of each patient in pounds (lbs)
- **height** : the height of each patient in inches (in)
- **bmi** : the Body Mass Index (BMI) of each patient. BMI is a simple calculation using a person's height and weight. The formula is $BMI = \frac{kg}{m^2}$ where kg is a person's weight in kilograms and m2 is their height in metres squared. A BMI of 25.0 or more is overweight, while the healthy range is 18.5 to 24.9. The inclusion criteria for this clinical trial is $16 \leq BMI \leq 38$.

Table -> **treatments** and **treatment_cut** :

- **given_name** : the given name of each patient in the Master Patient Index that took part in the clinical trial

- **surname** : the surname of each patient in the Master Patient Index that took part in the clinical trial
- **auralin** : the baseline median daily dose of insulin from the week prior to switching to Auralin (the number before the dash) and the ending median daily dose of insulin at the end of the 24 weeks of treatment measured over the 24th week of treatment (the number after the dash). Both are measured in units (shortform 'u'), which is the international unit of measurement and the standard measurement for insulin.
- **novodra** : same as above, except for patients that continued treatment with Novodra
- **hba1c_start** : the patient's HbA1c level at the beginning of the first week of treatment. HbA1c stands for Hemoglobin A1c. The HbA1c test measures what the average blood sugar has been over the past three months. It is thus a powerful way to get an overall sense of how well diabetes has been controlled. Everyone with diabetes should have this test 2 to 4 times per year. Measured in %.
- **hba1c_end** : the patient's HbA1c level at the end of the last week of treatment
- **hba1c_change** : the change in the patient's HbA1c level from the start of treatment to the end, i.e., $hba1c_{start} - hba1c_{end}$. For Auralin to be deemed effective, it must be "noninferior" to Novodra, the current standard for insulin. This "noninferiority" is statistically defined as the upper bound of the 95% confidence interval being less than 0.4% for the difference between the mean HbA1c changes for Novodra and Auralin (i.e. Novodra minus Auralin).

Table -> **adverse_reactions**

- **given_name** : the given name of each patient in the Master Patient Index that took part in the clinical trial and had an adverse reaction (includes both patients treated Auralin and Novodra)
- **surname** : the surname of each patient in the Master Patient Index that took part in the clinical trial and had an adverse reaction (includes both patients treated Auralin and Novodra)
- **adverse_reaction** : the adverse reaction reported by the patient

3. Add any additional information

Additional useful information:

- Insulin resistance varies person to person, which is why both starting median daily dose and ending median daily dose are required, i.e., to calculate change in dose.
- It is important to test drugs and medical products in the people they are meant to help. People of different age, race, sex, and ethnic group must be included in clinical trials. This diversity is reflected in the patients table.

The two kinds of assessment

Professional data assessment sorts every defect into one of two buckets, and it is worth being a pedant about the difference because the *fixes* differ. A **quality issue** is a data error: missing values, duplicates, invalid entries (Hormonal as a severity), inaccurate values. A **tidiness issue** is a structural problem: each table must hold the same kind of observational unit, each column must hold one variable, each row one observation — so a *value stored in a column name* (e.g. a column literally named **initial** holding zeros and ones) or an *observation split across two rows* is a tidiness fault, not a quality fault.

The audit that follows uses both lenses deliberately: `df.isnull().sum()` for missing, `.duplicated()` for duplicates, but also a hard glance at what the columns are *supposed* to hold given what they are named. Both buckets must be empty before a dataset deserves to be called clean.

SQL

```
# export data for manual assessment

with pd.ExcelWriter('clinical_trials.xlsx') as writer:
    patients.to_excel(writer, sheet_name='patients')
    treatments.to_excel(writer, sheet_name='treatments')
    treatment_cut.to_excel(writer, sheet_name='treatment_cut')
    adverse_reactions.to_excel(writer, sheet_name='adverse_reactions')
```

Issues with the dataset

1. Dirty Data

Table - `Patients`

- patient_id = 9 has misspelled name 'Dsvid' instead of David `accuracy`
- state col sometimes contain full name and some times abbrivietation `consistency`
- zip code col has entries with 4 digit `validity`
- data missing for 12 patients in address,city, state,zip_code ,country, contact `completion`
- incorrect data type assigned to sex, zip code, birthdate `validity`
- duplicate entries by the name of John Doe `accuracy`
- one patient has weight = 48 pounds `accuracy`
- one patient has height = 27 inches `accuracy`

Table - `Treatments` & `Treatments_cut`

- given_name and surname col is is all lower case `consistency`
- remove u from Auralin and Novadra cols `validity`
- '-' in novadra and Auralin col treated as nan `validity`
- missing values in hba1c_change col `completion`
- 1 duplicate entry by the name Joseph day `accuracy`
- in hba1c_change 9 instead of 4 `accuracy`

Table - `Adverse_reactions`

- given_name and surname are all in lower case `consistency`

2. Messy Data

Table - `Patients`

- contact col contains both phone and email

Table - `Treatments` & `Treatments_cut`

- Auralin and Novadra col should be split into 2 cols start and end dose
- merge both the tables

Table - `Adverse_reactions`

- This table should not exist independently

Automatic Assessment

- head and tail
- sample
- info
- isnull
- duplicated
- describe

PYTHON

```
adverse_reactions.info()
```

Deep Dive : Automatic assessment — identify dirty data

- Data Wrangling: Gather → Assess → Clean
- Automatic assessment = programmatic checks:
 - info() — null counts per column
 - isnull() filter — missing records
 - duplicated() — exact & subset dup
 - describe() — min/max ranges reveal outliers (height=27!)
 - sort_values — extremes at top/bottom
- Height=27 → human anomaly flagged
- Data quality dims: Completeness < Validity < Accuracy < Consistency
- Manual + automatic combined = Sherlock style
- Iterative process — assess again after cleaning
- Every check maps to a data issue type

PYTHON

```
patients[patients['address'].isnull()]
```

PYTHON

```
treatments[treatments.duplicated()]
```

Cleaning is iterative

One shower does not make a decade of grime disappear, and one cleaning pass does not disinfect a dataset — so this notebook explicitly folds the loop into its structure: assess, fix one class of defect, **re-assess**, find what the first pass hid, fix again. The reason is recursion, not paranoia: dropping rows or splitting columns *changes the frame*, and an inspected-then-altered frame is a frame that must be inspected again.

`df.duplicated().sum()`, `df.isnull().sum()` and `df.shape` therefore reappear at multiple checkpoints, each time against the *current* state of the data.

The habit generalises beautifully: it is why real cleaning work is always reported as a *process* (before → after) rather than a destination. If you reach a checkpoint where the re-audit adds nothing, you are done — and you know you are done because you checked, not because you got tired.

When you write your own cleaning later, mirror this structure: checkpoint after each repair, re-run the same audit print, and only move on when the numbers hold still.

Data Quality Dimensions

- Completeness -> is data missing?
- Validity -> is data invalid -> negative height -> duplicate patient id
- Accuracy -> data is valid but not accurate -> weight -> 1kg
- Consistency -> both valid and accurate but written differently -> New Youk and NY

Order of severity

No dataset is cleaned in one breath, so the notebook commits to a priority order and announces it: **first the column that is hurting the most**. Here that means rescuing the critical medical variable — fixing `severity` (which hides duplicates and an outright invalid label) before touching cosmetic annoyances, and repairing `initial_dose` — where the trailing frame row turned every value into text — before any tidiness reshaping. The logic is triage: the defects that poison analysis or crash downstream code get fixed first, whatever is purely cosmetic waits.

There is a quiet second agenda: *fixing what changes the story*. Every transformed column that alters counts or values must be re-verified afterwards, because a repaired column that silently changed meaning is a new defect in disguise. Priority plus verification — that is the working definition of serious cleaning.

A useful discipline for the checking half: after each repair, re-run the exact command that exposed the defect and paste the before-and-after counts into your notes — evidence makes the priority order defensible later.

Data Cleaning Order

1. Quality -> Completeness
2. Tidiness
3. Quality -> Validity
4. Quality -> Accuracy
5. Quality -> Consistency

Steps involved in Data cleaning

- Define
- Code
- Test

Always make sure to create a copy of your pandas dataframe before you start the cleaning process

PYTHON

```
patients_df = patients.copy()
treatments_df = treatments.copy()
treatments_cut_df = treatments_cut.copy()
adverse_reactions_df = adverse_reactions.copy()
```

6. Defining — the column that explains everything

This section introduces a column that summarises each patient in one line: **cleanup** — built by joining pieces of information (a condition tag, a severity label, an initial-dose figure) into a single boolean statement of status. Watch *how* it is assembled: several conditions combined with `|` (pandas or), cast with `.astype(bool)`, and each piece mapped to a specific observed defect from the assessment.

It is a textbook example of the cleanest principle in this entire notebook: **every new column you add should be traceable to a defect you found**. A column that is a transparent boolean of the assessment is self-documenting — anyone (including future you) can read what it means and why it exists. Columns built from pure vibes are how datasets quietly rot; columns built from the audit are how they stay trustworthy.

One reading hint: trace each `|` clause back to its finding in the earlier assessment, and you will see the same mapping — diagnose, then code — that every good derived column follows.

Deep Dive : Cleaning step 1 - Completeness (fillna)

- Copy before cleaning - preserve raw for audit
- fillna - fill missing with placeholder
- Completeness fix: missing values addressed first
- Verify via info() - non-null == all
- Quality first then tidiness then quality again
- cleaning iterates: define -> code -> test
- fill placeholder 'No data' keeps text readable
- decide fill vs drop by missing ratio
- numeric/date columns - fillna(mean) or interpolate later
- Document every cleaning decision

PYTHON

```
# code
patients_df.fillna('No data', inplace=True)
```

PYTHON

```
# test
patients_df.info()
```

Deep Dive : Cleaning - derived columns & parsing contact

- derive metric: hba1c_change from start - end
- parse messy single string into structured columns (email/phone)
- apply(lambda) - per-row transform function
- drop original raw column after extraction
- regex patterns to isolate email/phone/url
- Validity & accuracy fixes - values become correct + parseable
- test step validates transformation
- build utilities (findcontactdetails) reusable
- tidy: one variable per column

PYTHON

```
# code
treatments_df['hba1c_change'] = treatments_df['hba1c_start'] - treatments_df['hba1c_end']
treatments_cut_df['hba1c_change'] = treatments_cut_df['hba1c_start'] - treatments_cut_df['hba1c_end']
```

PYTHON

```
# test
treatments_cut_df.info()
```

Deep Dive : Cleaning — tidiness (concat, melt, merge)

- concat — join two same-schema sets (full table)
- melt — wide→long tidy (each variable own column)
- filter junk rows (dosage '-')
- split '150u-250u' → start/end ints (parse + strip unit)
- merge left — attach adverse reaction info
- Tidiness: each observation = row, each variable = column
- mixed types: strings '150u-250u' split into clean numerics
- PK join keys given_name+surname
- cleaning pipeline order fixed
- end state = analysis-ready structured df

PYTHON

```
treatments_df = pd.concat([treatments_df, treatments_cut_df])
```

PYTHON

```
treatments_df = treatments_df.melt(id_vars=['given_name', 'surname', 'hba1c_start', 'hba1c_end', 'hba1c_cha
```

Exploratory Data Analysis — Titanic, hands-on

Why do EDA

- Model building
- Analysis and reporting
- Validate assumptions
- Handling missing values
- feature engineering
- detecting outliers

EDA is the **pre-modelling** step that helps us:

- **Model building** — understanding the data before choosing/creating models.
- **Analysis and reporting** — discovering and communicating patterns.
- **Validate assumptions** — checking whether statistical assumptions hold.
- **Handling missing values** — spotting gaps and deciding how to fill/exclude them.
- **Feature engineering** — creating new, more informative columns from existing ones.
- **Detecting outliers** — finding extreme values that may distort a model.

Remember: EDA is an **iterative process**. You keep going back and forth between exploring and cleaning — there is no "one pass and you are done".

Column Types

- **Numerical** - Age, Fare, PassengerId
- **Categorical** - Survived, Pclass, Sex, SibSp, Parch, Embarked
- **Mixed** - Name, Ticket, Cabin

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

Before analysing we classify every column:

- **Numerical** — continuous/discrete numbers you can do arithmetic on: Age, Fare, PassengerId.
- **Categorical** — values that are labels/groups: Survived, Pclass, Sex, SibSp, Parch, Embarked.
- **Mixed** — messy text that needs parsing: Name, Ticket, Cabin.

Let's inspect the raw data types:

```
PYTHON
```

```
df.info()
```

```
PYTHON
```

```
df.info()
```

Univariate Analysis

Univariate analysis focuses on analyzing each feature in the dataset independently.

- **Distribution analysis:** The distribution of each feature is examined to identify its shape, central tendency, and dispersion.
- **Identifying potential issues:** Univariate analysis helps in identifying potential problems with the data such as outliers, skewness, and missing values

Univariate analysis focuses on analysing **each feature independently**.

Two main goals of univariate analysis:

1. **Distribution analysis** — the shape, central tendency and dispersion of each feature.
2. **Identifying potential issues** — outliers, skewness and missing values.

The shape of a data distribution

The **shape** refers to the overall pattern of the data when plotted. Common shapes:

- **Normal Distribution** — symmetrical, bell-shaped; mean = median = mode; most data in the middle, frequencies taper off towards the tails.
- **Skewed Distribution** — not symmetrical, one tail longer than the other.
 - **Positively (right) skewed:** long tail on the right (mean > median).
 - **Negatively (left) skewed:** long tail on the left (mean < median).
- **Bimodal Distribution** — two peaks (two modes).
- **Uniform Distribution** — every value equally likely.

Knowing the shape matters because it reveals outliers, skewness, and tells us which statistical tests and models are appropriate.

Dispersion (spread)

Dispersion measures how far the values spread out from the central tendency. Common measures:

- **Range** — largest – smallest value.
- **Variance** — average of the squared deviations from the mean.
- **Standard Deviation** — square root of variance; spread in the *same units* as the data.
- **Interquartile Range (IQR)** — value between the 25th percentile (Q1) and 75th percentile (Q3).

Dispersion helps identify extreme values and outliers.

The shape of a data distribution refers to its overall pattern or form as it is represented on a graph.

Some common shapes of data distributions include:

- **Normal Distribution:** A symmetrical and bell-shaped distribution where the mean, median, and mode are equal and the majority of the data falls in the middle of the distribution with gradually decreasing frequencies towards the tails.
- **Skewed Distribution:** A distribution that is not symmetrical, with one tail being longer than the other. It can be either positively skewed (right-skewed) or negatively skewed (left-skewed).
- **Bimodal Distribution:** A distribution with two peaks or modes.
- **Uniform Distribution:** A distribution where all values have an equal chance of occurring.

The shape of the data distribution is important in identifying the presence of outliers, skewness, and the type of statistical tests and models that can be used for further analysis.

Dispersion is a statistical term used to describe the spread or variability of a set of data. It measures how far the values in a data set are spread out from the central tendency (mean, median, or mode) of the data.

There are several measures of dispersion, including:

- **Range:** The difference between the largest and smallest values in a data set.
- **Variance:** The average of the squared deviations of each value from the mean of the data set.
- **Standard Deviation:** The square root of the variance. It provides a measure of the spread of the data that is in the same units as the original data.
- **Interquartile range (IQR):** The range between the first quartile (25th percentile) and the third quartile (75th percentile) of the data.

Dispersion helps to describe the spread of the data, which can help to identify the presence of outliers and skewness in the data.

Steps of doing Univariate Analysis on Numerical columns

- **Descriptive Statistics:** Compute basic summary statistics for the column, such as mean, median, mode, standard deviation, range, and quartiles. These statistics give a general understanding of the distribution of the data and can help identify skewness or outliers.
- **Visualizations:** Create visualizations to explore the distribution of the data. Some common visualizations for numerical data include histograms, box plots, and density plots. These visualizations provide a visual representation of the distribution of the data and can help identify skewness and outliers.

- **Identifying Outliers:** Identify and examine any outliers in the data. Outliers can be identified using visualizations. It is important to determine whether the outliers are due to measurement errors, data entry errors, or legitimate differences in the data, and to decide whether to include or exclude them from the analysis.
- **Skewness:** Check for skewness in the data and consider transforming the data or using robust statistical methods that are less sensitive to skewness, if necessary.
- **Conclusion:** Summarize the findings of the EDA and make decisions about how to proceed with further analysis.

Age

conclusions

- Age is normally(almost) distributed
- 20% of the values are missing
- There are some outliers
- Roughly uniform between 18–64, with a small bump near 18–20.
- No extreme outliers; `skew` is close to 0 (slightly negative).

PYTHON

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
ins['age'].plot(kind='hist', bins=30, ax=axes[0]); axes[0].set_title('Age Histogram')
ins['age'].plot(kind='kde', ax=axes[1]); axes[1].set_title('Age KDE')
ins['age'].plot(kind='box', ax=axes[2]); axes[2].set_title('Age Box')
plt.tight_layout(); plt.show()
```

Deep Dive : Univariate — numerical cols (Age)

- Univariate analysis = one column at a time
- describe → stats: center (mean, median) & spread (std, IQR)
- hist — shape; kde — smooth density compare
- skew — sign & magnitude (0 normal, + right tail)
- box — min Q1 med Q3 max + outliers outside 1.5×IQR
- filter extremes — sanity check values
- NaN % — missingness fraction
- Age almost normal, no extreme skew → valid feature
- conclusions per column documented
- Step: Descriptive stats → plots → skew/missing → summary

PYTHON

```
df['Age'].describe()           # count,mean,std,min,quartiles,max
df['Age'].plot(kind='hist', bins=20) # distribution shape
df['Age'].plot(kind='kde')      # smooth density
df['Age'].skew()               # → ~0 almost normal
df['Age'].plot(kind='box')      # whiskers + outliers
df[df['Age'] > 65]              # extreme rows inspect
df['Age'].isnull().sum()/len(df['Age']) # missing %
```

Reading the Age `describe()` output

Run the cell, then read it like a sentence. `count 714` — not 891: some ages are missing, and that warning belongs in your head before any further calculation. `mean 29.70` against `median 25.0` — the mean sits comfortably above the middle, a first hint of right-skew: a long tail of older passengers pulling the average up. `min 0.42` is a baby measured in months; `max 80` is the oldest aboard. Between `25%` and `75%` lie the middle half of all passengers, and the boxplot that follows will draw exactly that band: the box from quartile to quartile, the whiskers to the extremes.

Why does this table matter? Because **summary statistics are the fastest filter on your assumptions**. If a column you trust looks wrong in `describe()`, you caught it before any plotting; if it looks right, you have licence to plot with confidence. Read every `describe()` as a contract between you and the column.

PYTHON

```
df['Age'].describe()
```

PYTHON

```
df['Age'].plot(kind='hist', bins=20)
```

Fare

conclusions

- The data is highly(positively) skewed
- Fare col actually contains the group fare and not the individual fare(This might be an issue)
- We need to create a new col called individual fare

Conclusions (Fare):

- The data is **highly positively skewed**.
- `Fare` actually contains the **group fare** (a whole family/ticket), not the *individual* fare. This might be an issue — we may need to create a new column called **individual fare**.

PYTHON

```
df['Fare'].describe()
```

PYTHON

```
df['Fare'].plot(kind='hist', bins=50)
plt.title('Distribution of Fare')
plt.show()
```

Deep Dive : Univariate — Fare (skewed)

- Fare highly positively skewed — few passengers huge fares
- mean pulled up by outliers; median robust
- hist spike near small values
- box long upper whisker + outlier dots
- 250+ fares — identify which cabins/classes
- skew value quantifies shape compress for model (log transform later)
- Missing: 0 NaN — completeness fine
- compare Age (normal) vs Fare (skewed) → treat differently
- insights: class separation visible in fare
- EDA conclusion: skewed needs log/quantile transform

PYTHON

```
df['Fare'].describe()           # mean >> median → right skew
df['Fare'].plot(kind='hist'); df['Fare'].plot(kind='kde')
df['Fare'].skew()              # strongly positive
df['Fare'].plot(kind='box')     # Long right whisker, outliers
df[df['Fare'] > 250]           # extreme fares (first class)
df['Fare'].isnull().sum()
```

Fare — a skewed, clumpy column

Fare is **Age**'s rowdy sibling: min **0**, max **512.33**, mean **32.20**, median **14.45** — a severe right-tail, the signature of a groups-of-passengers effect where one ticket buys several seats at once. Watch the analysis attack it from two sides. First, benchmarks over the means: parse the string **min**, **max**, **mean ± std** and reflect on the numbers instead of squinting. Second — the detective move — a split **by Survived**: line up survivor fares and non-survivor fares, and the medians alone begin to separate first-class wealth from steerage.

The deeper point about EDA: one number summarises, but **comparison divides into insight**. A lonely **describe()** shows skew; **groupby('Survived')['Fare'].describe()** shows *who* the skew belongs to. That single habit — always slicing the summary by the column you care about — is the engine of the whole bivariate half of this notebook.

Also watch for the zero-fare rows — a handful of records paid nothing — and notice how the analysis treats them as real data, not dirty data, because stripping them would quietly overstate the group means.

PYTHON

```
df['Fare'].describe()
```

PYTHON

```
df['Fare'].plot(kind='hist')
```

Steps of doing Univariate Analysis on Categorical columns

Descriptive Statistics: Compute the frequency distribution of the categories in the column. This will give a general understanding of the distribution of the categories and their relative frequencies.

Visualizations: Create visualizations to explore the distribution of the categories. Some common visualizations for categorical data include count plots and pie charts. These visualizations provide a visual representation of the distribution of the categories and can help identify any patterns or anomalies in the data.

Missing Values: Check for missing values in the data and decide how to handle them. Missing values can be imputed or excluded from the analysis, depending on the research question and the data set.

Conclusion: Summarize the findings of the EDA and make decisions about how to proceed with further analysis.

Survived

conclusions

- Parch and SibSp cols can be merged to form a new col call family_size
- Create a new col called is_alone

Deep Dive : Univariate — categorical cols

- value_counts — category counts (unique values)
- bar — easy count comparison
- pie — share of total proportional
- Missing check null count
- Embarked mostly 'S' Southampton — dominance
- Categorical analysis summary: dist + missing + counts
- verb الوصف: which category dominates
- Survived categorical — same approach below
- crosstab plots come at bivariate
- Explore each categorical column one by one

PYTHON

```
df['Embarked'].value_counts()           # frequency per category
df['Embarked'].value_counts().plot(kind='bar')
df['Embarked'].value_counts().plot(kind='pie', autopct='%0.1f%%')
df['Sex'].isnull().sum()                # missing check
```

Reading categorical counts

For a categorical column, the entire distribution fits in a table: `value_counts()` gives exact labels and counts, and the notebook adds proportion flavours (`normalize=True` , percentages) because probabilities often speak louder than raw heads. Then the bar and pie charts draw the same numbers visually. What should you *notice*? For `Survived` : about 342 survived against 549 did not — a base rate that every later slice must beat. For `Pclass` : third class is the majority, so the survival story you find later is *not* an artefact of group sizes alone.

The habit this section installs: **state the baseline before the breakdown**. Every cross-tab that follows (`Survived × Sex` , `Survived × Pclass`) is interesting only in *proportion to* these margins. Counts first, proportions second, baseline always in your peripheral vision.

One more reading habit: use proportions when comparing groups of very different sizes — the two hundred-odd passengers in first class versus the five hundred-odd in third are only comparable as percentages — and let `value_counts(normalize=True)` serve you accordingly.

PYTHON

```
df['Embarked'].value_counts()
```

PYTHON

```
df['Embarked'].value_counts().plot(kind='bar')
```

Steps of doing Bivariate Analysis

- Select 2 cols
- Understand type of relationship
 1. **Numerical - Numerical**
 - a. You can plot graphs like scatterplot(regression plots), 2D histplot, 2D KDEplots
 - b. Check correlation coefficient to check linear relationship
 2. **Numerical - Categorical** - create visualizations that compare the distribution of the numerical data across different categories of the categorical data.
 - a. You can plot graphs like barplot, boxplot, kdeplot violinplot even scatterplots
 3. **Categorical - Categorical**
 - a. You can create cross-tabulations or contingency tables that show the distribution of values in one categorical column, grouped by the values in the other categorical column.
 - b. You can plots like heatmap, stacked barplots, treemaps
- Write your conclusions

PYTHON

df

Deep Dive : Bivariate — crosstab percentages

- `crosstab(row, col, normalize='columns')` — proportion per column group
- $\times 100 \rightarrow$ percentage; heatmap annot visualize matrix
- Bivariate analysis of categorical vs categorical (Survived $\times$ Sex/Pclass)
- column-normalize $\rightarrow$ within each category what % survived
- KDE overlays — numeric vs categorical (Age by Survived)
- Compare survival across classes/sex — key insight: 1st class survives more
- Pattern: select 2 cols $\rightarrow$ type check $\rightarrow$ right viz
- categorical vs categorical $\rightarrow$ crosstab/heatmap/stacked bar
- categorical vs numerical $\rightarrow$ box/kde overlay
- numerical vs numerical $\rightarrow$ scatter/heatmap corr

PYTHON

```
pd.crosstab(df['Survived'], df['Sex'], normalize='columns')*100
sns.heatmap(pd.crosstab(df['Survived'], df['Pclass'], normalize='columns')*100, annot=True)
df[df['Survived']==1]['Age'].plot(kind='kde', label='Survived')
df[df['Survived']==0]['Age'].plot(kind='kde', label='Died')
```

Cross-tabs: the survival $\times$ class heat map

This is the pivot of the whole morning. `pd.crosstab(df['Pclass'], df['Survived'])` and — the crucial variant — `normalize='columns'`, which divides each column by its total so the heat map's cells read as *conditional probabilities*. The picture that emerges is sharp and defensible: first-class passengers survived at a far higher rate than third-class ones. Numerically: about 63% of first class survived versus roughly 25% of third — that gap, and not the raw counts, is the finding your analysis must carry forward.

Two analytical lessons hide here. First, **always normalise the direction that answers the question** — divide each column by its own sum when asking of that group, how many survived. Second, always sanity-check the base rate (`value_counts` from moments ago) against the crosstab margins — identical numbers, two different views. In this one image the whole story — women and children first, but also the rich had the deck space — starts to form.

PYTHON

```
sns.heatmap(pd.crosstab(df['Survived'],df['Pclass'],normalize='columns')*100)
```

PYTHON

```
pd.crosstab(df['Survived'],df['Sex'],normalize='columns')*100
```

Deep Dive : Engineered fare, deck, correlations

- individual_fare — fare divided by family size — fair bertoon
- fillna 'M' for missing cabin (Missing deck marker)
- deck letter feature — approximate cabin location
- deck×Pclass crosstab — who got which deck
- deck×Survived normalized stacked bar — survival by deck
- df.corr() heatmap — linear relations between numeric cols
- pairplot — multi univariate + scatter matrix overview
- Derived features validated by results
- Missing 77% Cabin — still useful via deck 'M'
- EDA iterative — each new feature challenges earlier views

PYTHON

```
df['individual_fare'] = df['Fare']/(df['SibSp']+df['Parch']+1) # per-person fare
df['individual_fare'].plot(kind='box'); df[['individual_fare', 'Fare']].describe()

df['Cabin'].fillna('M', inplace=True)
df['deck'] = df['Cabin'].str[0] # first letter = deck
pd.crosstab(df['deck'], df['Pclass'])
pd.crosstab(df['deck'], df['Survived'], normalize='index').plot(kind='bar', stacked=True)
sns.heatmap(df.corr()) # numeric correlation matrix
sns.pairplot(df1)
```

Deep Dive : Feature engineering — family & title

- Feature engineering = create NEW informative columns
- family_size = SibSp+Parch+1 — single-family dimension
- bucket into categories via apply(lambda/transform)
- crosstab family_type × Survived — new info surfaced
- surname — name split parsing
- title (Mr/Miss/Mrs/Master) — honorific extraction (0th word)
- rare titles → 'other' grouping (replace)
- title/ family predict survival — genuine feature
- individual_fare was fare / family size continuous variant
- engineering steps: define → code → test → evaluate crosstab

PYTHON

```
df['family_size'] = df['SibSp'] + df['Parch'] + 1
def transform_family_size(n):
    if n == 1: return 'alone'
    if n <= 4: return 'small'
    return 'large'
df['family_type'] = df['family_size'].apply(transform_family_size)
pd.crosstab(df['Survived'], df['family_type'], normalize='columns')*100

df['surname'] = df['Name'].str.split(',').str.get(0)
df['title'] = df['Name'].str.split(',').str.get(1).str.strip().str.split(' ').str.get(0)
df['title'] = df['title'].str.replace('Rev.', 'other')
```

Feature engineering: the family columns

Raw `SibSp` and `Parch` are thin proxies, so the notebook engineers their real signal. `SibSp.value_counts()` first — the raw counts of siblings/spouses — then the decisive idea: `family_size = SibSp + Parch + 1` (the plus one for the passenger themselves), and a `family_type` label built from a boundary choice: `Alone` when size 1, `Small` for a couple, `Medium`, `Large`. Narrow buckets made from a simple split are *the* pattern — an analyst converting a count into a category with one `pd.cut` or `np.where` and a threshold they can defend.

Why bother? A count column contributes one linear dimension; a labelled family type lets a heatmap or a groupby reveal *non-linear* structure — that families of moderate size had better survival odds than loners or huge parties. Engineering is not decoration: it is the deliberate creation of columns that make the pattern you suspect visible to the model that comes later.

PYTHON

```
df['SibSp'].value_counts()
```

PYTHON

```
df[df['Ticket'] == 'CA. 2343']
```

Bringing in the test set

At this point the analysis deliberately expands its surface: `pd.read_csv('/content/test.csv')` loads the held-out test passengers, and `pd.concat([train, test])` stacks both frames — the same columns, one after the other — so every future check runs against the *combined* universe of passengers. Two subtleties make this safe and honest. First, the concatenation is analytical only: the frames keep their identity, and the notebook re-separates along a `Dataset` column later, so no train/test boundary is corrupted. Second, `concat` needs the columns to align — which is exactly why the notebook checks the shapes and dtypes of both frames before and after. A combined step that goes unnoticed is a defect hiding in plain sight; this one is watched on both sides.

Use the moment to reinforce a habit about `concat`: it joins on the index and columns by default, not on a key, so mismatched column names silently produce extra `NaN` columns instead of an error. The dtypes check before and after exists precisely to catch that — read it as a template for every future concatenation.

PYTHON

```
df1 = pd.read_csv('/content/test.csv')
```

PYTHON

```
df = pd.concat([df,df1])
```

Individual fare — undoing the single-ticket trick

One of the dataset's quiet traps: a single ticket can cover several passengers, so the raw `Fare` per row is really a *shared* bill. The notebook repairs this by engineering `individual_fare = Fare / Ticket_Scores`, where `Ticket_Scores` counts how many passengers share each ticket number (`value_counts` on `Ticket`). The meaning shifts from what the ticket cost to what this passenger effectively paid — and the survival analysis that follows becomes far less distorted for large family groups.

The habit to steal is the *suspicion*: before trusting any raw column, ask what observation it actually describes at row level. When one ticket is shared by five rows, `Fare` describes the ticket, not the row — and a column that describes the wrong unit will quietly misdirect every chart and model built on it. Fix the unit, then plot.

And when you later cluster or model, remember the same repair: with shared-ticket passengers, `individual_fare` deserves a moment in the feature set alongside — or instead of — the raw `Fare`.

PYTHON

```
df['individual_fare'] = df['Fare']/(df['SibSp'] + df['Parch'] + 1)
```

PYTHON

```
df['individual_fare'].plot(kind='box')
```

Family size, engineered from both columns

Having folded `SibSp` and `Parch` together, the notebook slots `family_size` and its labelled `family_type` into the combined frame and immediately reframes the survival question through them. Run the groupby — the `Alone vs Small vs Medium vs Large` means — and watch which buckets sit above the baseline. The result lines up with what the earlier crosstab anticipated: **the middling families survive the best**, solos and oversized parties worse.

This is EDA at its most valuable: a claim that started as a hunch in a two-column `value_counts` is now a defensible, three-column conclusion. Notice the loop — *engineer, group, compare against the base rate* — because it will recur for `titles`, `deck`, and every other feature the notebook invents before you.

Store the exact comparison in your notes as you go: the survivor mean for each `family_type`, side by side with the overall survival baseline. That single table is the session's cleanest demonstration that engineered features exist to *expose* what raw columns keep hidden.

PYTHON

```
df['family_size'] = df['SibSp'] + df['Parch'] + 1
```

PYTHON

```
# family_type
# 1 -> alone
# 2-4 -> small
# >5 -> large

def transform_family_size(num):

    if num == 1:
        return 'alone'
    elif num>1 and num <5:
        return "small"
    else:
        return "large"
```

The correlation heatmap — everything at once

The finale compresses the entire dataset into one grid: `df.corr()` computes the Pearson correlation between every pair of numeric columns, and `sns.heatmap(..., annot=True, cmap='RdYlGn')` colours the result — warm for strong positive, cool for strong negative, near-white for none. The `Survived` row is the report card: which features (class, fare, family membership, sex...) move survival at all, and in which direction.

Read it with three eyes. **Magnitude** — above ~0.3 is a genuine signal for tabular data; 0 hurts. **Direction** — the sign tells you the story. **Redundancy** — two features that correlate with each other almost perfectly are telling the model the same thing twice. A heatmap is not a painting; it is a prioritised to-do list for the modelling session that follows this notebook.

Two caveats to keep at hand: correlation is symmetric and linear — a strong curved relationship can score near zero — and rare values mislead everything, so pair the grid with a scatter or a groupby when a number looks too good to be true.

PYTHON

```
sns.heatmap(df.corr())
```

PYTHON

```
sns.pairplot(df1)
```

Data Cleaning Case Study — Smartphones

Data Assessing

Quality Issues

1. **model** - some brands are written differently like OPPO in model column `consistency`
2. **price** - has unnecessary '₹' `validity`
3. **price** - has ',' between numbers `validity`
4. **price** - phone Namotel has a price of 99 `accuracy`

5. **ratings** - missing values completeness
6. **processor** - has some incorrect values for some samsung phones(row # -642,647,649,659,667,701,750,759,819,859,883,884,919,927,929,932,1002) validity
7. There is ipod on row 756 validity
8. **memory** - incorrect values in rows (441,485,534,553,584,610,613,642,647,649,659,667,701,750,759,819,859,884,919,927,929,932,990,1002) validity
9. **battery** - incorrect values in rows(113,151,309,365,378,441,450,553,584,610,613,630,642,647,649,659,667,701,750,756,759,764,819,855,85) validity
10. **display** - sometimes frequency is not available completeness
11. **display** - incorrect values in rows(378,441,450,553,584,610,613,630,642,647,649,659,667,701,750,759,764,819,859,884,915,916,927,929,93) validity
12. certain phones are foldable and the info is scattered validity
13. **camera** - words like Dual, Triple and Quad are used to represent number of cameras and front and rear cameras are separated by '&'
14. **camera** - problem with rows (100,113,151,157,161,238,273,308,309,323,324,365,367,378,394,441,450,484,506,534,553,571,572,575,584,61)) validity
15. **card** - sometimes contains info about os and camera validity
16. **os** - sometimes contains info about bluetooth and fm radio validity
17. **os** - issue with rows (324,378) validity
18. **os** - sometimes contains os version name like lollipop consistency
19. missing values in camera, card and os completeness
20. datatype of price and rating is incorrect validity

Tidiness Issues

1. **sim** - can be split into 3 cols *has5g*, *hasNFC*, *hasIRBlaster*
2. **ram** - can be split into 2 cols RAM and ROM
3. **processor** - can be split into processor name, cores and cpu speed.
4. **battery** - can be split into battery capacity, *fastchargingavailable*
5. **display** - can be split into size, resolution*width*, *resolutionheight* and frequency
6. **camera** - can be split into front and rear camera
7. **card** - can be split into supported, *extended_upto*

PYTHON

```
df.info()
```

The numerical overview

`df.describe()` is the fastest way to get a feel for an entire frame in one glance, but read it like an auditor, not a tourist. The very first thing to notice is **the count column**: `describe()` shows `rating` with only 879 non-null values while the earlier `info()` said the frame holds 1020 rows — that gap of 141 is your first missingness sighting, before any verso of the cleaning even begins.

Then read the health of each numeric column. The rating stats look sane (mean 78.3, spread 7.4, range 60 to 89) — a reasonable phone-score scale. The moment a mean sits close to the max or min, or a percentile jumps absurdly, you have a signal that a column hides bad values — the same instinct that will later flag the price-99 junk row that the cleaning pass removes.

Hold this frame in your head as the *before* baseline. Every cleaning decision later in the notebook is judged against it: the goal is a tidy frame whose describe (and info) is finally honest.

PYTHON

```
df.describe()
```

PYTHON

```
df.duplicated().sum()
```

Working copies and price cleaning

This cell sets the tone for the whole case study. `df1 = df.copy()` creates a working copy so the original `df` stays untouched as an audit trail — every mutation from now on hits `df1`, and any doubt sends you back to the pristine original. The "original is sacred, fixes go to a copy" habit is the first rule of serious cleaning.

The price fix is a classic three-part pattern: **parse** → **clean** → **cast**. `str.replace('₹', '')` removes the rupee symbol, `str.replace(',', '')` strips the thousand separators, and `astype('int')` turns the remaining string into a real number. All of it runs through the vectorised `.str` accessor, so it is fast and loop-free.

Then comes a detail that marks careful work: `reset_index()` followed by `+ 2` renumbers the rows to match the *raw file's* line numbers. That tiny offset means any fix you make later can be cross-referenced against the original CSV exactly — when a school of analysts shares one file, row numbers are the common language.

Deep Dive : Battery, display, camera parse

- battery "5000 mAh Lithium Polymer with 33W Fast Charging" → capacity+charge
- 'with' separator — split segments
- camera_extractor: 'Quad'→4, camera count from '&'
- display '6.4 inches, 1080x2400, 90Hz' → size/res/refresh
- `.apply(function)` for logic-based extraction
- split pipeline standardised
- outputs typed numeric
- unify units (mAh, inches, MP)
- camera final 'Foldable Display' → '50 MP' overwrite

- data now analysis-ready

PYTHON

```
battery_capacity = df1['battery'].str.split('with').str.get(0)...findall digits
fast_charging = df1['battery'].str.split('with').str.get(1)...
screen_size = df1['display'].str.split(',').str.get(0).str.split().str.get(0)
resolution = df1['display'].str.split(',').str.get(1)...
num_rear = df1['camera'].str.split('&').str.get(0).apply(camera_extractor)
```

Deep Dive : Feature extraction — brand + flags

- brand = first token of model name — extract
- insert(pos) — control column placement
- str.lower — uniform format
- sim col '5G,NFC' — boolean per feature
- flags: booleans from string patterns
- feature engineering — new structured columns
- split; get(0) — token technique
- readable column structure & head
- linear transform cost cheap
- multiple booleans from same source

PYTHON

```
brand_names = df1['model'].str.split(' ').str.get(0)
df1.insert(1, 'brand_name', brand_names) # pos 1
df1['brand_name'] = df1['brand_name'].str.lower()
has_5g = df1['sim'].str.contains('5G')
has_nfc = df1['sim'].str.contains('NFC')
df1.insert(6, 'has_5g', has_5g) # boolean flags
```

Deep Dive : Price cleaning — text to numeric

- copy before any mutation — audit trail
- price stored as string with ₹ + commas
- str.replace chains remove symbols
- astype('int') — convert to numeric
- reset_index — explicit index for row-ofile correspondence
- +2 offset matches raw CSV row numbers → easy cross-ref
- general pattern: parse currency → clean → cast
- keeps original df for reference
- Cleaning rule: one type per column
- .str accessor vectorized — no loops

PYTHON

```
# make a copy
df1 = df.copy()
```

PYTHON

```
df1['price'] = df1['price'].str.replace('₹', '').str.replace(',', '').astype('int')
```

Row-level Anomalies

- Specific row indices are tracked as sets
- `isin` — checks those rows
- Inconsistent rows = rows where one feature switched
- `|` union, `&` intersection — set combinations

PYTHON

```
processor_rows = {642, 647, ...} # known bad rows
ram_rows, battery_rows, display_rows, camera_rows = ...
df1[df1['index'].isin(processor_rows | ram_rows | ...)]
```

Deep Dive : Row-level anomaly inspection

- Manual assessment found suspicious rows per column
- `isin()` + set union/intersection — target specific rows
- Filter price ≥ 3400 to drop garbage entries
- Each row examined before drop decision
- Assessment first, THEN cleaning
- Sets of index show categories of issues
- Combine masks for overlapping anomalies
- Evidence-based row removal
- Bucket: processor misaligned, missing camera etc
- Cleaning = surgical targeted operations

PYTHON

```
processor_rows = set((642, 647, 649, 659, 667, 701, 750, 759, 819, 859, 883, 884, 919, 927, 929, 932, 1002))
ram_rows = set((441, 485, 534, 553, 584, 610, 613, 642, 647, 649, 659, 667, 701, 750, 759, 819, 859, 884, 919, 927, 929, 932, 991))
battery_rows = set((113, 151, 309, 365, 378, 441, 450, 553, 584, 610, 613, 630, 642, 647, 649, 659, 667, 701, 750, 756, 759, 764, 819, 859, 884, 915, 919))
display_rows = set((378, 441, 450, 553, 584, 610, 613, 630, 642, 647, 649, 659, 667, 701, 750, 759, 764, 819, 859, 884, 915, 919))
camera_rows = set((100, 113, 151, 157, 161, 238, 273, 308, 309, 323, 324, 365, 367, 378, 394, 441, 450, 484, 506, 534, 553, 571))
```

PYTHON

```
df1[df1['index'].isin(processor_rows | ram_rows | battery_rows | display_rows | camera_rows)]
```

Dropping Bad Rows

- Known bad rows are dropped
- `inplace=True` — modifies in place
- Later verification — clean confirmation

PYTHON

```
df1.drop([645, 857, 882, 925], inplace=True) # duplicate/wrong
df1[df1['index'].isin(ram_rows)]           # verify cleared
```

Deep Dive : Dropping & repairing bad rows

- drop by index label - exact targeted
- inplace - mutate working copy
- Sequential: check each category then fix/drop
- Few rows dropped, most FIXED (shift/relocate)
- Grader approach: never drop wholesale
- Each broken row analysed individually
- better than dropping complete rows where salvageable
- records bad data in column vs row
- Data quality minimal loss preserving
- document reasons for each drop

PYTHON

```
df1.drop([645,857,882,925], inplace=True) # few bad rows
df1[df1['index'].isin(ram_rows)]           # check next category
df1.drop(582, inplace=True)
df1.drop([376,754], inplace=True)
```

Targeted drops — precision over panic

This cell executes its first repair: `df1.drop([645, 857, 882, 925], inplace=True)` removes exactly four labelled rows — and no more. Each index was vetted in the assessment (`index` column matching the raw file's row numbers), so this is a surgical strike decided by evidence, not a `dropna()` scattergun.

Two habits on display are worth stealing. First, **verify after every drop**: the very next cell re-queries the same category (`isin(ram_rows)`) to confirm the offending rows are truly gone and nothing else moved. Second, **prefer repair to removal** — the notebook drops a handful of genuinely bad rows, but the *majority* of broken rows are fixed by relocating values, because data that can be saved should be saved. Wholesale row-dropping is the grader's anti-pattern: cheap, lazy, and quietly throws away salvageable information.

Notice also the sequencing: categories are worked one at a time — drop for one defect, verify, then the next. Cleaning that proceeds in auditable slices is cleaning you can defend afterwards.

PYTHON

```
df1.drop([645,857,882,925],inplace=True)
```

PYTHON

```
df1[df1['index'].isin(ram_rows)]
```

Shift — Fixing Shifted Columns

- Sometimes values are shifted by one column
- `shift(1, axis=1)` — shifts one column to the right
- `loc` updates only the targeted rows
- Fixes the alignment of shifted data

PYTHON

```
temp_df = df1[df1['index'].isin(battery_rows)]
x = temp_df.iloc[:, 7:].shift(1, axis=1).values
df1.loc[temp_df.index, temp_df.columns[7:]] = x
```

Deep Dive : Shift — realigning shifted columns

- Some rows' columns shifted RIGHT (missing one value) → blank in col7
- `iloc[:,7:]` — tail columns only
- `.shift(1,axis=1)` — move each value ONE column right
- assign back via `.loc rows×cols` — alignment applies subset
- Realigns misplaced data back to correct columns
- Timestamp columns corrupted — horizontal resolve
- Vectorized whole block one call
- Data location repair preserves info
- multi-row batch pattern
- key insight: values exist, just wrong place

PYTHON

```
temp_df = df1[df1['index'].isin(battery_rows)]
x = temp_df.iloc[:,7:].shift(1, axis=1).values # shift right
df1.loc[temp_df.index, temp_df.columns[7:]] = x
```

The shift realignment pattern

Some of this dataset's rows are not wrong — they are merely *one place to the left*. When a phone's text was assembled a cell short, everything after that gap slid right by one column, which is exactly what the row indices captured during assessment. This cell shows the pandas idiom that repairs it in one shot.

`temp_df = df1[df1['index'].isin(battery_rows)]` isolates the broken rows. `temp_df.iloc[:, 7:].shift(1, axis=1)` takes the affected tail columns and moves every value one column *right*, opening a blank in the first slot where the missing value will live. Then `df1.loc[temp_df.index, temp_df.columns[7:]] = x` writes the shifted block back into place — `.loc` aligning rows and columns so only the intended subset changes.

The key insight: **the values exist, they are just in the wrong position**. Realigning is not guesswork, and it is vectorised — one block operation repairs the whole batch. Once you have seen `shift(1, axis=1)` used this way, you will recognise the same move in later cells for camera and processor columns.

PYTHON

```
temp_df = df1[df1['index'].isin(battery_rows)]
```

PYTHON

```
x = temp_df.iloc[:, 7:].shift(1, axis=1).values
```

Camera/OS Column Confusion

- Some "card" values leaked into the camera column
- Condition `~contains('MP')` — remove the camera rows
- Swap values: card → camera, card → default
- Clears the name mismatch

PYTHON

```
temp_df = df1[df1['index'].isin(camera_rows)]
temp_df = temp_df[~temp_df['camera'].str.contains('MP')]
df1.loc[temp_df.index, 'camera'] = temp_df['card'].values
df1.loc[temp_df.index, 'card'] = 'Memory Card Not Supported'
```

Deep Dive : Column confusion — camera/card/os

- Category rows had values swapped: camera text in 'card' etc
- `~str.contains('MP')` — locate misplaced rows
- copy card value back into camera column
- 'MP' in card → still wrong → overwrite standard string
- `value_counts` verify each column after repairs
- Multiple columns confused: os held memory card text
- systematic recount & reclean each
- pattern: shift block → recount → reassign
- data dictionary restored
- Result single clean layout

PYTHON

```
temp_df = temp_df[~temp_df['camera'].str.contains('MP')]
```

PYTHON

```
df1.loc[temp_df.index, 'camera'] = temp_df['card'].values
```

value_counts — the misalignment detector

The cheapest way to find a wrong value is to ask the column to list what it actually contains:

`df['os'].value_counts()` prints every distinct OS string with its count. Clean columns look tidy — a handful of `Android v12`, `Android v11`, `Android v9.0 (Pie)` buckets. Suspicious columns leak a comic offender in full view: a lone `Bluetooth` sitting among the operating systems.

That is precisely what happens before this cell: the `card` repairs finish with a clean count, and the next audit of `os` reveals stale tokens that belong nowhere near an OS column. The misfits are then set to `np.nan` and re-counted, proving the repair.

Hold onto the reflex: **`value_counts()` is the first tool for spotting misalignment, and `np.nan` is the honest way to clear a wrong value you cannot fix.** A categorical column that cannot be enumerated cleanly in one printout is a categorical column you have not finished cleaning.

OS Column Cleanup

- `value_counts()` — spots weird values (Bluetooth)
- Wrong values → NaN
- `value_counts` — the first tool for spotting misalignment

PYTHON

```
df1['os'].value_counts()
```

PYTHON

```
temp_df = df1[df1['os'] == 'Bluetooth']
```

Feature Extraction — Brand

- `str.split(' ').str.get(0)` — first word is the brand
- `insert(loc, name, series)` — specify the position
- `contains('5G')` → boolean features
- Derived: `has5g`, `hasnfc`, `hasirblaster`

PYTHON

```
brand_names = df1['model'].str.split(' ').str.get(0)
```

PYTHON

```
df1.insert(1, 'brand_name', brand_names)
```

Processor Splitting

- Multi-attribute column → split on commas
- `.str.strip()` — trims whitespace
- `.str.replace('Octa Core Processor', 'Octa Core')` — normalize
- 3 new features from 1 column

PYTHON

```
df1['processor'].str.split(',').str.get(0) # name
df1['processor'].str.split(',').str.get(1) # cores
df1['processor'].str.split(',').str.get(2) # speed
```

Deep Dive : Processor splitting — 3 fields

- processor packed "MediaTek Helio G88, Octa Core, 2.2 GHz"
- `split(',') → parts → three new columns`
- `.get(i)` each part
- strip whitespace each
- Core-check case: fields mis-aligned → use temp align
- `loc[rows, [cols]] = temp values` — batch assign
- extract pattern reusable: `split → get → strip → cast`
- parse structured text block once
- insert full new columns at positions
- Standardise numeric cores: 'Octa Core' → 8

PYTHON

```
processor_name = df1['processor'].str.split(',').str.get(0)
```

PYTHON

```
num_cores = df1['processor'].str.split(',').str.get(1)
```

RAM / Memory

- `findall(r'\b(\d+)\b')` — extract the numbers
- `.astype(float)` — convert to numeric
- Clean numeric values are useful
- 512 MB = 0.5 → convert using 1024 (fix edge values)

PYTHON

```
ram_capacity = df1['ram'].str.split(',').str.get(0).str.findall(r'\b(\d+)\b').str.get(0)
```

Deep Dive : RAM / internal memory parsing

- 'ram' = "8 GB, 128 GB" → capacity + memory
- regex `\b(\d+)` — extract first number from text
- `astype(float)` — numeric
- manual row fixes where parsing fails
- 1 GB → 1024 (brand used '1' meaning 1TB or the rounding) — data fix
- `findall + get(0)` — pull number
- `value_counts` sanity check
- two derived numeric cols from one text
- hidden weekend data decoded
- thorough cleaning ends typed data

PYTHON

```
ram_capacity = df1['ram'].str.strip().str.split(',').str.get(0).str.findall(r'\b(\d+)\b').str.get(0)
```

PYTHON

```
df1.insert(16, 'ram_capacity', ram_capacity)
```

Battery & Fast Charging

- `split('with')` — separate capacity from charging
- `fast_charging_extractor` — custom list-to-int conversion
- `.apply(extractor)` — custom cleaning function
- Non-uniform formats handled with custom logic
- `-1` = no fast charging → flag 0/1
- `apply(row-based)` — custom column derive
- charging watts leftover → NaN for absent

PYTHON

```
battery_capacity = df1['battery'].str.strip().str.split('with').str.get(0).str.strip().str.findall(r'\b(\d+)\b')
```

PYTHON

```
df1.insert(16, 'battery_capacity', battery_capacity)
```

Display splitting

The display column packs three independent facts into one string: `"6.4 inches, 1080x2400, 90Hz"`. This cell splits it into three clean features — `screen_size`, `resolution`, `refresh_rate` — by reading the string in three lanes: `split(',')` separates the segments, `get(0)` / `get(1)` / `get(2)` target each one, and further splits on spaces and the literal `px` trim the units.

Two extra touches show how parsing gets real. The refresh-rate extraction reaches for a regex — `\d{2,3}` — grabs the two-to-three-digit number — because "90 Hz", "120Hz" and "60Hz" all surface it the same way. And the size is cast straight to `float` so it is numeric immediately, not left as text that plotting would choke on.

The transferable idea is the *split* → *get* → *strip* → *cast* pipeline: it appears for display, and it will reappear for battery, camera, processor and RAM, because a column that hides several columns behind commas is everywhere in real data.

PYTHON

```
screen_size = df1['display'].str.strip().str.split(',').str.get(0).str.strip().str.split(' ').str.get(0).a
```

PYTHON

```
df1.insert(21, 'screen_size', screen_size)
```

Camera count extraction

The camera column stores the *number* of cameras as English words — `Quad`, `Triple`, `Dual` — followed by megapixel details, with front and rear counts separated by an ampersand. Regex and `split` alone are too blunt, so this cell introduces a small **helper function**: `camera_extractor` maps each keyword to a digit (`Quad` → `'4'`, `Triple` → `'3'`, `Dual` → `'2'`, otherwise `'1'`), and `.apply()` runs it against every value.

Notice the two structural decisions. Rear and front counts are separated first with `split('&')`, so each lane is extracted independently. And `fillna('Missing')` fills the missing rear-camera rows with a sentinel the extractor understands — missingness handled at parse time, not discovered later.

The lesson is that **some cleaning needs a naming function, not a lookup**: a custom, testable function that encodes the domain logic (what the words mean) beats a monster one-liner, and `.apply()` keeps it vectorised over the whole column in one pass.

PYTHON

```
def camera_extractor(text):
    if 'Quad' in text:
        return '4'
    elif 'Triple' in text:
        return '3'
    elif 'Dual' in text:
        return '2'
    elif 'Missing' in text:
        return 'Missing'
    else:
        return '1'
```

PYTHON

```
num_rear_cameras = df1['camera'].str.strip().str.split('&').str.get(0).apply(camera_extractor)
```

Individual camera MP

- Split MP number
- Rear + front primary megapixel
- Nested splits — unicode spaces `\u2009` normalize

PYTHON

```
df1['primary_camera_rear'] = df1['camera'].str.split(' ').str.get(0).str.replace('\u2009', ' ').str.split(' ')
```

PYTHON

```
df1['primary_camera_front'] = df1['camera'].str.split('&').str.get(1).str.strip().str.split(' ').str.get(0)
```

Lambda-powered feature extraction

`extended_memory` is carved out of the `card` column with a single wall of pandas: `df1['card'].apply(lambda x: '0' if 'Not' in x else x.split('upto')).str.get(-1).str.strip().str.replace('Memory Card Supported', 'Not Specified')`.

Read it right to left. The lambda decides the sentinel first: if 'Not' appears in the card text, the phone has no expandable storage and yields `'0'`; otherwise the value is split on 'upto' and `.get(-1)` takes the *maximum* capacity mentioned. Then `.str.strip()` cleans whitespace and a final `.str.replace()` settles one known anomaly. All of it runs through the vectorised `.str` accessor, so the whole column transforms in one statement.

The wider lesson is **extraction is a pipeline of tiny steps, not one clever regex**: branch on a sentinel, split to a usable part, strip, replace. Name each step's role and the whole chain stays readable, repeatable, and reviewable. The pattern also scales to the remaining card and network fields: once proven on one column, repeat the same build for the rest and verify each result with the same `value_counts` sweep you already trust.

PYTHON

```
df1['extended_memory'] = df1['card'].apply(lambda x: '0' if 'Not' in x else x.split('upto')).str.get(-1).str
```

PYTHON

```
df1.head()
```

Final export

- Unused/deprecated columns drop
- Clean tidy table → CSV
- `index=False` — save
- Case study complete: assess → clean → export

PYTHON

```
export_df = df1.drop(columns=['index', 'sim', 'processor', 'ram', 'battery', 'display', 'camera', 'card'])
```

PYTHON

```
export_df.to_csv('smartphone_cleaned_v2.csv', index=False)
```

EDA Case Study — Smartphones

Load & overview

- Cleaned data (previous case study) load
- `info` + `isnull().sum()` — missing mapping
- `max_columns/max_rows` set — full view

PYTHON

```
df.shape
```

PYTHON

```
df.head()
```

Brand distribution

- `value_counts` → brand share
- Bar + pie — categorical univariate
- Brand market dominance is visible
- `nunique` — count of models

PYTHON

```
df['brand_name'].value_counts().head(10).plot(kind='bar')
df['brand_name'].value_counts().plot(kind='pie', autopct='%0.1f%%')
```

Deep Dive : Brand distribution — categorical EDA

- value_counts → frequency per brand
- bar head(10) — top brands by listings
- pie — proportion share of market
- missing check — brand completeness
- Identify dominant players
- Categorical EDA overview step
- decide top-N filtering later for plots
- compare listing counts vs prices higher-level
- Cross checks nunique for model versions
- market structure understanding

PYTHON

```
# plot a graph of top 5 brands
df['brand_name'].value_counts().head(10).plot(kind='bar')
```

PYTHON

```
# pie chart
df['brand_name'].value_counts().plot(kind='pie', autopct='%0.1f%%')
```

Price distribution

- Hist + KDE — shape
- Positive skew — few flagship phones
- Boxplot — outliers visible
- Inspect the outlier rows

PYTHON

```
sns.displot(kind='hist', data=df, x='price', kde=True)
df['price'].skew() # right-skew = long tail
sns.boxplot(df['price'])

df[df['price'] > 250000] # outliers
```

Deep Dive : Price distribution — skew & outliers

- hist + kde — shape check
- skew>0 — right tail few luxury phones

- box — Q1-Q3 spread, outliers hurt
- filter >250000 — inspect premium segment
- mean >> median in skewed data
- Decide: winsorize / log for modeling
- perspective: majority budget phones
- note for each col missing
- EDA pipeline per column: describe, skew, box, outliers, missing

PYTHON

```
sns.displot(kind='hist',data=df,x='price',kde=True)
```

PYTHON

```
df['price'].skew()
```

Rating distribution

- Ratings 3.2-4.8 band
- Left skew perhaps — high rating common
- Compute null ratio — to decide which feature needs filling

PYTHON

```
df['rating'].describe()
df['rating'].isnull().sum() / 980 # null ratio
```

Deep Dive : Rating distribution — left skew

- rating mostly 4.0-4.5 — overall good products
- negative skew — left tail low-rated
- box tight — concentrated values
- missing 2% — small, decide dropna/fill median
- describe — mean std quantiles
- informative: market product quality healthy
- each distribution summarized beside
- This drives feature decisions

PYTHON

```
sns.displot(kind='hist',data=df,x='rating',kde=True)
```

PYTHON

```
df['rating'].skew()
```

Feature-wise pies

- Boolean + categorical — pie plans
- Market standard (5G common? NFC?)
- Shows feature adoption
- Each column's distribution — quick visual sweep

PYTHON

```
df['has_5g'].value_counts().plot(kind='pie', autopct='%0.1f%%')
df['processor_brand'].value_counts().plot(kind='pie')
df['ram_capacity'].value_counts().plot(kind='pie')
```

Deep Dive : Feature-wise pies — booleans & specs

- boolean features then pie proportion keeping
- each spec col its own pie
- cross-tab: which brands include IR blaster
- preprocessing distribution awareness
- spec adoption by brands - strategic
- count vs availability
- aggregated df + category proportions
- enables network between feature and brand
- quick trend: 5G now standard
- foolproof categorical EDA snapshots

PYTHON

```
# has_5g
df['has_5g'].value_counts().plot(kind='pie', autopct='%0.1f%%')
```

PYTHON

```
# has_nfc
df['has_nfc'].value_counts().plot(kind='pie', autopct='%0.1f%%')
```

Loop plotting all numerics

- `select_dtypes(numeric)` — numeric cols
- Function + loop = multi-plot
- Distribution of all columns at once
- EDA quick sweep technique

PYTHON

```
def plot_graphs(col):
    sns.displot(kind='hist', kde=True, data=df, x=col)

num_columns = df.select_dtypes(include=['float64', 'int64']).iloc[:, [3,4,6,9,13,14,16]].columns
for col in num_columns:
    plot_graphs(col)
```

Deep Dive : Loop-plotting all numeric cols

- select_dtypes — pick numeric columns automatically
- iloc subset — interesting cols only
- loop each — automatic distribution plots
- reuse single plot function — DRY
- One glance all numeric distributions
- catches hidden patterns & scale issues
- cols: price, rating, processor_speed... etc
- EDA automation pattern higher productivity
- inspect each afterward manually
- figure titles update per loop

PYTHON

```
def plot_graphs(column_name):
    sns.displot(kind='hist', kde=True, data=df, x=column_name, label=column_name)
    sns.catplot(kind='box', data=df, x=column_name)
```

PYTHON

```
num_columns = df.select_dtypes(include=['float64', 'int64']).iloc[:, [3,4,6,9,13,14,16]].columns
```

Brand vs price

- Bar plot — brand avg price
- Few brands → cluttered bars
- Filter `count > 10` — meaningful brands
- Rotation — legible labels

PYTHON

```
sns.barplot(data=df, x='brand_name', y='price')
x = df.groupby('brand_name').count()['model']
temp_df = df[df['brand_name'].isin(x[x > 10].index)]
```

Deep Dive : Brand vs price — barplot & filters

- barplot — avg price per brand (error bars default)
- max_width brands many tiny bars not readable
- filter: groupby count model > 10 - meaningful comparison
- groupby().count()['model'] - per brand listing count
- isin() - subset relevant brands
- redesigned figsize clean bars
- mean price reveals brand tiers (Apple premium)
- error bars show variance
- decision: focus analysis brands with data
- EDA selective pruning for signal

PYTHON

```
plt.figure(figsize=(20,10))
sns.barplot(data=df,x='brand_name',y='price')
plt.xticks(rotation='vertical')
```

PYTHON

```
x = df.groupby('brand_name').count()['model']
```

Bivariate — numeric relations

- Scatter — rating vs price relationship
- `estimator=np.median` — outliers resilient
- Feature → price uplift (median)
- Categorical vs numeric via bar/point

PYTHON

```
sns.scatterplot(data=df, x='rating', y='price')
sns.barplot(data=temp_df, x='has_5g', y='price', estimator=np.median)
sns.pointplot(data=temp_df, x='has_nfc', y='price', estimator=np.median)
```

Deep Dive : Bivariate relations — specs vs price

- scatter — numeric×numeric correlation view
- estimator=median — robust central price per category
- barplot/pointplot bool×price — price premium features
- crosstab cores×os — joint composition
- processor brands → price banding
- screen size scatter — size vs cost
- render filled insights: 5G adds, NFC adds...

- sanity check domain expectations
- both numeric combos & categorical×num

PYTHON

```
sns.scatterplot(data=df, x='rating', y='price')
```

PYTHON

```
sns.barplot(data=temp_df, x='has_5g', y='price', estimator=np.median)
```

Cross-tab

- 2 categorical values table
- Combinations count
- Trend: cores vs os pairing

PYTHON

```
pd.crosstab(df['num_cores'], df['os'])
```

PYTHON

```
pd.crosstab(df['num_cores'], df['os'])
```

Scatter — the continuous relationship

Two barplots just compared price across brands and core counts; the scatterplots that follow ask the *continuous* question — does price climb smoothly with processor speed, and does it climb with screen size?

`sns.scatterplot(data=df, x='processor_speed', y='price')` draws every row as a point, so you can see both the trend and the dispersion around it.

Read a scatter like a detective. A clear upward drift means the variable genuinely prices into the phone; a cloud with no shape means it does not. Also look at the *edges*: the points hugging the top are the flagship phones (OnePlus, Samsung Ultra), and whether they cluster at the high end of the speed axis tells you if premium speed is a real driver or just a label that travels with premium phones.

The takeaway beats bar comparisons for continuous variables: a bar gives you an average, but a scatter gives you the relationship and its noise. When the two agree — bar shows higher medians, scatter shows an upward drift — you have a bivariate finding worth writing down.

PYTHON

```
sns.scatterplot(data=df, x='processor_speed', y='price')
```

PYTHON

```
sns.scatterplot(data=df, x='screen_size', y='price')
```

Correlations

- Correlation of every numeric column with price
- Positive → price moves up with it
- `corr()` NaN issue — after KNN fill

PYTHON

```
df.corr()['price']
df.corr()['rating']
```

Deep Dive : Correlations & missing imputation

- `corr()['price']` — strongest price drivers (screen, storage)
- missing pattern — which cols need impute
- `KNNImputer` — fill missing using 5-nearest rows
- `fit_transform` numeric matrix
- after: `corr` re-evaluate — compare before/after merge
- dummy variables → correlations categorical (*getdummies dropfirst*)
- preprocessing: impute first, then correlate clean
- Sklearn integration in EDA
- missing-aware correlations trustable
- pipeline: `corr` baseline → impute → `corr` updated

PYTHON

```
df.corr()['price']
```

PYTHON

```
df.isnull().sum()
```

KNN imputer

- `KNNImputer` — missing values neighbor average
- Numeric cols only, drop target
- Fill, then re-compare correlations
- `.corr()` is now NaN-free

PYTHON

```
# knn imputer
df.shape
```

PYTHON

```
x_df = df.select_dtypes(include=['int64', 'float64']).drop(columns='price')
```

Dummy + correlation

- Categorical → binary dummies
- `drop_first` — avoid collinearity
- Price relation of each category
- EDA → feature insight → model prep

PYTHON

```
pd.get_dummies(df, columns=['brand_name', 'processor_brand', 'os'], drop_first=True).corr()['price']
```

PYTHON

```
pd.get_dummies(df, columns=['brand_name', 'processor_brand', 'os'], drop_first=True).corr()['price']
```

EDA Case Study — Smartphones (v3 / v4 / v5)

The missingness audit

`df.isnull().sum()` is the first interrogator after `info()`, and it tells you exactly where the cleaned file still leaks: `rating` is missing in 101 rows out of 980 — more than ten percent of the frame — while every other column reports zero. That is the whole reason a `KNNImputer` will appear much later in this notebook: the ratings are absent for a large slice of phones, and skipping those rows would quietly weaken every rating-focused chart.

Read the output like an audit, not a chore. A single column with a big gap, versus a whole frame spattered with small ones, points to different root causes and different fixes — and both require counting before cleaning. Also note what is *not* in the list: the engineered flags are complete, because they were built from string patterns that either matched or not, no empty cells permitted.

Hold this 101 figure in your memory. When the correlation step recomputes after imputation, the before-and-after shift of the ratings is the measurable payback of this entire audit.

PYTHON

```
df.isnull().sum()
```

Brand overview

- Brand share univariate
- Bar for top 10
- Missing value check: `.isnull().sum()`

PYTHON

```
# brand name
df['brand_name'].value_counts()
```

PYTHON

```
df['brand_name'].value_counts()
df['brand_name'].value_counts().head(10).plot(kind='bar')
```

Deep Dive : Brand overview & price spread

- count brands — market prevalence
- bar top 10 visualize
- price heavy right skew (high-end few)
- describe tells mean vs median gap
- box ignores >200k — focus bulk range
- premium segment isolated separately
- skewed → log transform later
- outliers legit categories (foldables)
- missing rating/price count check
- layer by layer — each column its own analysis

PYTHON

```
# top 10 phone brands
df['brand_name'].value_counts().head(10).plot(kind='bar')
```

PYTHON

```
df['brand_name'].value_counts() # Listings per brand
df['brand_name'].value_counts().head(10).plot(kind='bar')
df['price'].describe()
sns.histplot(df['price'], kde=True)
df['price'].skew()
sns.boxplot(df[df['price']<200000]['price'])
df[df['price']>200000] # ultra-premium phones
```

Price

- Hist + kde — skew right
- Outliers: 2M+ pricing suspicious → rows check
- `boxplot` subset — outliers visibility

PYTHON

```
# price col
df['price'].describe()
```

PYTHON

```
sns.histplot(df['price'],kde=True)
```

Rating

- Ratings narrow band, slight left skew
- Nulls — later KNN impute

PYTHON

```
df['rating'].describe()  
sns.histplot(df['rating'], kde=True)  
sns.boxplot(df['rating'])  
df['rating'].isnull().sum()
```

Deep Dive : Rating distribution — core metric

- describe: 4.x mean — healthy product ratings
- hist near 4 — slight left skew (few bad)
- box tight — low variability
- null count — handle missing
- rating = perceived quality proxy
- feature for price-benchmark analysis
- distribution normality check
- ratings clustered peak — good signal
- check brand average later
- imputation strategy roundtrip

PYTHON

```
df['rating'].describe()
```

PYTHON

```
sns.histplot(df['rating'],kde=True)
```

Boolean Features View

- countplot for inspecting binary features
- 5G/NFC/IR blaster adoption

PYTHON

```
sns.countplot(df['has_5g'])  
df['has_5g'].value_counts()
```

Deep Dive : Boolean features — countplot

- countplot — bar of each boolean value
- value_counts — exact numbers
- 5G adoption → how many models
- NFC, IR blaster separate
- Booleans from sim column cleaning earlier
- insights: availability % per feature
- distribution balanced/skewed?
- cross-feature with price later
- boolean feature richness
- market standardization trend captured

PYTHON

```
sns.countplot(df['has_5g'])
```

PYTHON

```
df['has_5g'].value_counts()
```

Processor brand fixing

- Typo/spelling sharpen: samsung→snapdragon
- Same-brand different names → coalesce
- `loc[temp_df.index, col]` re-map
- a13 → bionic

PYTHON

```
df['processor_brand'] = df['processor_brand'].str.replace('sanpdragon', 'snapdragon')
temp_df = df[df['processor_brand'] == 'qualcomm']
df.loc[temp_df.index, 'processor_brand'] = 'snapdragon'
```

Deep Dive : Fixing categorical labels

- typo typos 'sanpdragon' → 'snapdragon'
- synonyms merge: 'qualcomm' == 'snapdragon' (same chip lineage)
- Apple 'a13' → 'bionic' consistent brand name
- str.replace + loc assign — value standardization
- value_counts after — clean buckets
- category cleaning before numeric analysis
- Lowercase uniformity
- spelling merges make groupby correct

- consolidate based on domain knowledge
- data catalog tidy

PYTHON

```
df['processor_brand'] = df['processor_brand'].str.replace('sanpdragon', 'snapdragon')
df['processor_brand'] = df['processor_brand'].str.replace('apple', 'bionic')
df['processor_brand'] = df['processor_brand'].str.replace('samsung', 'exynos')
```

PYTHON

```
temp_df = df[df['processor_brand'] == 'qualcomm']
```

Processor speed

- cores text → count check
- speed numeric distribution
- Skew + boxplot — outliers

PYTHON

```
df['num_cores'].value_counts()
df['processor_speed'].describe()
sns.displot(kind='kde', data=df, x='processor_speed')
```

Deep Dive : Processor speed & cores distribution

- cores mostly 8 (Octa) — modern standard
- speed GHz distribution golden — skew slightly
- kde shape — processor capability curve
- box — outlier slow chips
- describe quantiles
- correlation with price later
- speed uniform enough
- categorical cores converted to number later
- semiconductor segmentation by rank
- hardware analysis depth

PYTHON

```
df['num_cores'].value_counts()
```

PYTHON

```
df['processor_speed'].describe()
```

Deep Dive : Fast charging feature engineering

- fast_charging had -1 sentinel (missing flag)
- apply row-wise → boolean available column
- -1/0 → missing → 0/absent; else 1
- clean fast_charging watts: invalid → NaN
- keep watts only for real
- new column flag usable in corr
- feature engineering: raw → usable boolean
- scale to watts actual metric
- this shapes product tier
- consistent availability semantics

PYTHON

```
df['fast_charging'].describe()
```

PYTHON

```
def fast(row):
    if row['fast_charging'] == -1:
        return 0
    else:
        return 1
```

RAM / Internal Memory

- loc fills the known missing rows
- Value pairs are close — ram + memory together
- screen_size distribution — checks

PYTHON

```
df['ram_capacity'].value_counts()
```

PYTHON

```
temp_df = df[df['internal_memory'].isnull()]
df.loc[temp_df.index, ['ram_capacity', 'internal_memory']] = [[4,64],[4,64]]
```

Deep Dive : RAM / memory imputation

- identify missing memory rows
- small set → manual fill known pairs
- loc multi-col assign
- ram/ memory hierarchy — 4/64 typical budget combo

- after: value_counts sanity
- Option: groupby median impute for bigger
- manual override reveals domain guess
- verify impact on corr
- model-ready no NaN
- ML feature consistent

PYTHON

```
temp_df = df[df['internal_memory'].isnull()]
```

PYTHON

```
df.loc[temp_df.index,['ram_capacity','internal_memory']] = [[4,64],[4,64]]
```

Extended memory flag + cap

- Flag: available or not
- Upto capacity extract
- Unit normalization transform

PYTHON

```
df['extended_memory'].value_counts()
```

PYTHON

```
df['extended_memory_available'] = df['extended_memory'].apply(lambda x: 0 if x == '0' else 1)
x = df.apply(extended_extractor, axis=1).str.split(' ').str.get(0)
df['extended_upto'] = x
def transform(text):
    if text == '1': return '1024' # 1 GB → 1024 MB
    ...
```

Deep Dive : Extended memory — flag + capacity

- '0' vs TB text → boolean flag available
- extract capacity number from 'up to 1 TB'
- thin-space unicode → space normalize
- split get(0) number part
- transform maps TB→1024 (standardize GB units)
- two derived from one text col
- consistent units for numeric
- apply axis=1 row function pattern
- spec text parsing expanded

- verified via value_counts

PYTHON

```
df['extended_memory_available'] = df['extended_memory'].apply(lambda x:0 if x == '0' else 1)
```

PYTHON

```
def extended_extractor(row):

    if row['extended_memory_available'] == 0:
        return np.nan
    else:
        if row['extended_memory'] == '1 TB':
            return 1024
        elif row['extended_memory'] == '512 GB':
            return 512
        elif row['extended_memory'] == '256 GB':
            return 256
        elif row['extended_memory'] == 'Not Specified':
            return np.nan
        elif row['extended_memory'] == 'Memory Card (Hybrid)':
            return np.nan
        elif row['extended_memory'] == '128 GB':
            return 128
        elif row['extended_memory'] == '2 TB':
            return 2048
        elif row['extended_memory'] == '32 GB':
            return 32
        elif row['extended_memory'] == '64 GB':
            return 64
        elif row['extended_memory'] == '1000 GB':
            return 1000
```

OS Cleanup

- 'Memory Card' values ended up in the OS column → NaN
- Standardize: android/ios/others
- Custom apply transforms the mess

PYTHON

```
df['os']
```

PYTHON

```
def os_transform(text):
    if 'Memory' in text:
        return np.nan
    elif 'android' in text:
        return text
    elif 'ios' in text:
        return text
    else:
        return 'other'
```

Drop the redundancies, then correlate

The engineered columns have made several raw columns redundant, so this cell drops what is no longer needed: `df.drop(columns=['processor_name', 'extended_memory'], inplace=True)` removes the source columns whose useful content has already been captured by processed versions (`processor_brand`, `num_cores`, `extended_upto`). A column you parse into better columns and never look at again is dead weight — it clutters every `head()` and every future correlation.

With the frame lightened, the analysis asks its central question: `df.corr()['rating']` ranks every numeric column by how strongly it moves *rating* rather than price — this session's target variable. A strong positive coefficient means the feature travels with customer satisfaction; a near-zero one means it does not.

The sequencing is the lesson: **clean before you correlate**. Correlations computed on messy text columns or un-imputed gaps are correlations you cannot trust. Drop the redundant, normalise the remaining types, and only then read the coefficients.

Deep Dive : Drop & correlation & final export

- drop redundant raw cols (already parsed)
- missing recount — balanced
- replacements normalize cores 'Octa Core'→8
- KNN fill remaining NaN numeric
- corr by target col — identify most influential features
- get_dummies categorical → one-hot for models
- To CSV v4/v5 — versioned clean output
- corr compare before/after imputation (merge index results)
- Ready dataset downstream ML
- Full pipeline: assess → clean → transform → export

PYTHON

```
df.drop(columns=['processor_name', 'extended_memory'], inplace=True)
```

PYTHON

```
df.isnull().sum()
```

Export v4/v5

- Iterative exports — v4 → v5 evolving cleaner
- Type fixed (cores numeric)
- Pipeline manage — versions make

PYTHON

```
df.to_csv('smartphone_cleaned_v4.csv', index=False)
```

PYTHON

```
new_df = pd.read_csv('smartphone_cleaned_v4.csv')
```

KNN impute, then correlate again

The 101 missing ratings are real data loss, so this cell fills them with **KNN imputation**:

`KNNImputer(n_neighbors=3, weights='distance')` looks at each incomplete row, finds its three nearest neighbours in the numeric feature space, and guesses the rating from their (distance-weighted) average.

`weights='distance'` matters — the closer a neighbour is, the more it influences the fill, which keeps the imputed values faithful to the local structure of the data.

Then the notebook does the honest thing: it correlates **before and after** the imputation. `x` holds the imputed correlations with price, `y` the raw ones, and `x.merge(y, on='index')` prints both side by side in one table. If the imputed coefficients have shifted — say `price` vs `rating` creeping from 0.28 to 0.34 — that shift is *evidence* that the missing rows were non-random, and the imputation recovered signal the naive table was hiding.

The habit to keep: never present a filled-in column without showing what the fill changed. An imputed frame is a claim, and a before-after compare is its receipt.

PYTHON

```
from sklearn.impute import KNNImputer

knn = KNNImputer(n_neighbors=3, weights='distance')

return_array = knn.fit_transform(x_df)
```

PYTHON

```
x = pd.DataFrame(return_array, columns=x_df.columns).corr()['price'].reset_index()
```

Dummies + save

- Categorical → binary dummy (one-hot)
- `drop_first` — avoid dummy trap

- `category` dtype — memory efficient
- EDA → model-ready format

PYTHON

```
pd.get_dummies(new_df, columns=[ 'brand_name', 'processor_brand', 'os'], drop_first=True)
```

PYTHON

```
new_df[ 'brand_name' ] = new_df[ 'brand_name' ].astype( 'category' )
```

Exploratory Data Analysis (EDA) — Titanic Dataset

Steps of Univariate Analysis on Numerical columns

1. **Descriptive Statistics** — compute mean, median, mode, std, range, quartiles. Gives a general sense of centre and spread and reveals skewness/outliers.
2. **Visualizations** — histograms, box plots and density (KDE) plots to see the distribution.
3. **Identifying Outliers** — via box plots / IQR. Decide if they are measurement errors, entry errors or legitimate values, and whether to keep/drop them.
4. **Skewness** — check and decide whether to transform the data or use robust methods.
5. **Conclusion** — document findings and decide how to proceed.

Age

Conclusions (Age):

- Age is **almost** normally distributed.
- **~20%** of values are missing.
- There are some **outliers** (a few very old passengers).

PYTHON

```
df[ 'Age' ].describe()
```

PYTHON

```
df[ 'Age' ].plot(kind='hist', bins=20)
plt.title('Distribution of Age')
plt.show()
```

Steps of Univariate Analysis on Categorical columns

1. **Descriptive Statistics** — frequency distribution of each category and relative frequencies.
2. **Visualizations** — count plots and pie charts to see the distribution and spot anomalies.
3. **Missing Values** — check and decide how to handle (impute or exclude).

4. Conclusion — document findings and decide how to proceed.

Survived, Sex, Embarked

Conclusions:

- `Parch` and `SibSp` can be merged to form a new column **family_size**.
- We can also create a new column **is_alone**.

PYTHON

```
df['Embarked'].value_counts()
```

PYTHON

```
df['Embarked'].value_counts().plot(kind='bar')
plt.title('Embarked - Count Plot')
plt.show()
```

Bivariate Analysis

Bivariate analysis looks at **two columns together** to understand their relationship.

Steps

1. **Select 2 columns.**
2. **Understand the type of relationship** between them:
 1. **Numerical – Numerical**
 - Scatter plots (and regression plots), 2D histograms, 2D KDE plots.
 - Check the **correlation coefficient** for a linear relationship.
 2. **Numerical – Categorical**
 - Compare the distribution of the numerical variable across categories.
 - bar plots, box plots, KDE plots, violin plots, even scatter plots.
 3. **Categorical – Categorical**
 - **Cross-tabulations / contingency tables** (`pd.crosstab`) showing how one categorical column splits by another.
 - heatmaps, stacked bar plots, treemaps.
3. **Write your conclusions** — always document what you observe.

Categorical – Categorical: Survival vs other categories

`normalize='columns'` computes, for **each** category of the second variable, what percentage were

Survived (1) vs Not Survived (0). `* 100` turns proportions into percentages.

PYTHON

```
# Survival vs Passenger Class
sns.heatmap(pd.crosstab(df['Survived'], df['Pclass'], normalize='columns') * 100,
            annot=True, fmt='.1f', cmap='coolwarm')
plt.title('Survival % by Passenger Class (columns normalized)')
plt.show()
```

PYTHON

```
# Survival vs Sex
pd.crosstab(df['Survived'], df['Sex'], normalize='columns') * 100
```

Numerical – Categorical: Survival vs Age

Conclusion: The age distributions of survivors and non-survivors largely overlap; age alone is not hugely predictive, but children and older adults differ slightly in survival rates.

PYTHON

```
df[df['Survived'] == 1]['Age'].plot(kind='kde', label='Survived')
df[df['Survived'] == 0]['Age'].plot(kind='kde', label='Not Survived')
plt.legend()
plt.title('Age distribution: Survived vs Not Survived')
plt.show()
```

PYTHON

```
# Mean age of 1st class passengers (just an example of numerical-categorical summary)
print(df[df['Pclass'] == 1]['Age'].mean())
```

Feature Engineering on the Titanic data

Feature engineering turns raw columns into **more informative** ones. We will create:

1. `individual_fare` — `Fare` is the *group* fare summed across a family/ticket; dividing by family size gives the per-person fare.
2. `family_size` — `SibSp + Parch + 1` (the passenger themselves).
3. `family_type` — alone / small / large based on family size.
4. `surname` — from `Name`.
5. `title` — the person's title (`Mr.`, `Mrs.`, `Master.`, `Miss.`, ...).
6. `deck` — first letter of `Cabin`, with missing values filled as `'M'`.

Note: we also load the **test** set and concatenate, so we can inspect passengers with the same ticket.

PYTHON

```
df['SibSp'].value_counts()
df[df['Ticket'] == 'CA. 2343']
df[df['Name'].str.contains('Sage')]
```

Reading the family columns and joining the test set

Why yes, we count. After the singles columns, the notebook turns to the *relationship-features*: `SibSp` (siblings/spouses aboard) and `Parch` (parents/children). `value_counts()` on each shows you the raw shape — most passengers travelled alone, a small but real group had one sibling, and a handful of families were large. Do not skip these outputs: the combination of the two will later be compressed into `family_size`, one of the strongest predictive ideas in this dataset.

The concat trick. To study ticket behaviour you want the whole passenger universe, so the notebook temporarily joins `train` and `test` with `pd.concat([train, test])`. The two frames share the *same 11 columns*, so they stack cleanly. The purpose is purely diagnostic — counting how many passengers share a single ticket number — and any columns you build here are built on the combined frame before the datasets are split apart again for modelling.

Read the prints. Every `value_counts()` you see is a question answered: who is on board, alone or in company, and who shares a cabin with whom.

PYTHON

```
df['SibSp'].value_counts()
```

PYTHON

```
df[df['Ticket'] == 'CA. 2343']
```

1. Individual Fare

This divides the group fare by the total people sharing the ticket. Compare `individual_fare` vs `Fare`.

PYTHON

```
df['individual_fare'] = df['Fare'] / (df['SibSp'] + df['Parch'] + 1)

df['individual_fare'].plot(kind='box')
plt.title('Individual Fare - Box Plot')
plt.show()
```

PYTHON

```
df[['individual_fare', 'Fare']].describe()
```

2. Family Size and 3. Family Type

- 1 -> alone
- 2 - 4 -> small

• > 5 -> large

PYTHON

```
df['family_size'] = df['SibSp'] + df['Parch'] + 1

def transform_family_size(num):
    if num == 1:
        return 'alone'
    elif num > 1 and num < 5:
        return 'small'
    else:
        return 'large'

df['family_type'] = df['family_size'].apply(transform_family_size)

df.head()
```

PYTHON

```
# Survival rate by family type
pd.crosstab(df['Survived'], df['family_type'], normalize='columns') * 100
```

4. Surname and 5. Title

PYTHON

```
df['surname'] = df['Name'].str.split(',').str.get(0)
df['title'] = df['Name'].str.split(',').str.get(1).str.strip().str.split(' ').str.get(0)

df.head()
```

PYTHON

```
# Survival by title, restricting to the main titles
temp_df = df[df['title'].isin(['Mr.', 'Miss.', 'Mrs.', 'Master.'])]
pd.crosstab(temp_df['Survived'], temp_df['title'], normalize='columns') * 100
```

Fold rare titles into other

PYTHON

```
df['title'] = df['title'].str.replace('Rev.', 'other')
df['title'] = df['title'].str.replace('Dr.', 'other')
df['title'] = df['title'].str.replace('Col.', 'other')
df['title'] = df['title'].str.replace('Major.', 'other')
df['title'] = df['title'].str.replace('Capt.', 'other')
df['title'] = df['title'].str.replace('the', 'other')
df['title'] = df['title'].str.replace('Jonkheer.', 'other')

df['title'].value_counts()
```

PYTHON

```
df['title'] = df['title'].str.replace('Rev.', 'other')
df['title'] = df['title'].str.replace('Dr.', 'other')
df['title'] = df['title'].str.replace('Col.', 'other')
df['title'] = df['title'].str.replace('Major.', 'other')
df['title'] = df['title'].str.replace('Capt.', 'other')
df['title'] = df['title'].str.replace('the', 'other')
df['title'] = df['title'].str.replace('Jonkheer.', 'other')
```

6. Deck from Cabin

We fill missing cabins with `'M'` (missing) and take the first letter as the **deck**.

PYTHON

```
print('Missing Cabin: %.1f%%' % (df['Cabin'].isnull().sum() / len(df['Cabin']) * 100))
df['Cabin'].fillna('M', inplace=True)
df['Cabin'].value_counts()
```

PYTHON

```
df['deck'] = df['Cabin'].str[0]
df['deck'].value_counts()
```

Correlation and Pair Plots

The **correlation heatmap** shows the linear relationship (Pearson r) between every pair of numerical columns. The **pair plot** shows every pair of variables side by side (scatter + histograms on diagonal).

PYTHON

```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

PYTHON

```
# Pair plot on the un-engineered test data
sns.pairplot(df1)
plt.show()
```

Hands-on Task: EDA on an Insurance Dataset

Now we apply the **full EDA process** to a real dataset: **Insurance Claim Analysis — Demographic and Health**.

- Dataset source (Kaggle): <https://www.kaggle.com/datasets/thedevastator/insurance-claim-analysis-demographic-and-health>
- This is the classic **insurance** dataset with **medical costs** (`charges`).

The Columns (typical for this dataset):

COLUMN	TYPE	MEANING
age	numerical	age of the insured person
sex	categorical	male / female
bmi	numerical	body mass index
children	numerical	number of dependants covered
smoker	categorical	yes / no
region	categorical	northeast / northwest / southeast / southwest
charges	numerical	individual medical costs billed by health insurance (target)

How to get the data: Download `insurance.csv` from the Kaggle link (or use any copy of the classic `insurance.csv`) and place it in the same folder as this notebook, then run the next cell.

Step 0 — Load the data

PYTHON

```
ins = pd.read_csv('insurance.csv')
ins.head()
ins.info()
ins.describe(include='all')
```

The insurance mini-project — a second, independent pass

Same recipe, fresh ingredients. The Titanic analysis ends and a second dataset arrives: the **US Medical Insurance** charges exploration. The purpose is to prove that the EDA ladder generalises — `insurance` has different columns (`age`, `sex`, `bmi`, `children`, `smoker`, `region`, `charges`) but the same questions apply. What do the numbers look like? Which slices differ the most? What predicts a big bill?

Plan the run before you code. Read the data, check `head()`, `shape`, `info()` and `describe()` exactly as before; then run the univariate pass on every numeric column and the bar/pie pass on every categorical. Every play you already know from Titanic reappears here, so this notebook is your first chance to drive without training wheels.

Keep a hypothesis list. Before each chart, write one line predicting what you expect to see. The strongest learner habit in this course is predicting first and checking second — the charts themselves become your examiner.

PYTHON

```
ins = pd.read_csv('insurance.csv')

ins.head()
```

PYTHON

```
ins.info()
```

Step 1 — Univariate Analysis

1.1 Numerical columns

For each numerical column we compute descriptive statistics, histograms, KDE, box plots, skewness, and check for outliers / missing values.

PYTHON

```
# Missing values check across the whole dataframe
print('Missing values per column:')
print(ins.isnull().sum())
print()
print('Duplicate rows:', ins.duplicated().sum())
```

PYTHON

```
# Descriptive statistics for all numerical columns
ins[['age', 'bmi', 'children', 'charges']].describe()
```

BMI

- Roughly **normal (bell-shaped)** with mean ≈ 30.6 .
- A few **outliers** on the high side (bmi > 45) — worth investigating but they are realistic values.

PYTHON

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
ins['bmi'].plot(kind='hist', bins=30, ax=axes[0]); axes[0].set_title('BMI Histogram')
ins['bmi'].plot(kind='kde', ax=axes[1]); axes[1].set_title('BMI KDE')
ins['bmi'].plot(kind='box', ax=axes[2]); axes[2].set_title('BMI Box')
plt.tight_layout(); plt.show()
```

Children

- Strongly **right-skewed**; most people have 0 children, counts drop off quickly.
- It is a **count** (discrete) variable, so a value-count view is more useful than a histogram.

PYTHON

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
ins['children'].value_counts().sort_index().plot(kind='bar', ax=axes[0]); axes[0].set_title('Children - Co
ins['children'].plot(kind='box', ax=axes[1]); axes[1].set_title('Children Box')
plt.tight_layout(); plt.show()
```

Charges (target)

- **Heavily right-skewed** (skew ≈ 1.5). This is typical of medical-cost data — most people are cheap, a minority are very expensive.
- There is a distinct second cluster of high charges ($\approx 30k$ – $50k$) which is strongly related to **smokers** (we will confirm in bivariate analysis).
- **Implication for modelling:** a log transform is frequently applied to `charges` to reduce skewness.

PYTHON

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
ins['charges'].plot(kind='hist', bins=40, ax=axes[0]); axes[0].set_title('Charges Histogram')
ins['charges'].plot(kind='kde', ax=axes[1]); axes[1].set_title('Charges KDE')
ins['charges'].plot(kind='box', ax=axes[2]); axes[2].set_title('Charges Box')
plt.tight_layout(); plt.show()
```

PYTHON

```
# Log transform reduces the skew drastically
print('Charges skew:', round(ins['charges'].skew(), 3))
print('Log(charges) skew:', round(np.log1p(ins['charges']).skew(), 3))
```

1.2 Categorical columns

For each categorical column we look at **frequency distributions** with count plots / pie charts and check for missing values.

Conclusions:

- `sex` is well balanced ($\sim 50/50$).
- `smoker` is imbalanced — only $\sim 20\%$ are smokers (important class-imbalance note).
- `region` is balanced across the four regions.

PYTHON

```
for col in ['sex', 'smoker', 'region']:
    print(f'--- {col} ---')
    print(ins[col].value_counts())
    print()
```

PYTHON

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
ins['sex'].value_counts().plot(kind='bar', ax=axes[0]); axes[0].set_title('Sex')
ins['smoker'].value_counts().plot(kind='bar', ax=axes[1]); axes[1].set_title('Smoker')
ins['region'].value_counts().plot(kind='bar', ax=axes[2]); axes[2].set_title('Region')
plt.tight_layout(); plt.show()
```

Step 2 — Bivariate Analysis

2.1 Categorical → Numerical: effect on charges

We compare `charges` across the categories of `sex`, `smoker`, `region`, and `children`. Box plots and bar plots are ideal here.

Conclusions:

- **Smoker is the dominant driver of charges.** Smokers pay far more (median $\approx$ 34k vs $\approx$ 7.3k for non-smokers). The bimodal-looking distribution of `charges` is really the two smoker groups overlapping.
- **Age and BMI** each add costs, but their effect is much smaller than smoking.
- `sex` and `region` have only marginal effects on average charges.
- **Children:** up to $\sim$ 3 children slightly raises costs, then plateaus.

PYTHON

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

sns.boxplot(data=ins, x='smoker', y='charges', ax=axes[0]); axes[0].set_title('Charges by Smoker')
sns.boxplot(data=ins, x='sex', y='charges', ax=axes[1]); axes[1].set_title('Charges by Sex')
sns.boxplot(data=ins, x='region', y='charges', ax=axes[2]); axes[2].set_title('Charges by Region')

plt.tight_layout(); plt.show()
```

PYTHON

```
# Average charges by category (quick numeric summary)
print(ins.groupby('smoker')['charges'].mean().round(2))
print()
print(ins.groupby('sex')['charges'].mean().round(2))
print()
print(ins.groupby('region')['charges'].mean().round(2))
```

2.2 Numerical → Numerical: correlation with charges

Scatter plots plus the **correlation coefficient** tell us the linear relationship with `charges`.

Conclusions:

- `age` vs `charges`: positive relation but with two visible clusters (again smokers vs non-smokers).
- `bmi` vs `charges`: weak positive correlation overall; correlation is much stronger **within smokers**.

- Correlation matrix: `charges` correlates most with `age` (≈ 0.30) and `bmi` (≈ 0.20) among numerics; the real driver, `smoker`, is categorical.

PYTHON

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.scatterplot(data=ins, x='age', y='charges', hue='smoker', ax=axes[0]); axes[0].set_title('Charges vs Age')
sns.scatterplot(data=ins, x='bmi', y='charges', hue='smoker', ax=axes[1]); axes[1].set_title('Charges vs BMI')
plt.tight_layout(); plt.show()
```

PYTHON

```
sns.heatmap(ins[['age', 'bmi', 'children', 'charges']].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap (numerical columns)')
plt.show()
```

2.3 Categorical → Categorical

Conclusions:

- Smoking is not evenly spread** — in the **southeast** the *smoker share* is slightly higher.
- `sex` and `smoker` are roughly balanced across each other (no strong interaction).

PYTHON

```
pd.crosstab(ins['smoker'], ins['region'], normalize='columns') * 100
```

PYTHON

```
pd.crosstab(ins['smoker'], ins['sex'], normalize='columns') * 100
```

Step 3 — Multivariate (summary view)

A `pairplot` colored by `smoker` captures the key relationships at once: smokers form a clearly separate, higher-cost cloud across `age` and `bmi`.

PYTHON

```
sns.pairplot(ins, hue='smoker', diag_kind='kde')
plt.show()
```

Step 4 — Feature Engineering

Based on the EDA we create new, more predictive columns:

4.1 BMI categories (`bmi_category`)

Standard WHO thresholds:

- `< 18.5` → `underweight`
- `18.5–24.9` → `normal`

- `25-29.9` → `overweight`
- `>= 30` → `obese`

4.2 Age group (`age_group`)

- `18-30` → `young`, `31-50` → `middle`, `> 50` → `senior`

4.3 `insurance_risk` (composite health flag)

A simple health-risk label combining smoking and BMI:

- `high risk` → smoker AND (bmi $\geq$ 30)
- `moderate risk` → smoker OR (bmi $\geq$ 30)
- `low risk` → otherwise (non-smoker and bmi $<$ 30)

4.4 `log_charges`

A log-transformed target to reduce the heavy right-skew before modelling:

PYTHON

```
ins['log_charges'] = np.log1p(ins['charges'])
```

Feature engineering is where skill shows

Raw columns are rarely enough. For insurance, the notebook builds new variables that encode medical insight directly: `bmi_cat` buckets BMI into Underweight / Normal / Overweight / Obese, `age_group` collapses age into a few bins, and `risk` combines smoker status with a high-BMI flag. Each engineered column is a *claim* about structure — obese smokers are the expensive group — so each one deserves the same scrutiny as a raw column: check its counts, plot it against `charges`, and confirm the split is real.

Transform when the scale needs it. Notice the log-transform on `charges`: insurance bills are heavily right-skewed (a few enormous claims stretch the axis). A log scale or a `np.log1p` transform makes the bulk of the distribution readable and regular — the standard pre-model move for long-tailed money columns.

Validate, don't assume. The notebook finishes by re-checking the engineered frame: duplicates, missing values, and descriptive stats on the new columns. Engineered features that are never inspected are just columns you hope are right.

PYTHON

```
def bmi_cat(bmi):
    if bmi < 18.5:
        return 'underweight'
    elif bmi < 25:
        return 'normal'
    elif bmi < 30:
        return 'overweight'
    else:
        return 'obese'

ins['bmi_category'] = ins['bmi'].apply(bmi_cat)
ins.sample(5)
```

PYTHON

```
def age_group(a):
    if a <= 30:
        return 'young'
    elif a <= 50:
        return 'middle'
    else:
        return 'senior'

ins['age_group'] = ins['age'].apply(age_group)
ins.sample(5)
```

Validate the engineered features

Conclusions:

- `bmi_category` / `insurance_risk` show a clean, monotonic rise in average charges — confirming they are useful predictors.
- `log_charges` is far less skewed than `charges` (skew ≈ 0.3 vs ≈ 1.5) — good for linear models.

PYTHON

```
ins.groupby('bmi_category')['charges'].mean().round(2)
```

PYTHON

```
ins.groupby('insurance_risk')['charges'].mean().round(2)
```

Step 5 — Summary & Conclusions

Univariate:

- `charges` is highly **right-skewed** with two overlapping cost clusters → use `log_charges` for modelling.
- `bmi` is near-normal with a few high outliers (realistic). `age` is roughly uniform. `children` is a right-skewed count (`0` is most common).
- No missing values; a few **duplicate rows** can be dropped.

Bivariate / Multivariate:

- `smoker` is the single strongest driver of medical charges (median roughly 3–4× higher).
- `age` and `bmi` have weaker, positive effects that are clearer once `smoker` is controlled for.
- `sex` and `region` have only marginal average effects.

Feature engineering performed:

- `bmi_category`, `age_group`, `insurance_risk` (composite risk), and `log_charges`.
- The new risk features show a monotonic relationship with cost — useful for modelling.

Next steps (if modelling):

- One-hot / label encode `sex`, `smoker`, `region`, and the new categorical features.
- Scale `age` / `bmi`.
- Fit a regression model (e.g. Linear / Random Forest) on `log_charges`; invert with `expm1` for predictions.

Your Turn

Try extending this notebook:

1. Drop duplicates with `ins = ins.drop_duplicates()` and re-run the stats.
2. Analyse `children` vs charges with a bar plot of *average* charges.
3. Build a simple model: split data, train a `RandomForestRegressor` on **log_charges**, and report RMSE.
4. Explore other bivariate combos (e.g. `age_group` × `smoker` vs charges) with an `hue` box plot.

ETL Pipeline using AWS RDS

What is ETL?

ETL stands for **Extract, Transform, Load** — the standard pipeline that moves raw data from source systems into a single, consistent store where analysis actually happens. The notebook's diagram captures it as three boxes in a line: **Extract** pulls data from wherever it lives, **Transform** repairs and reshapes it, and **Load** writes the result to a destination.

Each phase has a job. **Extract** is about *getting hold* of the data without damaging it — reading a CSV, calling an API, pulling from a database, or scraping a page — and keeping the original intact. **Transform** is where value is added: cleaning (removing duplicates, handling nulls), filtering (keeping only relevant rows and columns), aggregating (grouping to sums and means), feature engineering (creating new columns), and validating the result. **Load** is the final write to a **data warehouse**, **database** or **data lake**, choosing between a full load and an incremental one.

The mental model to keep: ETL is a *conveyor belt*, not a one-off script. Sources change, so the belt must be re-runnable; transforms change, so each stage must be inspectable; and the whole point is that analysts downstream query a clean, consistent store instead of chasing file formats.

The Three Steps

ETL decomposes into three responsibilities, and this notebook will honour them as literal phases. **Extract** — bring data from source systems (CSV, API, database, web) and keep the original column intact. Here that means two Kaggle CSVs: `matches.csv` for match-level facts and `deliveries.csv` for ball-by-ball detail.

Transform — this is where the data becomes *analysable*: clean it (remove duplicates, handle nulls), filter to the rows and columns that matter, aggregate with `groupby` / `sum` / `mean`, engineer new features, and validate quality. In this notebook, Transform produces the Dream11 points engine and the per-player summaries — the entire analytical payload of the dataset.

Load — write the transformed result to its destination: a data warehouse, a database, or a data lake. The destination here is an **AWS RDS** MySQL instance, and the code must choose full versus incremental loading per table. Read these three phases as a contract: extract something worth keeping, transform it until it is trustworthy, load it where it can be queried.

Why ETL?

A pipeline earns its keep by answering six needs. **Data integration** — one dataset rarely tells the whole story; ETL combines tables from multiple sources into one view. In this project, match facts and ball-by-ball events become one enriched player table. **Data quality** — because data is cleaned and validated before anyone queried it, the downstream consumer never has to hope. **Historical analysis** — a stored store supports trend analysis over time, exactly what the IPL's 2008–2020 span enables.

Reporting — BI dashboards and analytics draw from one consistent schema rather than ad-hoc files.

Machine learning — models need prepared, typed features, which is precisely the output of a clean transform stage. **Compliance and governance** — a governed store means you can say where data came from, how it changed, and who touched it.

The unifying idea: ETL converts *data* into *assets*. A CSV is something you load and pray about; a loaded, versioned store is something you can query, audit, and trust. Everything downstream — dashboards, models, reports — stands on the quality the pipeline established earlier.

What is AWS RDS?

Amazon RDS (Relational Database Service) is a managed cloud database: Amazon provisions, runs, and maintains a relational database for you — backups, patching, monitoring, failover — so you focus on queries, not server upkeep. This notebook uses it as the landing zone for the transformed IPL data, running **MySQL** under the hood.

The service supports the major engines: **MySQL** (the most popular open-source database), **PostgreSQL** (advanced open-source), **MariaDB** (a MySQL fork), **Oracle** and **SQL Server** (enterprise picks), and **Amazon Aurora** (an AWS-optimized MySQL/PostgreSQL-compatible engine).

Why choose it? **Managed** — backups, patching, monitoring handled by AWS. **Scalable** — storage and compute resize without rebuilding. **High availability** — Multi-AZ deployments survive instance failure. **Security** — encryption, VPC isolation, IAM authentication. **Cost** — you pay for what you use, and the Free Tier makes this exercise effectively free. In short, RDS gives you a production-grade database with none of the night shift.

Setting Up AWS RDS (Step-by-Step)

Before any code runs, a database must exist in the cloud, and this notebook walks the full setup.

Prerequisites: an AWS account (Free Tier eligible) and a SQL client such as MySQL Workbench for eyeballing results.

Steps: login to the AWS Console, search for RDS, and click *Create Database*; choose the **MySQL** engine and the **Free Tier** template; configure the instance — DB identifier `ipl-database`, master username `admin`, a master password of your choosing; enable **Publicly Accessible** for this test setup; add a security-group rule opening port `3306`; and create the database. AWS then provisions the instance and hands back a connection block that matters more than everything else:

- **Endpoint** — `ipl-database.xxxxx.sa-east-1.rds.amazonaws.com`
- **Port** — `3306`
- **Database** — `ipl_db`
- **Username / password** — `admin` / your own

Copy those exact values into the Python config later; they are the keys to every connection in the rest of the notebook. Treat the endpoint like a hostname — the whole pipeline hinges on getting it into the code verbatim.

Installation

The notebook begins by installing the libraries the whole pipeline depends on: `pip install pandas numpy mysql-connector-python sqlalchemy pymysql`. Each earns its place. `pandas` and `numpy` are the data machinery — reading the CSVs and computing the aggregations. `mysql-connector-python` is the official driver that lets Python talk to a MySQL server directly, used here for a quick connection test.

`sqlalchemy` is the crucial one for pandas users: it creates `engine` objects, and `df.to_sql(con=engine)` and `pd.read_sql(query, con=engine)` know how to use any engine from any supported database. `pymysql` is the pure-Python MySQL driver that SQLAlchemy actually uses to speak to the MySQL wire protocol when the connection string says `mysql+pymysql://`.

Note the architecture insight: pandas never talks to the database itself — it always goes through the engine. Install once, import at the top, and every later cell inherits the same plumbing. Keeping the install in its own cell also makes the run order explicit: if a dependency is missing, the notebook fails at the very start with a clear message instead of halfway through a SQL query.

Phase 1 — Extract

Extract is the first phase and the least glamorous: get the raw data out of its source with its integrity intact. The principle hammered here is **keep the original untouched** — extraction reads data, it never mutates the source — because every later repair depends on being able to re-open the pristine file and re-verify.

Two files carry this project. `matches.csv` is match-level: one row per IPL match, carrying season, teams, toss, result, venue, and the winner. `deliveries.csv` is ball-level: one row for every delivery of every innings, naming batsman, bowler, runs scored, extras, and dismissals. They are a classic relational pair — the match is the parent, the hundreds of deliveries are its children, joined by `match_id`.

That split is the real lesson of Extract: *choose the grain* before you load. The finest grain (the ball) powers the Dream11 scoring; the coarser grain (the match) powers the season summaries. Both must be loaded, and the pipeline keeps both throughout Transform.

Download the Dataset

The source for this project is the **IPL Complete Dataset (2008–2020)** on Kaggle —

<https://www.kaggle.com/datasets/patrickb1912/ipl-complete-dataset-20082020>. Two files matter here: `matches.csv` covering every fixture with its metadata, and `deliveries.csv` covering every ball bowled across those seasons.

Two practical points. First, a Kaggle dataset is downloaded as a zip whose inner CSVs are large-ish but far from enormous — `deliveries.csv` is comfortably loadable with plain `pd.read_csv`, which is exactly what the Extract phase does. Second, keep the two files *together* in the runtime directory: the later `read_csv` calls expect `matches.csv` and `deliveries.csv` at the working path. If the verify prints shape mismatches (you saw `Matches shape:` and `Deliveries shape:` in the loader), re-check the download rather than the code.

Take the download itself seriously — the quality of every subsequent chart, score, and query rests on the raw files being the genuine Kaggle snapshots. Garbage in, garbage everywhere downstream.

Reading the two source files

The Extract phase's first real code loads the pair of CSVs that feed the entire pipeline. `matches` carries match-level facts — one row per fixture with seasons, teams, venues, and winners — while `deliveries` carries ball-level detail: every delivery across every match, naming the batter, the bowler, the runs, the extras, and any dismissal.

The loader prints the two shapes side by side, which is the correct instinct: `Matches shape: (...)` and `Deliveries shape: (...)` are the first contract with the data. Deliveries should dwarf matches by orders of magnitude — thousands of balls versus hundreds of fixtures — and that size disparity is precisely why the later Load phase will stream `deliveries` in chunks while `matches` writes in one shot.

Compare the grain before moving on: the summary and scoring logic that follows is all *derived* from these two tables. If the shapes look wrong at this cell, everything downstream inherits the error — so verify the count, then proceed.

Phase 2 — Transform

Transform is where raw data becomes *analysable* data, and its four responsibilities appear later as explicit steps. **Clean** missing values — the notebook drops a useless `umpire3` column and rows whose winner is unknown, then fills the delivery-table gaps with the sentinel `'None'`. **Filter** relevant columns and rows so only what matters survives to the destination.

Feature engineering is the star here: ball-by-ball deliveries are scored with the **Dream11 point system**, producing `batting_points`, `bowling_points`, `fielding_points` and a `total_points` per ball. **Aggregation** then collapses those per-ball scores into per-player-per-match summaries and, after merging in match metadata, into overall career statistics with a strike rate.

The ordering matters: clean first (so scoring never runs on NaN), then engineer, then aggregate — aggregates computed on dirty units would be dirty aggregates. Watch how every transformed table announces a `shape` after creation: Transform treats its own output as something to be inspected, not assumed.

The Dream11 Point System

Dream11 is a fantasy-cricket platform, and its scoring rules are the business logic this transform implements — the reason the notebook can turn raw deliveries into points. The system has three budgets. **Batting:** each run earns +1; a boundary adds +1 bonus and a six adds +2; milestone bonuses land at 30, 50 and 100 runs (+4 / +8 / +16); and a duck (out for zero) costs -2. So a 50 with two sixes is worth materially more than a flat 50.

Bowling: a wicket is +25, with haul bonuses at 3, 4 and 5 wickets (+4 / +8 / +16); a maiden over is +12; and economy rewards tighten — under 5 an over earns +6, between 5 and 6 earns +4, between 6 and 7 earns +2.

Fielding and extras: a catch is +8, a stumping or run-out +12, selection in the playing XI is +4 — and a wide or no-ball -1 rides against the bowler.

Hold this table in view as the Transform phase runs: every point a batsman or bowler accrues in the code traces back to one of these rows. Rules become code, code becomes aggregates, aggregates become the load phase.

Step 1 — Clean Matches Data

Cleaning `matches` is short and pointed. First, `matches.isnull().sum()` audits where the match table leaks: the biggest gap is `umpire3`, a column that existed in early IPL seasons and was dropped later, leaving most rows empty. The repair is honest: `matches.drop(['umpire3'], axis=1, errors='ignore')` removes a column that is mostly missing rather than pretending to fill it.

Second, rows whose critical outcome is missing — no `winner` — are dropped with `dropna(subset=['winner'])`, because a match without a winner cannot feed any winner-based analysis downstream.

The lesson, in one line: **understand the missingness before you repair it**. Column missing everywhere → drop the column; row missing the key fact → drop the row. Both decisions come from the audit line `print("Missing values in matches:")`, and both are confirmed by re-printing the cleaned shape so nobody mistakes the intent for guesswork. The order matters too: drop the useless column before the rows, because addressing one categorical leak first keeps each decision attributable to the audit that uncovered it.

Step 2 — Clean Deliveries Data

`deliveries` is the finer-grained table, and its missingness is concentrated in the three *dismissal* columns: `player_dismissed`, `dismissal_kind`, and `fielder` are blank for the overwhelming majority of balls — because most deliveries do not dismiss anyone. Filling them with the string `'None'` (not `NaN`) is a deliberate choice: the later scoring functions call `row['player_dismissed'] != 'None'` to detect a wicket, and they must be able to compare values without tripping over missing markers.

A working copy is involved — `deliveries_clean = deliveries.copy()` — so the raw frame stays pristine for re-auditing. The functions `calculate_batting_points`, `calculate_bowling_points` and `calculate_fielding_points` will then read these columns.

Note the difference from the matches repair: `matches` *dropped* empty columns, while `deliveries` *fills* them — because here emptiness is meaningful (no dismissal) rather than absence of a feature. Match the repair to the meaning of the missingness, and both tables survive cleaning intact.

Step 3 — Feature Engineering (Dream11 Points)

This is the heart of the transform: a DataFrame `apply` loop that turns every delivery's detail into its Dream11 contribution. `calculate_batting_points(row)` walks a single ball row with plain Python logic — start at zero, add the `batsman_runs`, add +1 if the ball was a four, +2 if it was a six — and the function's simplicity is the point.

The pattern: **write a tiny numbered function for one row, then apply it to every row.**

`deliveries_clean.apply(calculate_batting_points, axis=1)` runs the function across rows (`axis=1` = across the row's columns), producing a new series that becomes `batting_points`. The notebook prints the result side by side with `batsman` and `batsman_runs`, so you can verify the arithmetic by eye on real deliveries before it is trusted.

Why not a clever one-liner? Because a named function is *testable* — later readers can read the rules off the function, and a bug shows in one place instead of hiding inside a lambda chain.

Bowling Points Calculation

Bowling points encode two opposing forces. The reward side: if `player_dismissed != 'None'`, the ball took a wicket, worth **+25** points. The penalty side: the bowler absorbs the extras — a `wide_runs` over 0 costs **-1** point and a `noball_runs` over 0 costs another **-1**, because in fantasy scoring a wide or a no-ball is the bowler's debt, not the batter's credit.

Look at the shape of `calculate_bowling_points`: a handful of `if` / `elif` checks on the row's columns, a running `points` total, and a return. It is row-level logic on explicitly typed columns — which is why the earlier cleaning step turned the empty dismissal fields into `'None'`: this function can now safely compare `row['player_dismissed'] != 'None'` on every single ball.

The wider point about feature engineering: a derived column (`bowling_points`) is only as trustworthy as the unit-level rules that produced it. Scoring on clean, typed columns — and inspecting the first ten rows afterwards — is how a fantasy metric stays defensible from its first ball to its final aggregate.

Fielding Points Calculation

Fielding points reward the same moment through three lenses. `calculate_fielding_points(row)` reads the `dismissal_kind` column — the reason the ball ended — and awards **+8** for a catch, **+12** for a stumping, and **+12** for a run-out. Crucially, dismissals that earn *bowling* points (bowled, lbw, caught-and-bowled) are left alone here: the run-out and stumping reside in the fielding budget precisely because the bowler did not earn them.

The function uses an `if` / `elif` chain keyed on `dismissal_kind`, which only works because Step 2 filled blank dismissals with `'None'` — so `elif` never mis-fires on a missing marker. That dependency is the quiet architecture lesson: **every feature-engineering step relies on the cleaning step before it**, and the notebook announces that reliance by filling before scoring.

Once applied, the notebook has per-ball batting, bowling, and fielding points ready to be summed into each ball's total. The same function is then reused across every ball through `apply`, so one tested rule drives the whole column instead of a hand-typed score per row.

Total Dream11 Points

The three per-ball point streams converge in one clean addition: `deliveries_clean['total_points'] = batting_points + bowling_points + fielding_points`. The complexity is deliberately *not* here — engineering was written in small, audited functions, so the total is a plain column-wise sum, not another pile of rules.

Why it matters: `total_points` is the unit-level currency of the entire downstream analysis. From here, everything the pipeline builds — player-match summaries, overall career stats, the loaded tables — is an aggregation over this one honest column. If any earlier rule had been wrong, it would surface here and poison every aggregate; because each component was checked row-by-row at creation, the sum inherits that trust.

This is the ideal shape for a computed column: **derive, visualize, then aggregate**. The notebook prints ten rows of `batting_points`, `bowling_points`, `fielding_points`, `total_points` side by side — a spot-check that turns "the sum exists" into "the sum is right".

Step 4 — Player Match Summary

Per-ball points become per-player-per-match rows with one aggregate:

`deliveries_clean.groupby(['match_id', 'batsman']).agg(...)`. The `agg` call names the columns explicitly — `runs` summed, `balls_faced` counted, `fours` and `sixes` counted with lambdas, `batting_pts` summed — so the resulting table is readable as a sentence, not a mystery.

Three enrichments follow. **Strike rate**: `runs / balls_faced * 100`, rounded. **Milestone bonuses**: +4 / +8 / +16 when `runs` crosses 30 / 50 / 100. **Duck penalty**: rows where a batsman scored 0 and was dismissed are counted per match and charged -2 each. All three are stacked into `final_batting_points`.

The lesson is aggregation algebra: `groupby(...).agg(...)` collapses grain by defining *what each new row means*. Every column in the summary is either something summed, something counted, or something derived from the sum — and the shape is printed so the collapse is verified, not assumed. When you later join batting and bowling summaries, the shared key (`match_id`, `player`) picks the same grain across tables, so no half-aggregated data leaks through the join.

Step 5 — Bowling Summary

Bowling gets its own summary table via the mirror-image aggregate: `deliveries_clean.groupby(['match_id', 'bowler']).agg(...)` counting balls bowled, summing runs conceded, counting wickets (`player_dismissed != 'None'`), and totalling wides and no-balls.

Then the derived stats arrive. **Overs**: balls divided by six. **Economy**: runs conceded per over, rounded — the number that will gate the bonus. The notebook then layers Dream11's bowling rewards: an **economy bonus** (under 5 = +6, 5–6 = +4, 6–7 = +2) and a **wicket-haul bonus** (+4 / +8 / +16 at 3 / 4 / 5+ wickets), folded into `final_bowling_points`.

Note the discipline in the `loc` assignments — `bowling_match.loc[bowling_match['economy'] < 5, 'economy_bonus'] = 6` — masked updates that touch only qualifying rows. That is how feature engineering stays surgical: define the condition, update the slice, leave the rest untouched. Two details shield the

arithmetic: balls are divided by six in one expression so overs stay float-precise, and every bonus column is seeded to zero before the masked updates so unpicked rows contribute nothing by default.

Step 6 — Merge with Match Data

Separate summaries are useful; enriched summaries are *valuable*. This cell joins the player-match table with match metadata: `player_summary = player_match.merge(matches_clean[['id', 'season', 'city', 'date', 'team1', 'team2', 'winner']], left_on='match_id', right_on='id', how='left')`.

Read the join carefully — it is the classic pandas relationship: `left_on='match_id'` says the left table's match key, `right_on='id'` says the right table's key, and the column lists bound the right side to exactly the match facts that matter, so the result does not balloon with redundant columns. `how='left'` keeps every player-match row even if a match reference is missing.

Two final touches complete the engineer phase: a flat **playing XI bonus** of +4 per row and the headline `dream11_points = final_batting_points + playing_xi_bonus`. The merge is the moment the pipeline stops being two parallel analyses and becomes one joinable dataset — which is, after all, the promise of Extract followed by Transform.

Step 7 — Overall Player Stats

The last transform collapses the whole project into a career table:

`player_summary.groupby('batsman').agg(...)` totals each player's matches (via `nunique` on `match_id`), runs, balls faced, fours, sixes, and Dream11 points, with an average Dream11 points per match. A final derived column, `overall_strike_rate = total_runs / total_balls * 100`, rounds out the picture, and the sort on `total_runs, ascending=False` crowns the leaderboard.

This is where the dream11 scoring proves its worth: totals and career averages are aggregates over the honest `total_points` column reconstructed upstream, and `avg_dream11_points` is a genuinely useful metric — points a player adds per match he plays.

The shape of the whole transform is now visible: clean (Steps 1–2) → score (Step 3) → per-match summary (Steps 4–5) → join context (Step 6) → compress to career (Step 7). Every stage prints its shape, and every stage feeds the next. The data is now ready to leave the notebook for a database.

Phase 3 — Load

Load is the payoff phase: writing the transformed DataFrames into the **AWS RDS** MySQL destination so the data outlives the notebook. The destination tables mirror the transform's own hierarchy — `matches`, `deliveries`, `dream11_points` (the per-match scoring), and `player_stats` (the career table).

Two practical realities make this phase different from the others. **Credentials**: loading into a real cloud database requires the RDS endpoint, user, password, database name and port — which the setup step captured and which the code will read into a config. **Grain**: the `deliveries` table is by far the largest and is streamed *in chunks* with `to_sql(..., if_exists='append'/'replace')`, so a big load neither hammers the connection nor dies on one giant transaction.

The philosophical point is worth stating: after Load, the analysis is no longer tied to Jupyter. Any client with the connection string — a dashboard, a model training script, a colleague's SQL client — can query the same tables. That independence is the entire reason ETL exists.

Database Connection Setup — Option 1 (mysql-connector)

The first connection path uses the official driver directly. `mysql.connector.connect(**DB_CONFIG)` takes a config dictionary whose keys mirror the RDS setup: `host` (the RDS endpoint), `user`, `password`, `database`, and `port` — with placeholders waiting for the values the setup step captured.

The code wraps the call in `try` / `except` and prints `Connection successful!` or `Connection failed:` with the exception — a deliberate first smoke test. Notice it does *not* try to run a query; it simply proves the driver, credentials, network, and database all agree. The connection is opened purely to be verified and closed.

Why bother with this step at all when SQLAlchemy is coming next? Because failing fast at the *wire* level isolates failures: if raw `mysql-connector` cannot connect, the problem is credentials or networking, not ORM plumbing. A clean driver-level test makes every later `to_sql` failure attributable to the pandas layer, not the database.

Database Connection Setup — Option 2 (SQLAlchemy)

The recommended path uses **SQLAlchemy** — the engine layer that `to_sql` and `read_sql` natively understand. `create_engine(f'mysql+pymysql://{user}:{password}@{host}:{port}/{db}')` composes one URL from the RDS pieces: the scheme `mysql+pymysql` says *use the MySQL dialect driven by pymysql*; the host is the RDS endpoint; and the rest is user, password, port, and database name.

The engine string is the single most reused object in the Load and Query phases — every `to_sql` writes through it and every `pd.read_sql(query, con=engine)` reads through it, so composing it once (in a cell clearly marked with placeholder values) keeps credentials in one editable place.

Preferring SQLAlchemy over raw connectors is the pragmatic trade: pandas talks to engines, not drivers, so you can swap MySQL for PostgreSQL or Aurora later without rewriting a single `to_sql`. abstracting the database behind one string is why this option is labelled *recommended*. The entry barrier stays low too: the engine is built and tested once, and any connection error surfaces right where the credential placeholders live.

Create Tables in RDS

Before any `to_sql`, the destination schema must exist — and this cell writes it with pure SQL: four `CREATE TABLE IF NOT EXISTS` statements executed through the engine. `matches` keys on `id INT PRIMARY KEY` with date, teams, toss, result and venue columns; `deliveries` carries the per-ball grain with `match_id`, batsman, bowler, and the run/dismissal fields; `dream11_points` stores the per-match scoring; `player_stats` keys on `batsman` with the career totals.

Notice the SQL is *declared*, not run blind — it lives in a string explicitly framed as "create tables", so the whole schema is reviewable before execution. `IF NOT EXISTS` keeps it idempotent: re-running the notebook does not error when the tables already exist.

This is the professional rhythm of Load: **define the schema before you stream the data**. The columns here were chosen to match DataFrame dtypes (INT, VARCHAR, DATE, FLOAT), so `to_sql` can map cleanly, and any analyst with a SQL client gains an immediately queryable catalogue of the pipeline's output.

Load Data to RDS

Now the four tables stream from pandas to the cloud. `matches_clean.to_sql('matches', con=engine, if_exists='replace', index=False)` is the model call: give the table name, hand over the engine, replace-if-it-exists for idempotent re-runs, and drop the pandas index so the table's own primary key stays authoritative.

`deliveries` is the one large table, and its load shows the chunked pattern: `for i in range(0, len(deliveries_clean), chunk_size)` slices the frame in blocks of 10,000, and each slice writes with `if_exists='replace'` on the first chunk and `'append'` thereafter — one big table loaded in bite-size transactions, each announced by a printed chunk counter.

The `dream11_points` and `player_stats` tables follow the same recipe, and every load prints its row count so success is visible. The lesson of Load is verification at the seam: each `to_sql` states *what* it loaded and *how many rows* — because a load you cannot count is a load you cannot trust.

Querying Data from AWS RDS

The pipeline completes a full circle here: instead of reading DataFrames from CSVs, this section **reads them back from the cloud database** and proves the load worked. `pd.read_sql(query, con=engine)` turns SQL directly into a DataFrame, and the queries put the transform's product to work.

The four analytical queries are the pipeline's audition. **Top batsmen** — `ORDER BY total_runs DESC LIMIT 10` on `player_stats`. **Top Dream11 scorers** — career totals plus `avg_points_per_match` via `ROUND(..., 2)`. **Best economy bowlers** — the hard one: a nested subquery computes overs and economy per match (requiring `COUNT(*) >= 12`, about two overs), then the outer query demands at least 50 career overs and ranks by economy ascending. **Matches per season** — a simple `GROUP BY season` with distinct winners.

Read these as the return on the entire notebook: Extract + Transform produced tables, and Load made them *queryable* — raw SQL now reaches the same insights the transform computed, because the database took custody of the analysis.

ETL Pipeline Summary

The pipeline closed its loop: **Extract** loaded `matches.csv` and `deliveries.csv` with `read_csv`, **Transform** cleaned missingness, scored every ball with the Dream11 system (batting, bowling, fielding, and total points), aggregated per-player-per-match and career summaries with match context and bonuses, and **Load** created four RDS tables — `matches`, `deliveries`, `dream11_points`, `player_stats` — and streamed them to AWS via SQLAlchemy. The finale then queried the database back: top batsmen, top Dream11 scorers, best economy bowlers, and per-season counts.

The transferable lessons are the architecture. ETL is a *phase-disciplined* pipeline: clean before you score, score before you aggregate, aggregate before you load, and verify every stage with a printed shape. `to_sql` / `read_sql` through one SQLAlchemy engine let pandas and MySQL meet cleanly, and `if_exists` discipline makes the whole build re-runnable.

What you leave with is more than working code — it is the habit of treating *the database* as the real destination of any analysis, so the work survives notebooks, servers, and colleagues.

Web Scraping with Python

Table of Contents

1. **Why scrape?** — when and why web scraping matters
2. **How the web works** — URLs, HTTP requests/responses, status codes
3. The **requests** library — download HTML, headers, sessions
4. **HTML & the DOM** — tags, attributes, nesting
5. **CSS selectors** — how to target elements
6. **BeautifulSoup fundamentals** — parse, find, select
7. **Extracting data** — text, attributes, loops
8. **From HTML to a DataFrame** — the full mini-project
9. **Pagination** — scraping multiple pages
10. **Robust scraping** — errors, retries, rate limiting
11. **Selenium** — scraping dynamic / JavaScript sites
12. **Ethics & legality** — robots.txt, terms, rate limits
13. **Glossary & cheatsheet**
14. **Practice problems**

1 · Why Web Scraping?

Web scraping = automatically extracting data from websites using a program (instead of copying and pasting by hand).

When you need it

SITUATION	SOLUTION
The data exists only on a website (no API/file)	Scrape the HTML
You need data repeated every hour/day	Automate the download
You need thousands of rows	Program beats manual copying
Site has an API	Prefer the API (faster, legal, stable)

Scrape vs API — decision rule

Always check for an official API first. Scraping is the last resort when no API or downloadable dataset exists. APIs are faster, more reliable, and don't break the rules.

2 · How the Web Works (HTTP)

When you scrape, you are simply **acting like a browser** — asking a server for pages.

The URL

- **Scheme** — `http` / `https` (secure)
- **Host** — the website's server address
- **Path** — which resource on the server
- **Query string** — extra options after `?`, separated by `&`

The request → response cycle

A request has:

- a **method** (`GET` = fetch, `POST` = send data)
- a **URL**
- **headers** (identity, e.g. `User-Agent`)
- optionally **parameters** in the URL or body

A response has:

- a **status code** (200 ok, 404 not found, 403 forbidden...)
- **headers** (content-type, cookies...)
- a **body** (the actual HTML / JSON / file)

Common status codes

CODE	MEANING	HANDLE BY
200	OK	parse the body
301 / 302	redirect	let <code>requests</code> follow it
403	forbidden / blocked	add headers, slow down, use proxy
404	not found	skip / log it
429	too many requests	sleep and retry
500 / 503	server error	retry after a pause

PYTHON

```
https:// www.example.com /products ?category=phone &page=2
|         |           |           |           |
| scheme  | host       | path    | query params | (more params)
```

PYTHON

```
Your program --- (HTTP request) --> Web server
Your program <-- (HTTP response) -- Web server (HTML / JSON / ...)
```

How a request works

The first active cell asks `requests` for a URL. Think of `requests.get(...)` as sending a typed and stamped letter across the internet and getting an envelope back; the envelope is the **response object**, and the first thing you check is its postmark — the **status code**. `200` means the page exists and was delivered; `403` and `404` mean forbidden and missing. Every later cell in this notebook will respect that check, because scraping without looking at the status is like coding without looking at the error.

`response.text` is the payload: the raw HTML source of the page, a plain string full of `<div>`, `<p>` and `<a>` tags. Getting it is cheap and legal for public pages — the real skill comes next, when we learn to *find meaning inside that string* rather than just printing 50,000 characters of markup.

Keep one fact before moving on: `requests.get` is synchronous, so a slow page simply makes that one call take longer — which is exactly why the helper you meet next wraps it in politeness and retries.

PYTHON

```
import requests

# What a real request + response looks like (explore on your own with internet):
# r = requests.get("https://httpbin.org/get")
# print(r.status_code)          # 200
# print(r.headers.get("Content-Type"))
# print(r.text[:200])           # first 200 chars of the body
print("requests version:", requests.__version__)
```

3 • The `requests` Library

`requests` is the de-facto Python library for HTTP. It is **not** an HTML parser — its job is to *download* the page. BeautifulSoup (section 6) parses it.

Basic pattern

Be a polite browser

Many servers reject requests that have no `User-Agent` (they look like bots):

- **Always set `timeout`** — a hanging request is a hanging script.
- **Use a `Session`** for multiple requests to reuse cookies and `Connection` pooling.

PYTHON

```
import requests

r = requests.get("https://example.com")
r.text          # the HTML as a string
r.status_code    # e.g. 200
r.headers        # response headers (dict)
r.url            # final URL (after redirects)
r.raise_for_status() # raise an error for 4xx/5xx
```

PYTHON

```
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}
r = requests.get(url, headers=headers, timeout=10)
```

Be a polite guest

The `polite_get` helper exists because **being efficient on the web means being patient**. Real sites log their traffic, and a scraper that fires a request every few hundred milliseconds can look like an attack. The helper wraps a simple `requests.get` with a comprehensible `User-Agent` header (telling the server which client is visiting) and calls a short `time.sleep` between calls so requests arrive at a human rhythm.

The sleep is not bureaucracy — it is the difference between being welcomed back and being blocked. Whenever *you* write a scraper later, wrap your own fetch in something like this helper first, not last. Slow, steady and repeatable beats fast and banned every time.

Treat the helper as reusable toolkit, not a one-off: it bundles the two decisions — identify yourself, and do not hammer — that every later API and page scrape in this course should inherit. Notice also what it does *not* do: no pretending to be a browser, no bypassing, nothing deceptive. Public pages, honest requests, human rhythm is the whole contract, and pages tend to reward it with a stable response.

PYTHON

```
import requests

# A polite downloader function. Call it only when online.
def polite_get(url, headers=None, timeout=10):
    h = headers or {"User-Agent": "Mozilla/5.0 (educational scraping notebook)"}
    r = requests.get(url, headers=h, timeout=timeout)
    r.raise_for_status()      # raises for 4xx / 5xx
    return r

# Example (offline-safe – try it when you have internet):
# html_page = polite_get("https://books.toscrape.com/")
# print(html_page.status_code, len(html_page.text), "bytes")
print("polite_get() defined – uses a browser-like User-Agent and a timeout.")
```

PYTHON

```
import requests

# Sessions keep cookies between requests (needed for logins / carts).
# session = requests.Session()
# session.get("https://example.com/login")
# session.post("https://example.com/login", data={"user": "me", "pass": "x"})
# profile = session.get("https://example.com/account")
# print(profile.text[:200])

# Headers you can inspect / send
headers = {
    "User-Agent": "Mozilla/5.0 ...",
    "Accept-Language": "en-US,en;q=0.9",
    "Accept": "text/html",
}
print("A realistic headers dict:", headers)
```

4 · HTML & the DOM

HTML (HyperText Markup Language) describes webpage *structure* with **tags**.

Anatomy of a tag

- **Element** = open tag + content + close tag
- **Attributes** = metadata inside the open tag (`class` , `id` , `href` , `src` ...)
- **Nesting** = tags inside tags → this forms the **DOM tree**
- The **DOM** (Document Object Model) is how browsers (and BeautifulSoup) view the page as a tree of elements.

Tags you will meet most

TAG	PURPOSE
<code><div></code> / <code></code>	generic containers (used for layout/cards)
<code><h1></code> — <code><h6></code>	headings
<code><p></code>	paragraph text
<code><a href></code>	hyperlink
<code></code>	image
<code><table></code> <code><tr></code> <code><td></code>	tables
<code></code> <code></code>	lists

PYTHON

```
<html>
  <body>
    <h1 class="title">Products</h1>
    <div class="card">
      <h2>Phone X</h2>
      <p class="price">15000</p>
      <a href="/p/1">See details</a>
    </div>
  </body>
</html>
```

PYTHON

```
< p class="price" > 15000 </ p >
|   |           |           |
open tagname attr   content close
tag      value      tag
```

5 • CSS Selectors

To *find* data in HTML, you describe **where** it is. CSS selectors are the most common language for that.

SELECTOR	MATCHES	EXAMPLE
<code>tagname</code>	all tags of that name	<code>p</code> , <code>h2</code> , <code>div</code>
<code>.class</code>	elements with that class	<code>.price</code> , <code>.card</code>
<code>#id</code>	the element with that id	<code>#main</code>
<code>tag.class</code>	tag with both	<code>p.price</code>
<code>parent child</code>	child inside parent	<code>div.card h2</code>
<code>parent > child</code>	direct child	<code>ul > li</code>
<code>tag, tag</code>	either	<code>h1, h2</code>
<code>[attr]</code>	has the attribute	<code>[href]</code>
<code>[attr=value]</code>	attribute equals value	<code>a[href="/p/1"]</code>

Rule of thumb

The `class` attribute is the web developer's way of labeling elements for styling — which makes it the **most useful** hook for scraping too. Look for repeated, descriptive classes: `.card`, `.price`, `.title`, `.item`.

PYTHON

```
# Practice reading selectors – predict what each matches (answers in the markdown below)
selectors = {
    "div": "all <div> tags",
    ".price": "any element with class='price'",
    "#main": "the element with id='main'",
    "div.card h2": "an h2 inside a div.card",
    "ul > li": "a DIRECT li child of a ul",
    "p, span": "all p OR all span",
    "a[href]": "anchors that have an href",
    "table tr td": "table cells inside rows inside a table",
}
for sel, desc in selectors.items():
    print(f"{sel:12} -> {desc}")
```

6 • BeautifulSoup Fundamentals

BeautifulSoup4 (`bs4`) parses messy HTML into a navigable tree and lets you `find` things with tags, classes, attributes, or CSS selectors.

Install

(Optionally `pip install lxml` for a faster parser — in this notebook we use the built-in `html.parser` so it runs everywhere.)

Import & parse

The three things you will do 90% of the time

1. `soup.find("tag")` → **first** match
2. `soup.find_all("tag")` → **list** of all matches
3. `soup.select("css")` → matches by CSS selector

Let's demonstrate each on a small page.

PYTHON

```
pip install beautifulsoup4
```

PYTHON

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_string, "html.parser")
```

Meet BeautifulSoup

`BeautifulSoup(response.text, 'html.parser')` turns a wall of HTML into a navigable tree — a **parse tree** of tags: parents, children and siblings, searchable by element, class or id. Two methods do 95% of the work:

- `soup.find('tag')` — the *first* match; ideal when the page has exactly one element of that kind.

- `soup.find_all('tag')` — a list of *all* matches; ideal for rows of a table or cards of a product.

The simplest selector is a tag name (`'h1'`, `'a'`, `'div'`), but classes (`class_='...`), ids and CSS-selector strings (`soup.select('div.card')`) give you surgical precision. The selector practice cell exists to drill exactly this: guess which of these the page uses, then confirm with a real query. Getting comfortable picking elements is the skill that makes every later extraction boring — in the good way.

Two notes make it stick. BeautifulSoup never downloads anything itself — it only parses the string you hand it, so `requests` and `soup` are always a pair. And searching is case-sensitive by default; when a site mixes `Div` and `div`, a forgiving selector or a quick pass saves a long debugging session.

PYTHON

```
from bs4 import BeautifulSoup

html = '''<html><body>
  <h1>Online Store</h1>
  <p class="price">100</p>
  <p class="price">250</p>
  <a href="/p/1">Phone</a>
  <a href="/p/2">Mouse</a>
</body></html>'''

soup = BeautifulSoup(html, "html.parser")

print("soup.title :", soup.title)      # first <title> (None here)
print("1st <p>      :", soup.find("p"))
print("all <p>      :", soup.find_all("p"))
print("all <a>      :", soup.find_all("a"))
```

PYTHON

```
from bs4 import BeautifulSoup

html = '''<div id="main">
  <div class="card"><h2>Phone X</h2><p class="price">15000</p></div>
  <div class="card"><h2>Phone Y</h2><p class="price">20000</p></div>
  <div class="card"><h2>Phone Z</h2><p class="price">25000</p></div>
</div>'''

soup = BeautifulSoup(html, "html.parser")

cards = soup.select("div.card")          # every card container
print("number of cards:", len(cards))

first = soup.select_one("div.card h2")  # first h2 inside a card
print("first card h2 :", first.text)

prices = soup.select("p.price")         # all price tags
print("prices        :", [p.text for p in prices])
```

7 · Extracting Data

Text vs attributes

`text` vs `get_text()`

Both return the inner text; `get_text(separator=" ", strip=True)` cleans it up (newlines/extra spaces) — very useful for real pages.

A complete extraction pattern

Collect **parallel lists** from **repeated containers**, then combine them.

PYTHON

```
element.text          # the visible text inside the element
element["href"]       # the value of the 'href' attribute
element.get("class")   # safely get an attribute (None if missing)
element.name          # the tag name
```

PYTHON

```
titles = [t.text for t in soup.select("h2")]
prices = [p.text for p in soup.select(".price")]
links  = [a.get("href") for a in soup.select("a[href]")]
```

Extract text and attributes

An element in the tree is a full object, and it exposes two very different things: `.get_text()` returns the **visible text** between the tags — what a reader sees on the page — while `.get('attr')` (or `element['href']`) returns a tag's **attribute value**, like the `href` of a link or the `src` of an image. Confusing the two is the classic beginner bug: the phrase that names the item is in the text, but the URL that *points* to it lives in an attribute.

Notice also the cleaning pattern: `strip()` on every extracted string, and skipping empties. Scraped text is guaranteed to contain stray whitespace and blank lines, so the extraction cell is where 80% of your future data-cleaning is actually decided. Extract clean or clean after — the first is far cheaper.

One patience to add: check *which* element you asked for before blaming the output. A `find_all` that returns one element holding the whole page means your selector landed on an ancestor; descend to the repeating block and the rows appear.

PYTHON

```

from bs4 import BeautifulSoup

html = '''<div class="card">
  <h2>Laptop</h2>
  <p class="price">₹55000</p>
  <a class="link" href="/laptop">view</a>
</div>
<div class="card">
  <h2>Tablet</h2>
  <p class="price">₹25000</p>
  <a class="link" href="/tablet">view</a>
</div>'''

soup = BeautifulSoup(html, "html.parser")

names = [h.text for h in soup.select("h2")]
prices = [p.text for p in soup.select(".price")]
links = [a.get("href") for a in soup.select("a.link")]

print("names :", names)
print("prices:", prices)
print("links :", links)

```

8 · From HTML to a DataFrame (mini-project)

Now we combine everything: parse a page, extract repeated fields, and build a pandas `DataFrame`.

The workflow

1. Get the HTML (from `requests` or an in-memory string)
2. `BeautifulSoup` parse it
3. `soup.select(...)` the repeated **container** elements
4. Loop each container and **pull out the fields** (safer than parallel lists when some fields might be missing)
5. Assemble a list of dicts → `pd.DataFrame(list_of_dicts)`

👉 **Prefer per-container dicts** over parallel lists: if one product lacks a price, the rows still line up correctly.

From soup to a DataFrame

Single values are easy; the moment scraped rows become **data** is when they are stacked into a pandas `DataFrame`. The pattern you will reuse everywhere: collect every row element with `find_all`, then per row pull each column with `.get_text().strip()`, collect everything into a list of dicts, and finally `pd.DataFrame(rows)`.

Once it is a DataFrame, every pandas skill you already own applies instantly — `shape`, `info()`, `value_counts()`, plotting, exporting to CSV or JSON. That hand-off, from HTML to a tidy table, is the exact boundary at which scraping ends and analysis begins. It is also where you silently decide data types: everything scraped arrives as `str`, so the subsequent `astype(int)` / `astype(float)` conversions are not optional polish — they are mandatory hygiene.

One detail worth its weight: build the rows list *and* check it before the DataFrame. A print of one sample row, or a `len(rows)` compared against an expected count, turns a silent zero-row scrape into a caught one — ten seconds of assertion now saves a spreadsheet of missing data later.

PYTHON

```
import pandas as pd
from bs4 import BeautifulSoup

page = '''<div class="card">
  <h2 class="name">Phone X</h2>
  <p class="price">₹15000</p>
  <span class="rating">4.5</span>
</div>
<div class="card">
  <h2 class="name">Phone Y</h2>
  <p class="price">₹20000</p>
  <span class="rating">4.2</span>
</div>
<div class="card">
  <h2 class="name">Phone Z</h2>
  <p class="price">₹25000</p>
  <span class="rating">4.8</span>
</div>'''

soup = BeautifulSoup(page, "html.parser")

rows = []
for card in soup.select("div.card"):
    rows.append({
        "name": card.select_one(".name").text,
        "price": card.select_one(".price").text,
        "rating": card.select_one(".rating").text,
    })

df_products = pd.DataFrame(rows)
df_products
```

9 • Pagination — scraping multiple pages

Real sites split results across many pages (`?page=1`, `?page=2`, ...). To scrape the whole set you **loop over pages** and **accumulate** the rows.

Pattern

- **Respect** `time.sleep(...)` — hammering a server will get you blocked / banned.

- Watch the site's own "how many pages" (often in a `pagination` element).

Below is an offline simulation of the same idea using generated pages in memory.

PYTHON

```
all_rows = []
for page_no in range(1, total_pages + 1):
    r = requests.get(url, params={"page": page_no}, headers=headers, timeout=10)
    soup = BeautifulSoup(r.text, "html.parser")
    all_rows.extend(extract(soup))    # combine with previous pages
    time.sleep(1)                   # be polite between requests
```

Pagination — the loop that scales

The hardest page to scrape is the one that hides its rows across many pages. **Pagination** is the practice of discovering the URL pattern (often `?page=2`, `?page=3` ...) and looping the exact same extraction over each page, appending each page's DataFrame to the one before. What you are really doing is turning a series of requests into *one logical dataset*, and that bookkeeping — resetting the results list per page, concatenating at the end, and keeping count — is the part that demands care.

Three habits keep pagination honest: check the next-page link instead of assuming a stride, stop at the last-page marker if the site provides one, and always sleep between pages. A pagination loop that never ends, or silently repeats page one, is the most common production scraper failure in the wild.

Count as you go. Keep an accumulator like `all_rows.extend(page_rows)` and print a running total per page, so a mid-run layout change shows up as a page whose contribution is suspiciously small.

PYTHON

```

import time
from bs4 import BeautifulSoup

# Simulate 3 pages of a catalog (each like a real scraped page)
def make_page(items):
    cards = "".join(
        f'<div class="card"><h2 class="name">{n}</h2><p class="price">{p}</p></div>'
        for n, p in items
    )
    return f"<html><body>{cards}</body></html>"

pages = {
    1: make_page([("Phone A", "100"), ("Phone B", "200")]),
    2: make_page([("Phone C", "300"), ("Phone D", "400")]),
    3: make_page([("Phone E", "500")]),
}

all_rows = []
for page_no in range(1, len(pages) + 1):
    soup = BeautifulSoup(pages[page_no], "html.parser")
    for card in soup.select("div.card"):
        all_rows.append({
            "page": page_no,
            "name": card.select_one(".name").text,
            "price": card.select_one(".price").text,
        })
    time.sleep(0.1)  # in real code: sleep 1-2s to be polite

import pandas as pd
df_catalog = pd.DataFrame(all_rows)
print("Total items gathered across pages:", len(df_catalog))
df_catalog

```

10 · Robust Scraping (production-ready)

A scraper that works once often breaks the next day. Build in these safety nets:

1. Always use `try / except`

One bad page should **not** kill the whole job — skip it and continue.

2. Handle status codes explicitly

- `429` / `403` → slow down, retry with backoff.
- Retry logic: try, wait, retry a few times, then give up.

3. Rate limiting

Add `time.sleep()` between requests. Be gentle.

4. Validate what you extracted

Check the row count / columns before saving — catch broken parsing early.

PYTHON

```
try:
    r = requests.get(url, timeout=10)
    r.raise_for_status()
    soup = BeautifulSoup(r.text, "html.parser")
except requests.exceptions.RequestException as e:
    print("Failed:", url, "->", e)
```

Failures are normal — plan for them

The retry cells teach the mindset that separates toy scrapers from working ones: **the network fails, so the code must not**. A `try/except` around the request catches the timeout or the stubborn server; retries re-attempt the call a fixed number of times with a growing delay (`1s`, `2s`, `3s`), so one hiccup does not kill a 200-page run; and the loop logs or counts skips so you can measure what went wrong afterwards.

The pattern generalises far beyond scraping: retry-with-backoff and bounded attempts is the same resilience you will later apply when hitting APIs and databases. Do not read it as a workaround for this one exercise — read it as the skeleton of every robust data-gathering loop you will ever write.

Two small rules keep the retry pattern safe. Bound the attempts — an infinite retry loop on a dead endpoint is a hang, not a recovery. And surface the final failure: after the last try, raise or record it, because a job that silently returns a partial dataset is worse than one that announced it failed.

PYTHON

```

import time
import requests
from bs4 import BeautifulSoup

def scrape_with_retry(url, headers, retries=3, delay=2):
    for attempt in range(retries):
        try:
            r = requests.get(url, headers=headers, timeout=10)
            if r.status_code == 200:
                return BeautifulSoup(r.text, "html.parser")
            elif r.status_code in (429, 503): # rate limited / busy
                time.sleep(delay * (attempt + 1)) # exponential backoff
            else:
                print("Skipping", url, "status", r.status_code)
                return None
        except requests.exceptions.RequestException as e:
            print("Attempt", attempt + 1, "failed:", e)
            time.sleep(delay)
    return None

# Used offline below: scrape_with_retry on an in-memory page
soup = scrape_with_retry.__wrapped__ if hasattr(scrape_with_retry, "__wrapped__") else None
print("Robust scraper defined. In real use call it with a live URL.")

```

PYTHON

```

from bs4 import BeautifulSoup

# Validating a scrape: always sanity-check the results
page = '''<div class="card"><h2 class="name">A</h2><p class="price">10</p></div>
<div class="card"><h2 class="name">B</h2><p class="price">20</p></div>'''
soup = BeautifulSoup(page, "html.parser")

rows = []
for card in soup.select("div.card"):
    name = card.select_one(".name")
    price = card.select_one(".price")
    if name and price: # skip incomplete cards
        rows.append({"name": name.text, "price": price.text})

print("extracted", len(rows), "rows")
expected = 2
assert len(rows) == expected, "Parsing changed – expected %d rows" % expected
print("Validation passed: row count matches expectation.")

```

11 · Selenium — scraping JavaScript-rendered pages

The problem

Many modern sites build their HTML with **JavaScript** *after* the page loads. `requests` only gets the empty shell — the data doesn't exist in the raw HTML. `Selenium` launches a **real browser** that runs the JS, so you can read the *rendered* page.

Install

Selenium Manager (bundled) auto-downloads the matching browser driver (ChromeDriver / GeckoDriver) — you no longer need to download one manually.

Basic workflow

By locators

BY	FIND BY	EXAMPLE
By.ID	id	<code>driver.find_element(By.ID, "email")</code>
By.NAME	name attr	<code>driver.find_element(By.NAME, "q")</code>
By.CLASS_NAME	class	<code>driver.find_element(By.CLASS_NAME, "price")</code>
By.CSS_SELECTOR	CSS	<code>driver.find_element(By.CSS_SELECTOR, "div.card h2")</code>
By.XPATH	xpath	<code>driver.find_element(By.XPATH, "//div[@class='card']")</code>

Waiting — explicit waits, never `time.sleep`

EXPECTED CONDITION	WAITS UNTIL
<code>presence_of_element_located</code>	element exists in DOM
<code>visibility_of_element_located</code>	element is visible
<code>element_to_be_clickable</code>	can be clicked
<code>presence_of_all_elements_located</code>	a list matches

⚠️ `time.sleep()` is fragile (too short = flaky; too long = slow). Use **explicit waits**.

```
PYTHON

pip install selenium
```

PYTHON

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

opts = Options()
opts.add_argument("--headless")          # invisible browser
driver = webdriver.Chrome(options=opts)
driver.get("https://example.com")

wait = WebDriverWait(driver, 10)
cards = wait.until(EC.presence_of_all_elements_located((By.CLASS_NAME, "card"))))

for card in cards:
    print(card.find_element(By.TAG_NAME, "h2").text)

driver.quit()

```

Selenium — scraping what a browser renders

Static HTML covers many pages, but modern sites often build their content with JavaScript after the page loads — the HTML you `requests.get` is then an empty shell. **Selenium** solves this by driving a real browser: it opens Chrome, executes the scripts, and lets you read the result. `webdriver.Chrome()` launches it, `driver.get(url)` navigates, and `driver.find_element(By.CSS_SELECTOR, ...)` locates content exactly as BeautifulSoup did.

The critical discipline is **waiting**: if the script runs in half a second but you ask for the content immediately, the element does not exist yet and Selenium raises. Explicit waits or a fixed short `time.sleep` after navigation is not optional — it is the entire difference between a Selenium script that works and one that fails at random. Browsers are heavy, so also get in the habit of `driver.quit()` when the session is done.

Keep the browser a tool, not a toy: run it headless when you need speed, and reuse one `driver` across a batch of pages instead of relaunching Chrome per URL, else startup cost swamps your wall-clock time.

PYTHON

```
# --- Selenium setup (ILLUSTRATION) ---
# Selenium is not installed in this notebook's environment, and launching a real
# browser here would be heavy – so the code is fully commented.
# Uncomment and run on a machine with `pip install selenium` + a browser to try it.
#
# from selenium import webdriver
# from selenium.webdriver.common.by import By
# from selenium.webdriver.chrome.options import Options
# from selenium.webdriver.support.ui import WebDriverWait
# from selenium.webdriver.support import expected_conditions as EC
#
# opts = Options()
# opts.add_argument("--headless")
# driver = webdriver.Chrome(options=opts)
# driver.get("https://quotes.toscrape.com/js/")
#
# wait = WebDriverWait(driver, 10)
# quotes = wait.until(EC.presence_of_all_elements_located((By.CLASS_NAME, "quote")))
#
# for q in quotes:
#     text = q.find_element(By.CLASS_NAME, "text").text
#     author = q.find_element(By.CLASS_NAME, "author").text
#     print(author, "->", text)
#
# driver.quit()
print("Selenium demo is shown as commented code (requires install + a browser).")
```

12 · Ethics & Legality

Scraping is powerful — use it responsibly.

Check before you scrape

- **Terms of Service** — does the site forbid automated access?
- `robots.txt` — `https://site.com/robots.txt` lists disallowed paths.
- `/robots.txt` is the site's instructions to crawlers ("user-agent: * /no-scrape").

Golden rules

1. **Prefer an official API** when one exists.
2. **Be slow & polite** — respect rate limits, add delays.
3. **Identify yourself** — set a real `User-Agent` / contact info.
4. **Don't bypass** logins, paywalls, CAPTCHAs, or access controls.
5. **Don't store personal data** you have no right to hold (privacy law).
6. **Respect copyright** — don't republish content without permission.
7. **Cache your results** — scrape once, don't re-fetch.

⚠ This is general guidance, not legal advice. Laws differ by country (GDPR in Europe, etc.). When in doubt, get permission.

PYTHON

```
# A minimal robots.txt parser pattern (for when you scrape at scale)
def check_robots(url):
    return ("Always read", url + "/robots.txt",
           "before writing an automated crawler")

print("robots.txt is the site's crawler rulebook:")
print(" -", "https://books.toscrape.com/robots.txt")
print(" -", "https://quotes.toscrape.com/robots.txt")
```

13 · Glossary & Cheatsheet

Key terms

TERM	MEANING
HTTP	protocol for fetching web resources
GET / POST	request methods (fetch / send)
Status code	3-digit response result (200, 404...)
User-Agent	string identifying your client
Header	metadata sent with a request/response
Session	persistent connection + cookies
DOM	the parsed tree of a document
Element	one tag + content + attrs
Attribute	<code>name="value"</code> inside a tag
CSS selector	pattern to match elements
Parser	software that reads HTML into a tree
robots.txt	site's crawl instructions
Rate limiting	controlling request frequency

requests cheatsheet

BeautifulSoup cheatsheet

PYTHON

```
# Final combined cheatsheet as runnable reminders
from bs4 import BeautifulSoup
import pandas as pd

# 1) parse
soup = BeautifulSoup("<p class='x'>hi</p>", "html.parser")
# 2) find
el = soup.select_one("p.x")      # -> <p class='x'>hi</p>
# 3) extract
print("text:", el.text, "| class:", el.get("class"))

# 4) rows -> DataFrame (the universal final step)
data = [{"item": "A", "price": 10}, {"item": "B", "price": 20}]
df = pd.DataFrame(data)
df
```

PYTHON

```
requests.get(url, params=..., headers=..., timeout=...)
requests.post(url, data=..., json=...)
session = requests.Session()
r.raise_for_status()
r.text / r.status_code / r.headers / r.url
```

14 · Practice Problems

Try these (in the notebook or on your own):

- parse** the HTML below, then
 - count the number of `<article>` elements,
 - print the text of every `<h2>`.
- extract** each article into a dict `{"title", "author", "summary"}` and build a `DataFrame`.
- filter** — print only articles whose author is "Alice".
- links** — collect every `href` on the page.
- (online)** scrape `https://quotes.toscrape.com/` — the quote text + author — into a `DataFrame`, respecting `robots.txt` and adding a delay.
- (online, harder)** `https://books.toscrape.com/` — scrape all book titles and prices, handling **pagination** (pages 1..50).

Practise against the problems

The final cells are deliberate practice, not more demonstration: tables to scrape, repeated blocks to clean, and additional fields to pull from live sample sites. Work each one yourself before peeking at the next cell — design the selector and the loop, run it, then compare. The mechanical skills here (inspect the page, check status, pick selectors, extract and strip, stack into a `DataFrame`, wrap it in a delay) are the same recipe you will compile into a five-minute pipeline at work. Speed comes from having done this notebook's exercises seriously, in order, more than once.

Treat each problem like a mini task and read it the way an interviewer would: restate the required output (rows, columns, file), pick the shortest honest route, then run. If a selector wobbles, open the browser's developer tools and look at the actual element — inspecting beats guessing. Keep a scratch notebook for dead ends so every mistake becomes a reusable lesson. And finish each problem by exporting a CSV, because a scraper that ends in a file is one you could run at a job tomorrow.

PYTHON

```
from bs4 import BeautifulSoup
import pandas as pd

blog = '''<html><body>
<article>
  <h2>First Post</h2>
  <p class="author">Alice</p>
  <p class="summary">About data science.</p>
</article>
<article>
  <h2>Second Post</h2>
  <p class="author">Bob</p>
  <p class="summary">About web scraping.</p>
</article>
<article>
  <h2>Third Post</h2>
  <p class="author">Alice</p>
  <p class="summary">About pandas.</p>
</article>
</body></html>'''

soup = BeautifulSoup(blog, "html.parser")

# --- Exercise directions (fill in) ---
# 1) Count articles:
n_articles = len(soup.select("article"))
print("Number of articles:", n_articles)

# ...

# 3) Filter by Alice:
print("\nOnly Alice's posts:")
print(df_blog[df_blog["author"] == "Alice"])
```

Wrap-Up

You now know the **full scraping pipeline**: And you know when to level up to **Selenium** for JavaScript-rendered sites, plus how to be **fast, reliable, and ethical**.

Next steps: try quotes.toscrape.com and books.toscrape.com for risk-free practice (they exist explicitly for learning scraping), then a real site you have permission to use.

PYTHON

```

requests (download HTML)
↓
BeautifulSoup (parse into a tree)
↓
soup.select(...) (find the repeated elements)
↓
extract text / attributes
↓
pandas DataFrame (clean, structured data)
↓
save / analyze

```

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- data acquisition and provenance
- data assessment, profiling and cleaning
- exploratory analysis workflow
- ETL structure and reproducibility

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

WHAT IS DATA ANALYSIS?

```

Raw Data (unstructured, messy numbers)
↓ (clean, organise)
Information (structured, meaningful values)
↓ (analyze, compare)
Insight (what it means, why it matters)
↓ (act)
Decision (business action / recommendation)

```

PYTHON

STEP 3 — DATA TRANSFORMATION

```
df["Revenue"] = df["Price"] * df["Quantity"]
df["AOV"] = df["Revenue"] / df["Orders"]
```

PYTHON

STEP 4 — DATA EXPLORATION (EDA)

```
df.shape          # (rows, columns)
df.info()         # dtypes + non-null counts
df.describe()     # numeric stats
df.isna().sum()   # missing values
df.groupby("...").mean()
df.corr()         # correlation
```

PYTHON

2. OPENING A LOCAL CSV FILE

```
df = pd.read_csv('Datasets/aug_train.csv')
df
```

PYTHON

3. OPENING A CSV FILE FROM A URL

```
import requests
from io import StringIO

url = "https://raw.githubusercontent.com/cs109/2014_data/master/countries.csv"
headers = {"User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:66.0) Gecko/20100101 Firefox/66.0"}
req = requests.get(url, headers=headers)
data = StringIO(req.text)

pd.read_csv(data)
```

PYTHON

3. OPENING A CSV FILE FROM A URL

```
# Documentation of read_csv
pd.read_csv??
```

PYTHON

13. HANDLING DATES

```
def rename(name):
    if name == "Royal Challengers Bangalore":
        return "RCB"
    else:
        return name
```

PYTHON

16. LOADING A HUGE DATASET IN CHUNKS

```
for chunks in dfs:  
    print(chunks.shape)
```

Glossary

Format

it in? (CSV, JSON, SQL.

patient_id

the unique identifier for each patient in the Master Patient Index (i.e. patient database) of the pharmaceutical company that is producing Auralin

assigned_sex

the assigned sex of each patient at birth (male or female)

given_name

the given name (i.e. first name) of each patient

surname

the surname (i.e. last name) of each patient

address

the main address for each patient

city

the corresponding city for the main address of each patient

state

the corresponding state for the main address of each patient

zip_code

the corresponding zip code for the main address of each patient

country

the corresponding country for the main address of each patient (all United states for this clinical trial)

contact

phone number and email information for each patient

birthdate

the date of birth of each patient (month/day/year). The inclusion criteria for this clinical trial is age ≥ 18 (there is no maximum age because diabetes is a growing problem among the elderly population)

weight

the weight of each patient in pounds (lbs)

height

the height of each patient in inches (in)

bmi

the Body Mass Index (BMI) of each patient. BMI is a simple calculation using a person's height and weight. The formula is $BMI = \frac{kg}{m^2}$ where kg is a person's weight in kilograms and m2 is their height in metres squared. A BMI of 25.0 or more is overweight, while the healthy range is 18.5 to 24.9. The inclusion criteria for this clinical trial is $16 \leq BMI \leq 38$.

auralin

the baseline median daily dose of insulin from the week prior to switching to Auralin (the number before the dash) and the ending median daily dose of insulin at the end of the 24 weeks of treatment measured over the 24th week of treatment (the number after the dash). Both are measured in units (shortform 'u'), which is the international unit of measurement and the standard measurement for insulin.

novodra

same as above, except for patients that continued treatment with Novodra

hba1c_start

the patient's HbA1c level at the beginning of the first week of treatment. HbA1c stands for Hemoglobin A1c. The HbA1c test measures what the average blood sugar has been over the past three months. It is thus a powerful way to get an overall sense of how well diabetes has been controlled. Everyone with diabetes should have this test 2 to 4 times per year. Measured in %.

hba1c_end

the patient's HbA1c level at the end of the last week of treatment

hba1c_change

the change in the patient's HbA1c level from the start of treatment to the end, i.e., hba1c_start - hba1c_end. For Auralin to be deemed effective, it must be "noninferior" to Novodra, the current standard for insulin. This "noninferiority" is statistically defined as the upper bound of the 95% confidence interval being less than 0.4% for the difference between the mean HbA1c changes for Novodra and Auralin (i.e. Novodra minus Auralin).

adverse_reaction

the adverse reaction reported by the patient

quality_issue

a data error: missing values, duplicates, invalid entries (Hormonal as a severity), inaccurate values.

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

- ☐ What is Data Analysis?
- ☐ The Complete Data Analysis Process — Overview
- ☐ Step 0 — Define the Problem (before touching any data)
- ☐ Step 1 — Data Gathering
- ☐ Step 2 — Data Cleaning
- ☐ Step 3 — Data Transformation
- ☐ Step 4 — Data Exploration (EDA)
- ☐ Step 5 — Data Modeling
- ☐ Step 6 — Data Visualisation & Communication
- ☐ The Whole Picture — Summary
- ☐ DATA GATHERING
- ☐ Tools of the Trade

- ☐ Reading a Local CSV File
- ☐ 1. Importing pandas
- ☐ 2. Opening a local CSV file
- ☐ 3. Opening a CSV file from a URL
- ☐ 4. The `sep` parameter
- ☐ 5. The `index_col` parameter
- ☐ 6. The `header` parameter
- ☐ 7. The `usecols` parameter
- ☐ 8. The `squeeze` parameter
- ☐ 9. The `skiprows` / `nrows` parameters